

A short Introduction to Deep Learning

Carsten Rother



UNIVERSITÄT
HEIDELBERG
ZUKUNFT
SEIT 1386



European Research Council
Established by the European Commission



DAAD funded Master scholarships in Learning and Intelligent Systems

Eligible students: Bachelor or early Master students that applied for or are in a MSc Program and are interested in Learning and Intelligent Systems

Duration: 1 year (extension possible for one more year)

Funding: Scholarship of 992 EUR/month plus relocation support

Benefit: Be a part of ELIZA networking program across German sites & industry partners during your Master studies

Applications need to be submitted as one pdf (max. 4 MB) containing:
Short CV; motivation letter incl. your areas of interest within AI;
relevant undergraduate or graduate transcript, admission letter from ELIZA institution you applied to (if applicable),
recommendation letters are welcome

Please apply here (deadline 25th October 2025):

For more info please visit: www.eliza.school

Questions: eliza@iwr.uni-heidelberg.de



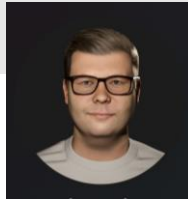
Zuse Schools
National Zuse Schools of Excellence
in Artificial Intelligence



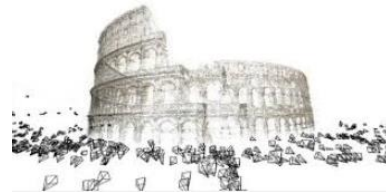
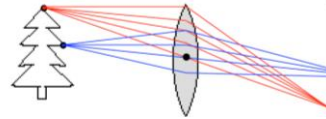
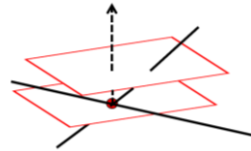
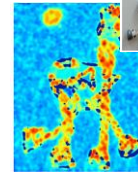
SPONSORED BY THE

Federal Ministry
of Education
and Research

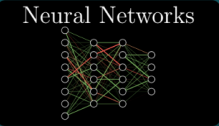
Lecture Content



- Introduction (~1.5h):
- Introduction to Deep Machine Learning (~2h):
- Basics of Image Processing (~2.5h)
- Basics of Projective Geometry (~1h)
- What is a Camera? (~1h)
- 3D Reconstruction (~5.5h)
- New View Synthesis (~1h)
- Differentiable RANSAC (~1h)
- Object Recognition (~2h)
- Image Generation (~2h)
- Training Data Generation (~1.5h)



Recommendations



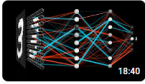
Neural networks

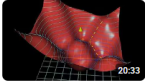
3Blue1Brown • Course
9 videos • Last updated on Sep 26, 2025

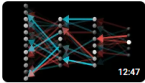
🔖 ⬅️ ⋮

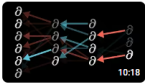
▶ Play 💬 Comments

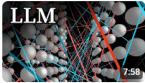
Learn the basics of neural networks and backpropagation, one of the most important algorithms for the modern world.

- 

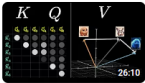
But what is a neural network? | Deep learning chapter 1
3Blue1Brown • 21M views • 8 years ago
18:40
- 

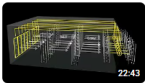
Gradient descent, how neural networks learn | Deep Learning Chapter 2
3Blue1Brown • 8.2M views • 8 years ago
20:33
- 

Backpropagation, intuitively | Deep Learning Chapter 3
3Blue1Brown • 5.5M views • 7 years ago
12:47
- 

Backpropagation calculus | Deep Learning Chapter 4
3Blue1Brown • 3.4M views • 7 years ago
10:18
- 

Large Language Models explained briefly
3Blue1Brown • 4M views • 11 months ago
7:58
- 

Transformers, the tech behind LLMs | Deep Learning Chapter 5
3Blue1Brown • 8M views • 1 year ago
27:14
- 

Attention in transformers, step-by-step | Deep Learning Chapter 6
3Blue1Brown • 3.2M views • 1 year ago
26:10
- 

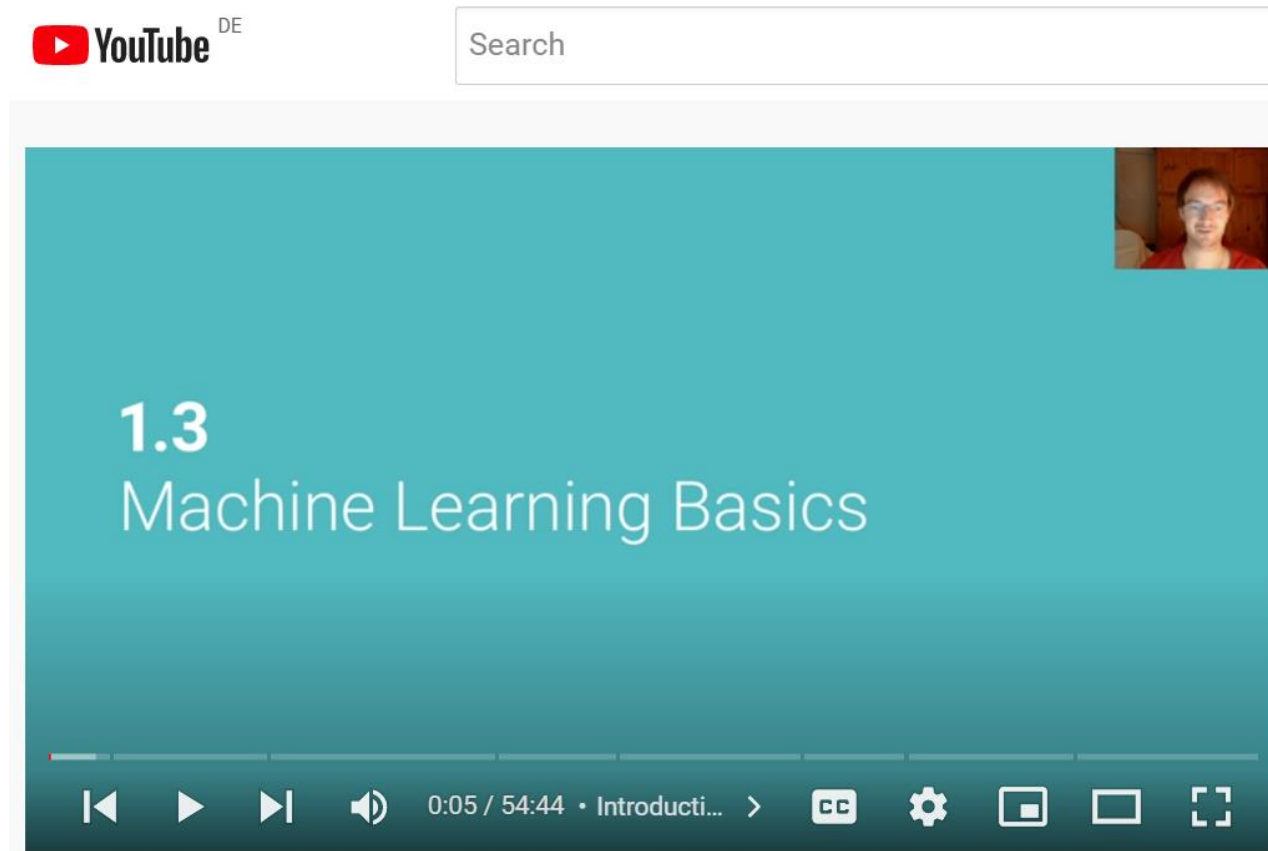
How might LLMs store facts | Deep Learning Chapter 7
3Blue1Brown • 1.6M views • 1 year ago
22:43
- 

But how do AI images and videos actually work? | Guest video by Welch Labs
3Blue1Brown and Welch Labs • 1.2M views • 2 months ago
37:20

https://www.youtube.com/playlist?list=PLZHQObOWTQDNU6R1_67000Dx_ZCJB-3pi

- Good overview: <https://slds-lmu.github.io/i2ml/>
- You can find the ML Playground here:
<https://playground.tensorflow.org/#activation=tanh&batchSize=10&dataset=circle®Dataset=reg-plane&learningRate=0.03®ularizationRate=0&noise=0&networkShape=4,2&seed=0.11795&showTestData=false&discretize=false&percTrainData=50&x=true&y=true&xTimesY=false&xSquared=false&ySquared=false&cosX=false&sinX=false&cosY=false&sinY=false&collectStats=false&problem=classification&initZero=false&hideText=false>

Recommendations



Deep Learning - Lecture 1.3 (Introduction: Machine Learning Basics)
From: Prof. Dr. Andreas Geiger, Uni Tübingen

<https://www.youtube.com/watch?v=UNbLjVEYvLQ&list=PL05umP7R6ij3NTWldtMbfbvX7Z-4WEXRqD&index=3>

- Types of Machine Learning Problems
- Decision Trees
- Deep Learning: Discriminative Models
 - Neural Networks
 - Convolutional Neural Networks
- Image Classification: ResNet
- Understanding Image Classification Networks
- Semantic Segmentation: U-Net
- Active and Semi-Supervised Learning

- Types of Machine Learning Problems
- Decision Trees
- Deep Learning: Discriminative Models
 - Neural Networks
 - Convolutional Neural Networks
- Image Classification: ResNet
- Understanding Image Classification Networks
- Semantic Segmentation: U-Net
- Active and Semi-Supervised Learning

Probability - Reminder

- A random variable is denoted with $x \in \{0, \dots, K\}$
- Discrete probability distribution: $p(x)$ satisfies $\sum_x p(x) = 1$
- Joint distribution of two random variables: $p(x, z)$
- Conditional distribution: $p(x|z)$
- Sum rule (marginal distribution): $p(z) = \sum_x p(x, z)$
- Product rule: $p(x, z) = p(z|x)p(x)$
- Bayes' rule: $p(x|z) = \frac{p(z|x)p(x)}{p(z)}$
- Usage: x – input variables, y – output variables, w, θ - parameters

Question

Given $x, z \in \{0,1\}$. Which answer is correct:

1)

	$z = 0$	$z = 1$
$x = 0$	0.6	0.2
$x = 1$	0.1	0.1

$p(x|z)$

2)

	$z = 0$	$z = 1$
$x = 0$	0.1	0.2
$x = 1$	0.9	0.8

$p(x|z)$

3) Both answers 1), 2)

4) I don't know

Machine Learning - Variants

Two Types of Models:

1. Discriminative Models: $y = f_w(x)$ or $p(y|x)$
2. Generative Models: $p(x)$, $p(x|y)$, or $p(y, x)$

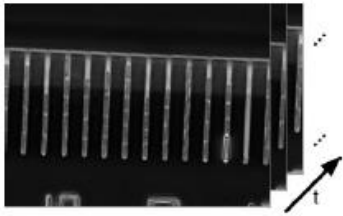
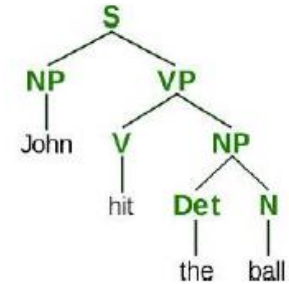
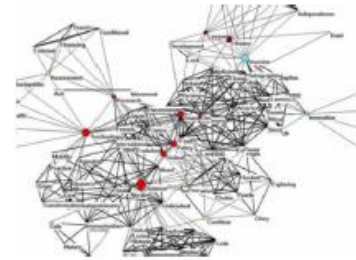
Variants of Machine Learning Models:

1. *Structured models versus un-structured models*
2. ...

Discriminative Machine Learning Models



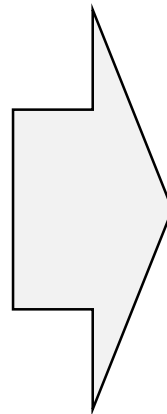
Database of Text



2D, 3D microscopy

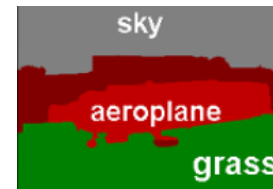
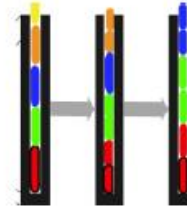


Images and videos



$$y = f_w(x)$$

$$p(y|x)$$



Images, Symbolic Data and Measurements

Semantics, Structure, Data Transfer

Structured Models versus un-structured Models

"Normal" Machine Learning:

$f : \mathcal{X} \rightarrow \mathbb{N}$ (classification)

$f : \mathcal{X} \rightarrow \mathbb{R}$ (regression)

Input: Image, text

Output: real number(s)

Jemand musste Josef K. verleumdet haben, denn ohne dass er etwas Böses getan hätte, wurde er eines Morgens verhaftet. »Wie ein Hund!« sagte er, es war, als sollte die Scham ihn überleben. Als Gregor Samsa eines Morgens aus unruhigen Träumen erwachte, fand er sich in seinem Bett zu einem ungeheueren Ungeziefer verwandelt. Und es war ihnen wie eine Bestätigung ihrer neuen Träume und guten Absichten, als am Ziele ihrer Fahrt die Tochter als erste sich erhob und ihren jungen Körper dehnte. »Es ist ein eigentümlicher Apparat«, sagte der Offizier zu dem Forschungsreisenden und überblickte mit einem gewissermaßen



Structured Models versus un-structured Models

"Normal" Machine Learning:

$f : \mathcal{X} \rightarrow \mathbb{N}$ (classification)

$f : \mathcal{X} \rightarrow \mathbb{R}$ (regression)

Input: Image, text

Output: real number(s)

Jemand musste Josef K. verleumdet haben, denn ohne dass er etwas Böses getan hätte, wurde er eines Morgens verhaftet. »Wie ein Hund!« sagte er, es war, als sollte die Scham ihn überleben. Als Gregor Samsa eines Morgens aus unruhigen Träumen erwachte, fand er sich in seinem Bett zu einem ungeheueren Ungeziefer verwandelt. Und es war ihnen wie eine Bestätigung ihrer neuen Träume und guten Absichten, als am Ziele ihrer Fahrt die Tochter erst sich erhob und ihren jungen Körper dehnte. »Es ist ein eigentümlicher Apparat«, sagte der Offizier zu dem Forschungsreisenden und überblickte mit einem gewissermaßen

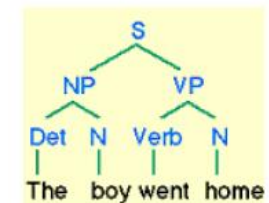
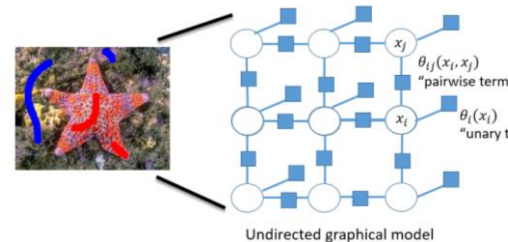


Structured Output Prediction:

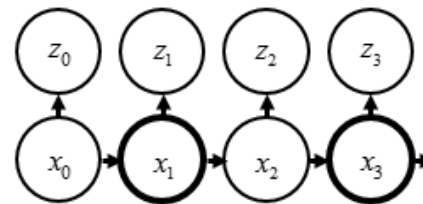
$f : \mathcal{X} \rightarrow \mathcal{X}$

Input: Image, text

Output: complex structure object
(labelling, parse tree)



Parse tree of a sentence



Markov Chain for tracking

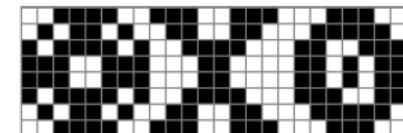


Image labelling

[Nowozin, Lampert '11]

Machine Learning - Variants

Two Types of Models:

1. Discriminative Models: $y = f_w(x)$ or $p(y|x)$
2. Generative Models: $p(x)$, $p(x|y)$, or $p(y, x)$

Variants of Machine Learning Models:

1. *Structured models versus un-structured models*
2. ...

Generative Machine Learning Models



Samples of Celebrities $p(x)$



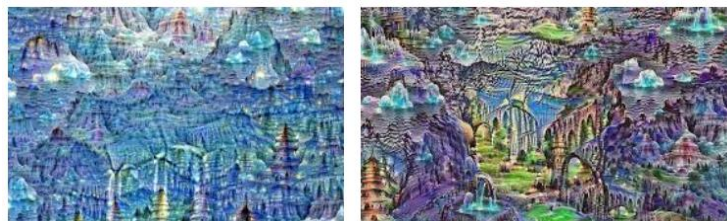
Samples of spiders $p(x)$



Samples of coloured images $p(x|y)$



Conditioning y



Dreaming Networks $p(x)$

Richard Feynman: **What I cannot create, I do not understand”.**

(theoretical physicist who received the Nobel prize in 1965 for his work developing quantum electrodynamics)

We model: $p(x)$, $p(x|y)$ or $p(x, y)$

Machine Learning - Variants

Two Types of Models:

1. Discriminative Models: $y = f_w(x)$ or $p(y|x)$
2. Generative Models: $p(x)$, $p(x|y)$, or $p(y, x)$

Variants of Machine Learning Models:

1. *Structured models versus un-structured models*
2. ...

Machine Learning - Variants

Two Types of Models:

1. Discriminative Models: $y = f_w(x)$ or $p(y|x)$
2. Generative Models: $p(x)$, $p(x|y)$, or $p(y, x)$

Variants of Machine Learning Models:

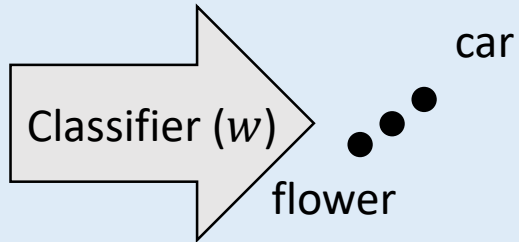
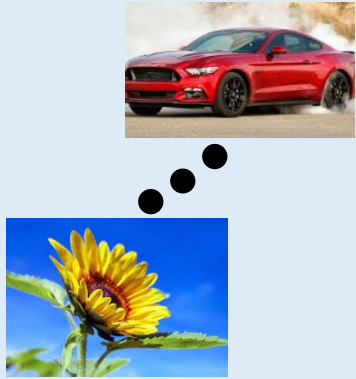
1. *Structured models versus un-structured models*
2. ...

Three Classes of Machine Learning Problems:

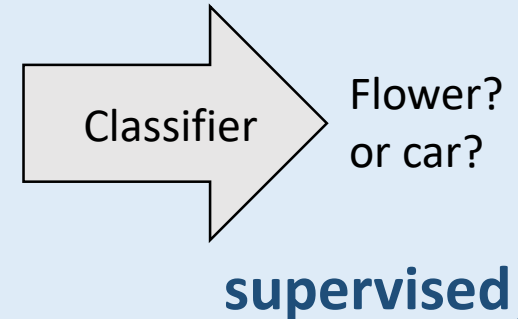
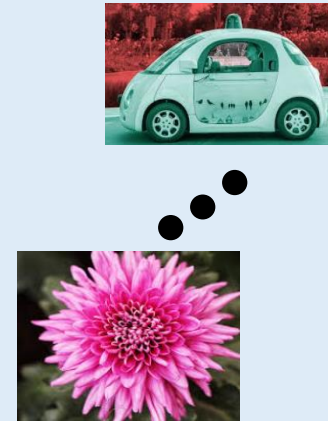
1. **Supervised learning:** fully labelled (annotated) dataset
 2. **Unsupervised learning:** no annotations at all (e.g. clustering)
 3. **Reinforcement learning:** there is a reward for correct action
- (we focus on supervised learning)*

Supervised versus Unsupervised Learning

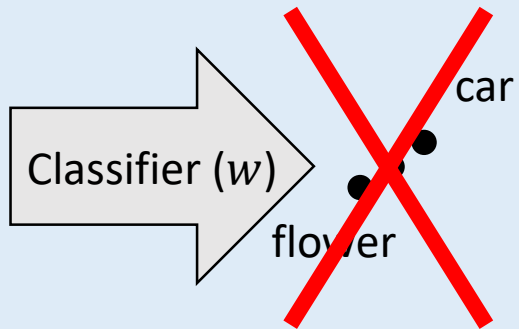
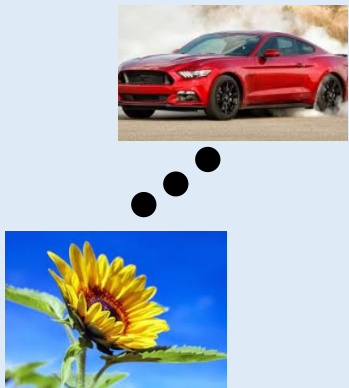
Training Time



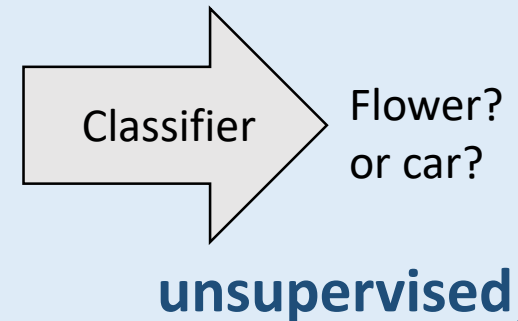
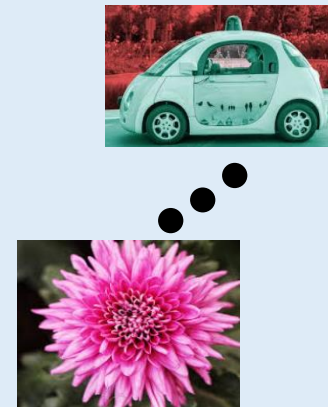
Test Time



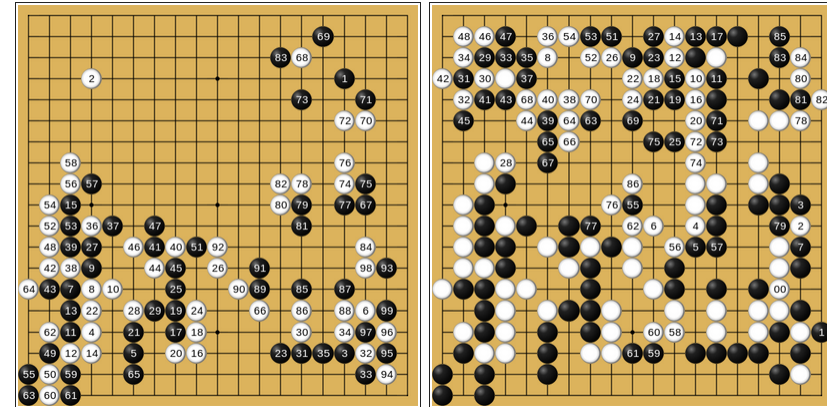
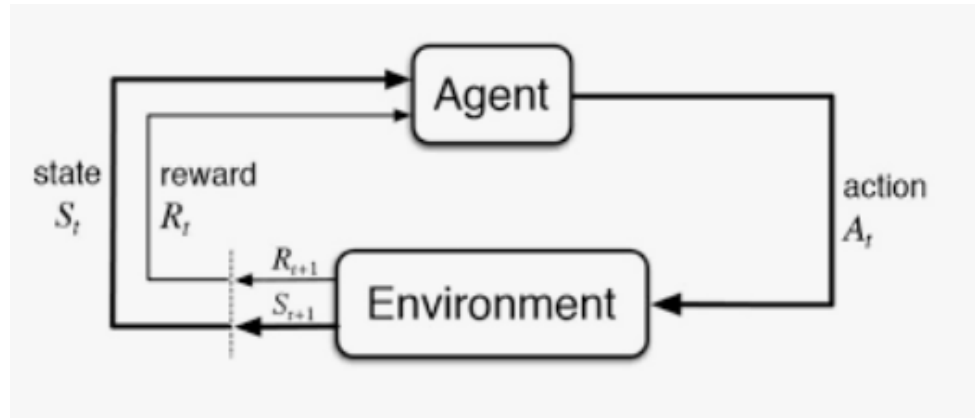
Training Time



Test Time



Reinforcement Learning / Agentic AI

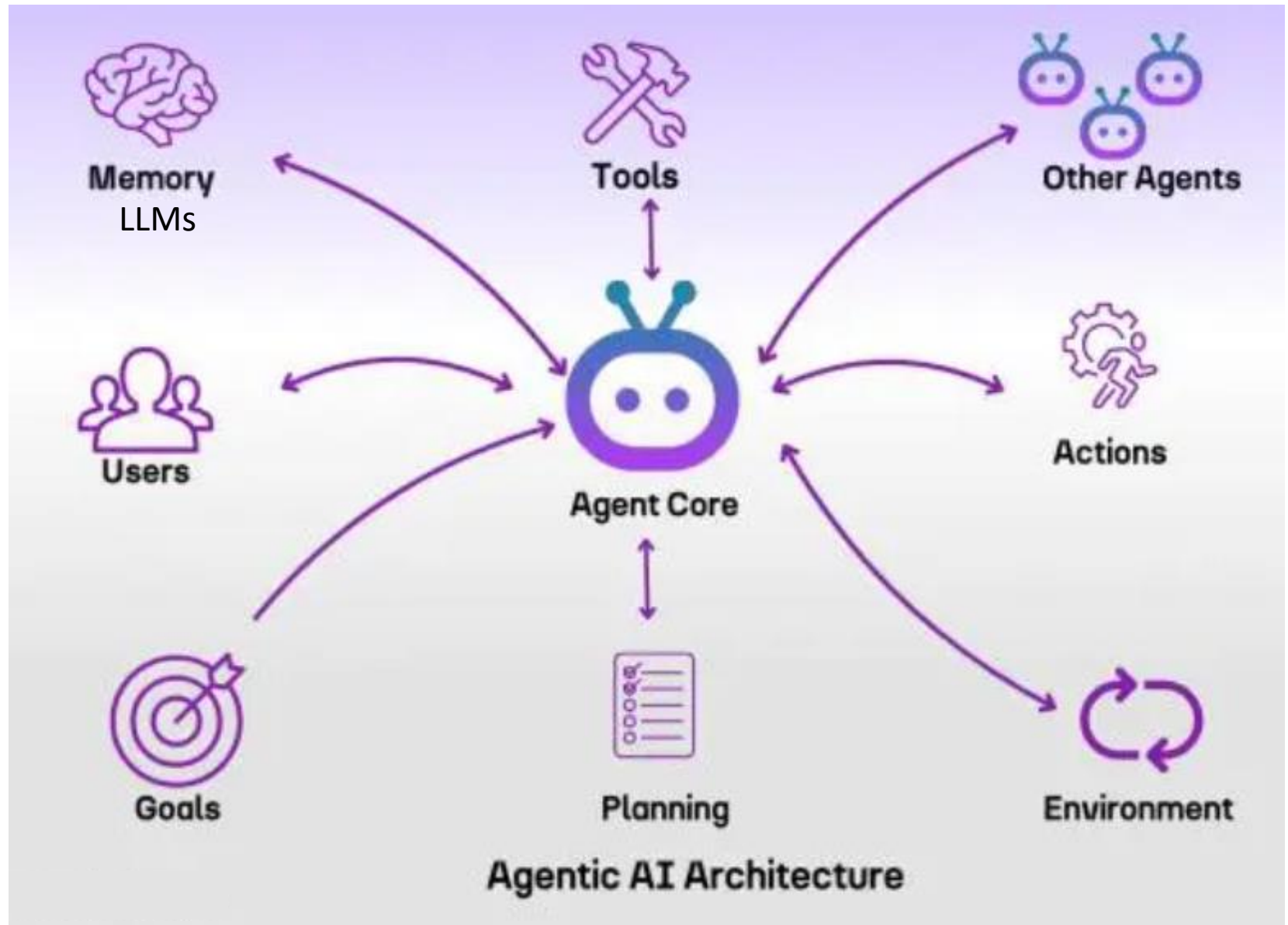


AlphaGo

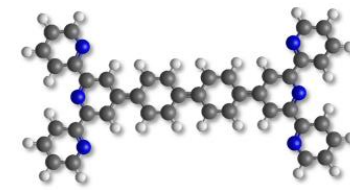
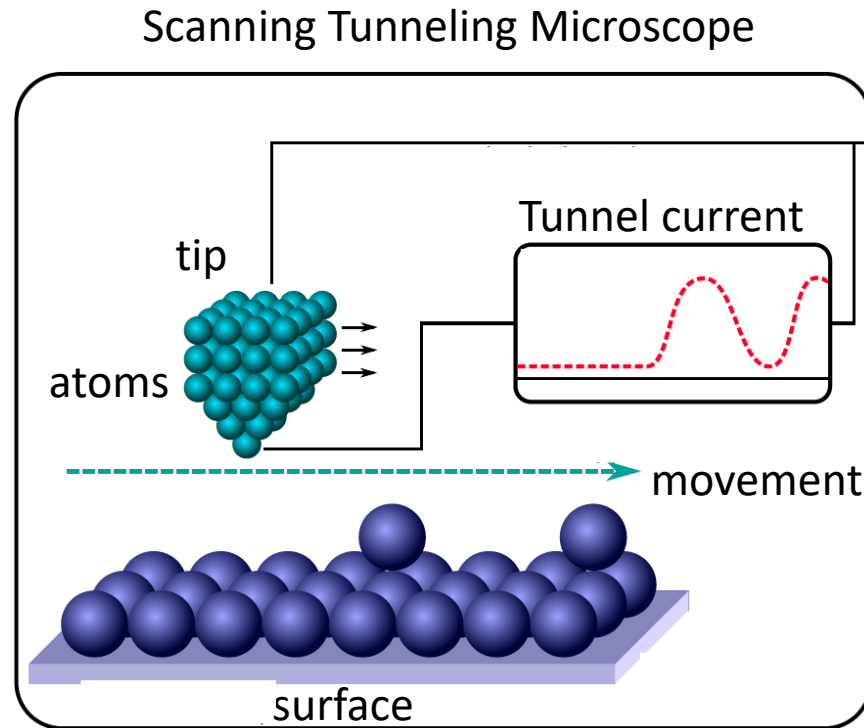


Playing Atari Games [Mnih 2013]

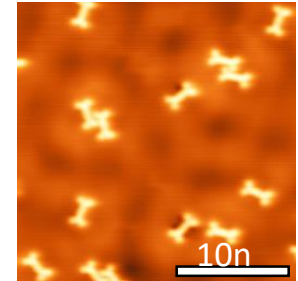
Reinforcement Learning / Agentic AI



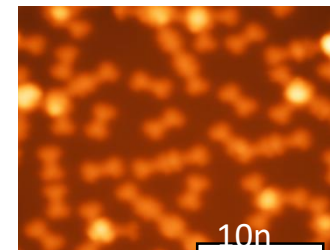
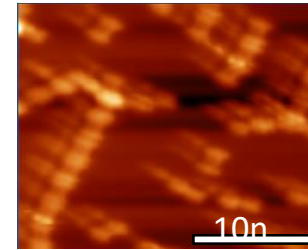
Reinforcement Learning and Microscopy



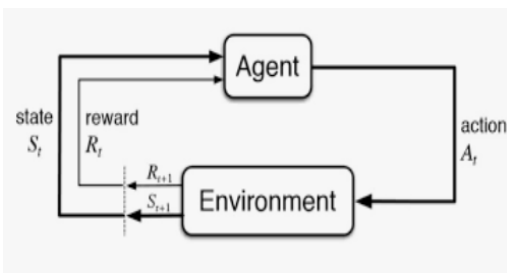
TPPT molecule



Measurement with **good** tip



Measurement with **bad** tips



- Preparation currently done by researchers (**Trial and error**)
- We do Reinforcement Learning approach
 - Environment/State: Measurement (image)
 - Actions: % force of crash, % Voltage (done by CNN)
 - Reward: A CNN that evaluates the Measurement

[Krull, P. Hirsch, C. Rother, A. Schiffrin, C. Krull. "Artificial-intelligence-driven scanning probe microscopy", Communications Physics volume 3, 54 (2020), 19 March 2020]

Machine Learning - Variants

Two Types of Models:

1. Discriminative Models: $y = f_w(x)$ or $p(y|x)$
2. Generative Models: $p(x)$, $p(x|y)$, or $p(y, x)$

Variants of Machine Learning Models:

1. *Structured models versus not structured models*
2. ...

Three Classes of Machine Learning Problems:

1. **Supervised learning:** fully labelled (annotated) dataset
 2. **Unsupervised learning:** no annotations at all (e.g. clustering)
 3. **Reinforcement learning:** there is a reward for correct action
- (we focus on supervised learning)*

Machine Learning - Variants

Two Types of Models:

1. Discriminative Models: $y = f_w(x)$ or $p(y|x)$
2. Generative Models: $p(x)$, $p(x|y)$, or $p(y, x)$

Variants of Machine Learning Models:

1. *Structured models versus not structured models*
2. ...

Three Classes of Machine Learning Problems:

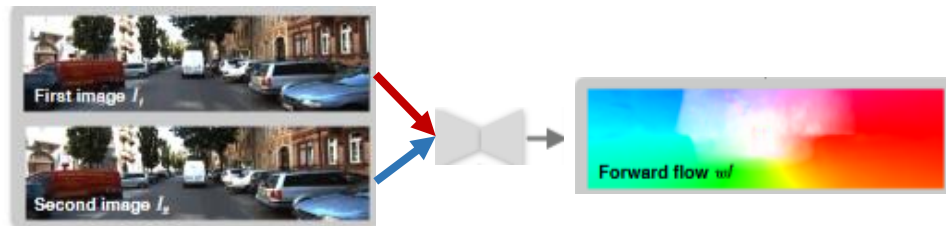
1. **Supervised learning**: fully labelled (annotated) dataset
2. **Unsupervised learning**: no annotations at all (e.g. clustering)
3. **Reinforcement learning**: there is a reward for correct action
(*we focus on supervised learning*)

Lost of variants of Machine Learning Problems:

1. On-line learning: Training samples are not available at once
2. Continual Learning: incrementally learning/adapting your model
3. **Self-Supervised**: Use the data itself for learning
4. **Domain adaption**: Train in Domain A and test in Domain B
5. **Pre-Text Tasks**: Train for Task A and fine-tune for task B
6. **Active learning**: What unlabeled data should I label next?
7. **Semi-Supervised learning**: Dataset is partially labelled
8.

Self-supervised Learning

Test:

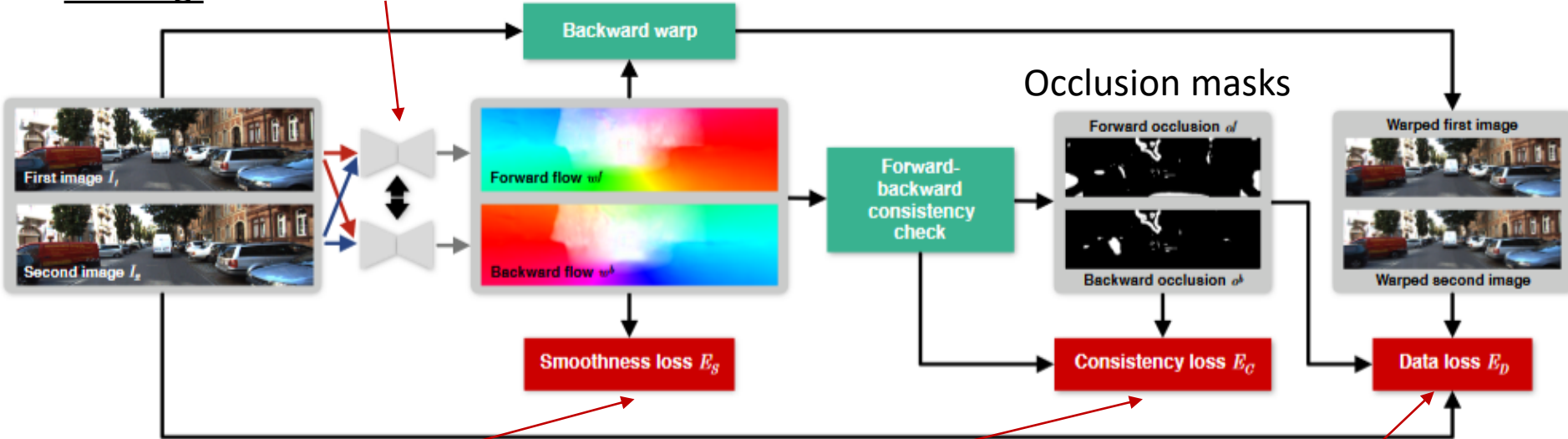


[UnFlow: Unsupervised Learning of Optical Flow with a Bidirectional Census Loss, Meister 2017]

Self-supervised Learning

Training:

Auto-encoder
architecture

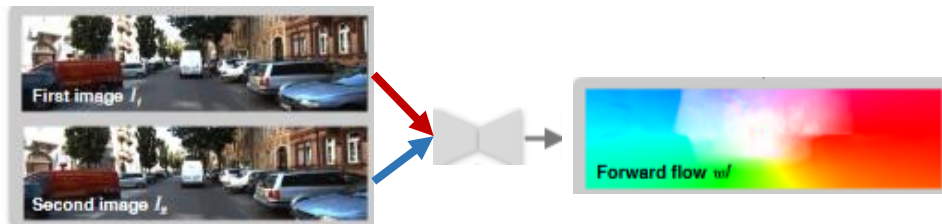


Smoothness of
both flow fields

For non-occluded pixels
the forward-backward
check should be small

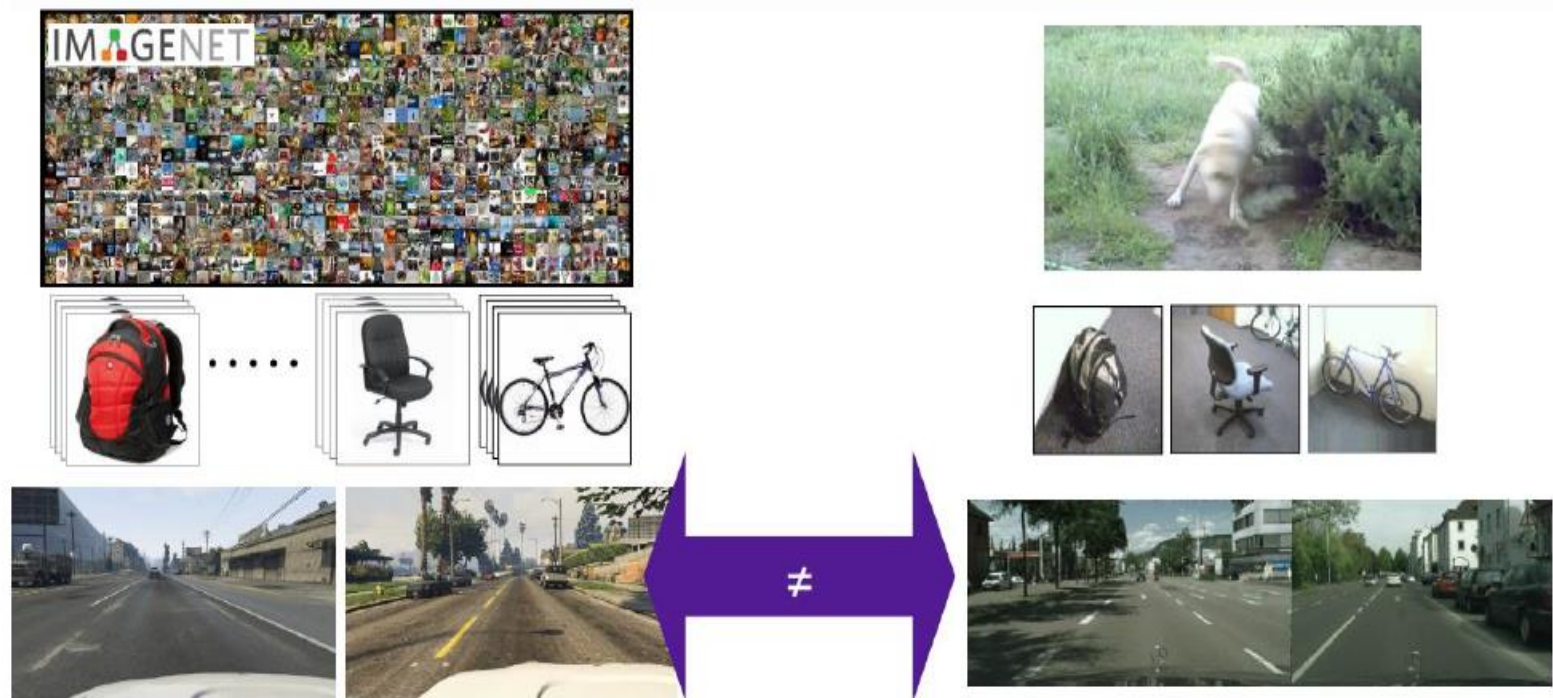
Photo-consistency check
for non-occluded pixels

Test:



[UnFlow: Unsupervised Learning of Optical Flow with a Bidirectional Census Loss, Meister 2017]

Domain adaption



Source Domain $\sim P_S(X_S, Y_S)$

lots of **labeled** data

Target Domain $\sim P_T(X_T, Y_T)$

unlabeled or limited labels

(we will look at this in last lectures)

- Types of Machine Learning Problems
- Decision Trees
- Deep Learning: Discriminative Models
 - Neural Networks
 - Convolutional Neural Networks
- Image Classification: ResNet
- Understanding Image Classification Networks
- Semantic Segmentation: U-Net
- Active and Semi-Supervised Learning

Mushroom example



eatable



not
eatable



eatable



not
eatable



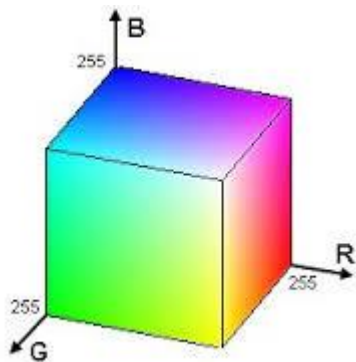
not
eatable



not
eatable

Measure attributes:

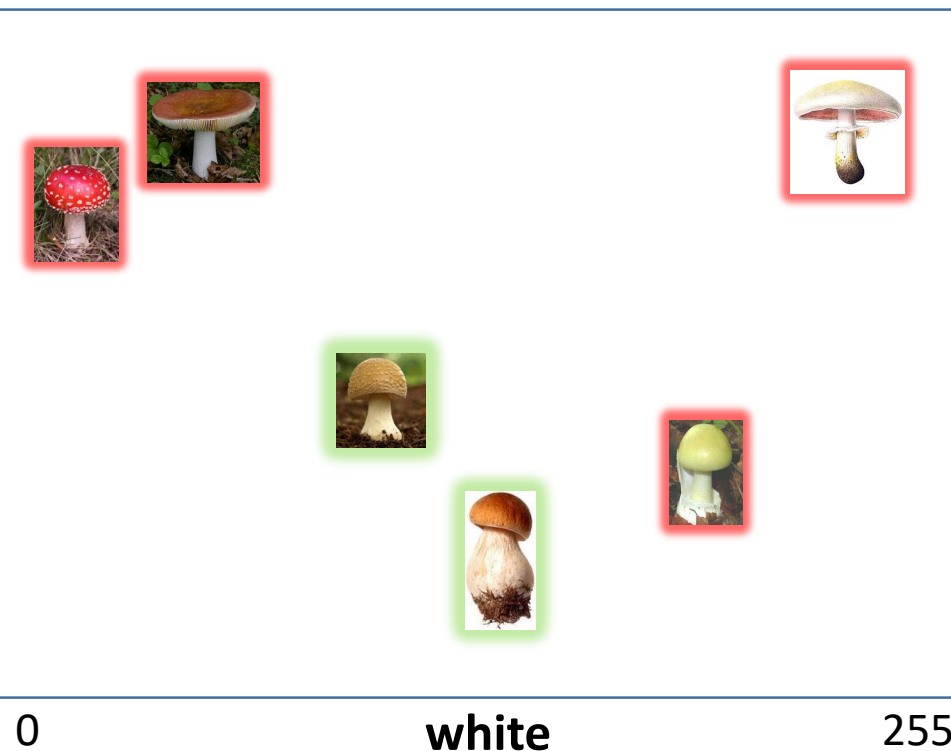
- Size in centimeters
- Color: Average “whiteness” of the mushroom (i.e. value along diagonal in RGB cube)



15cm

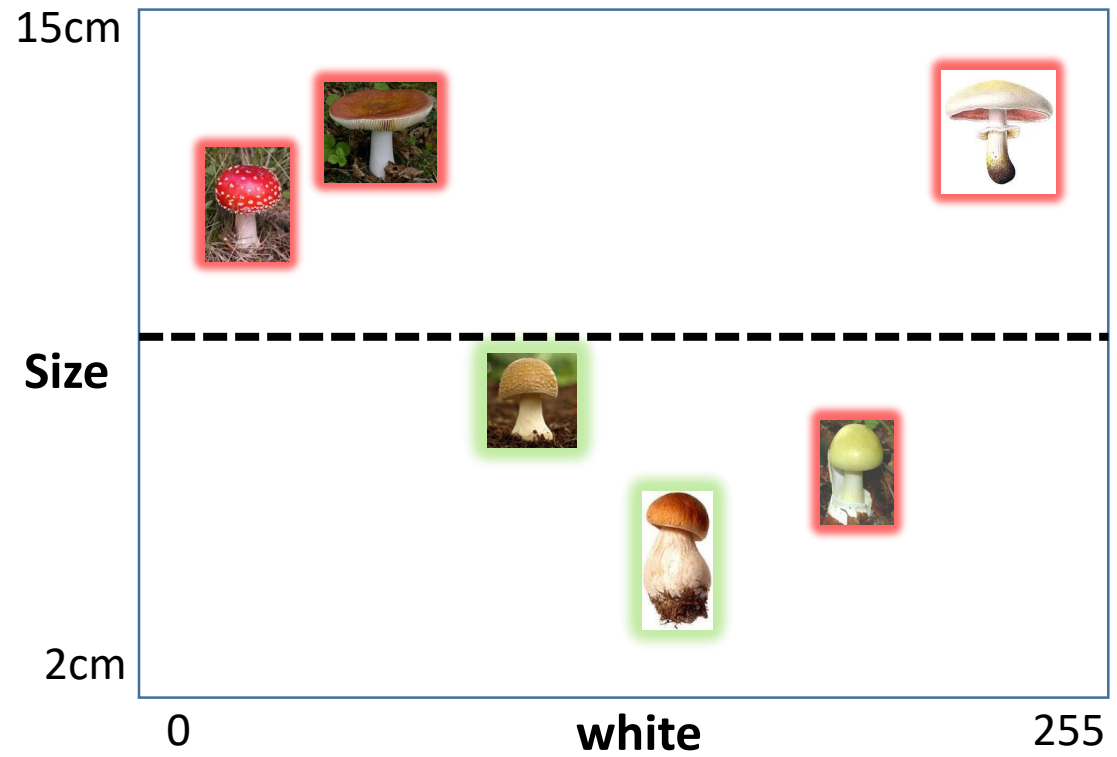
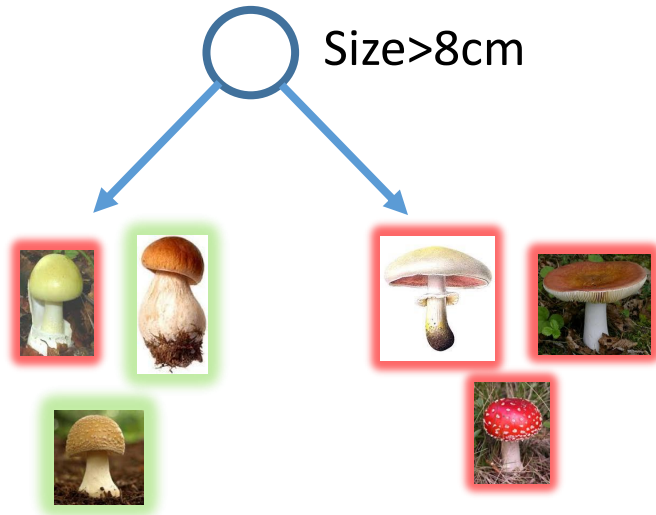
Size

2cm

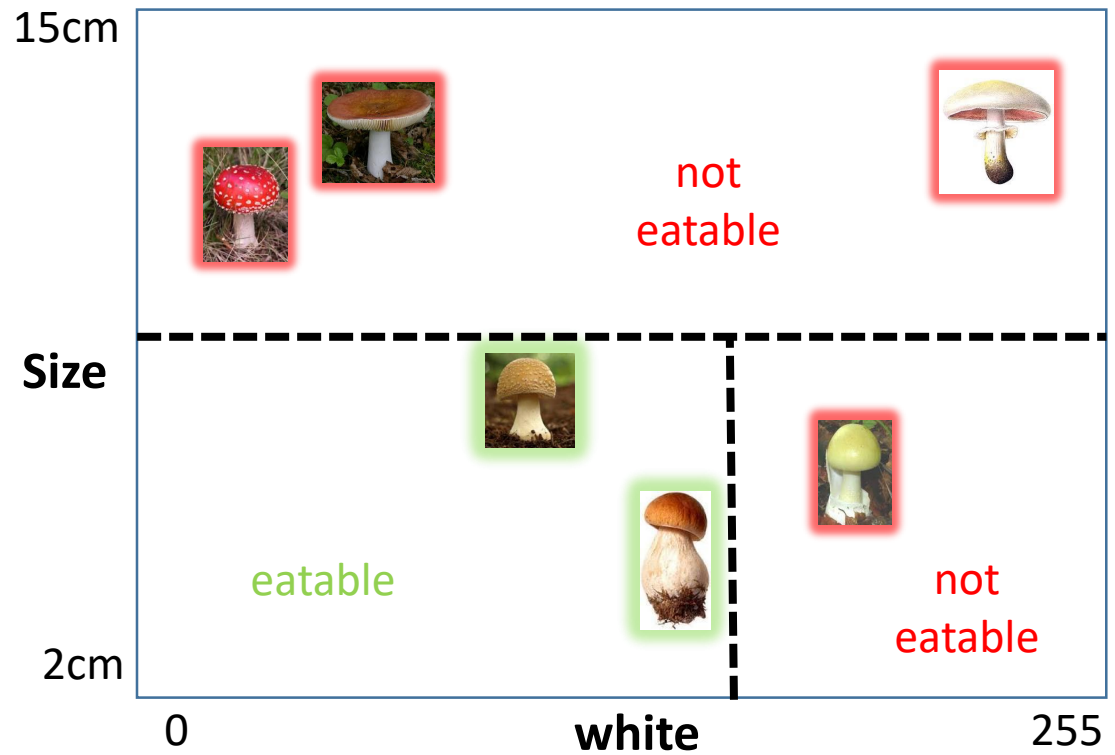
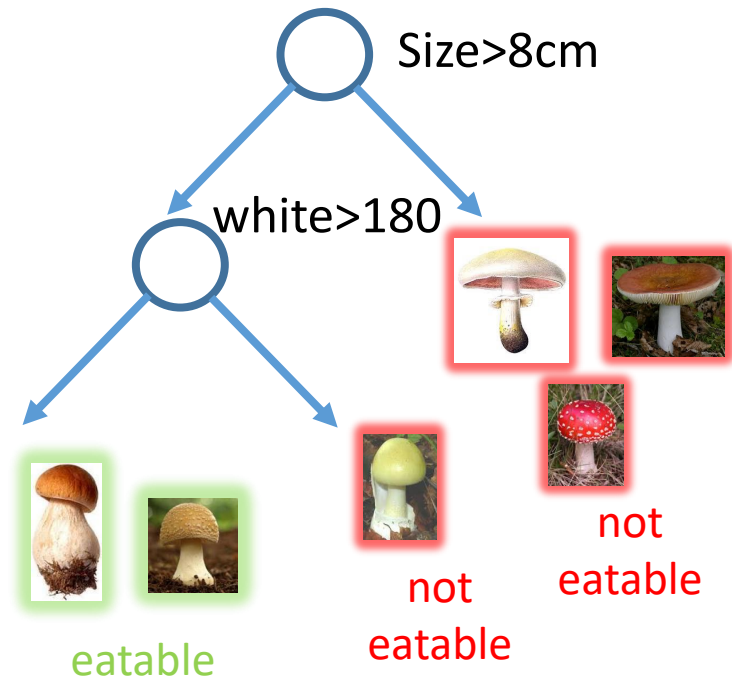


Task: Build a decision tree such that you can distinguish eatable from not-eatable mushrooms

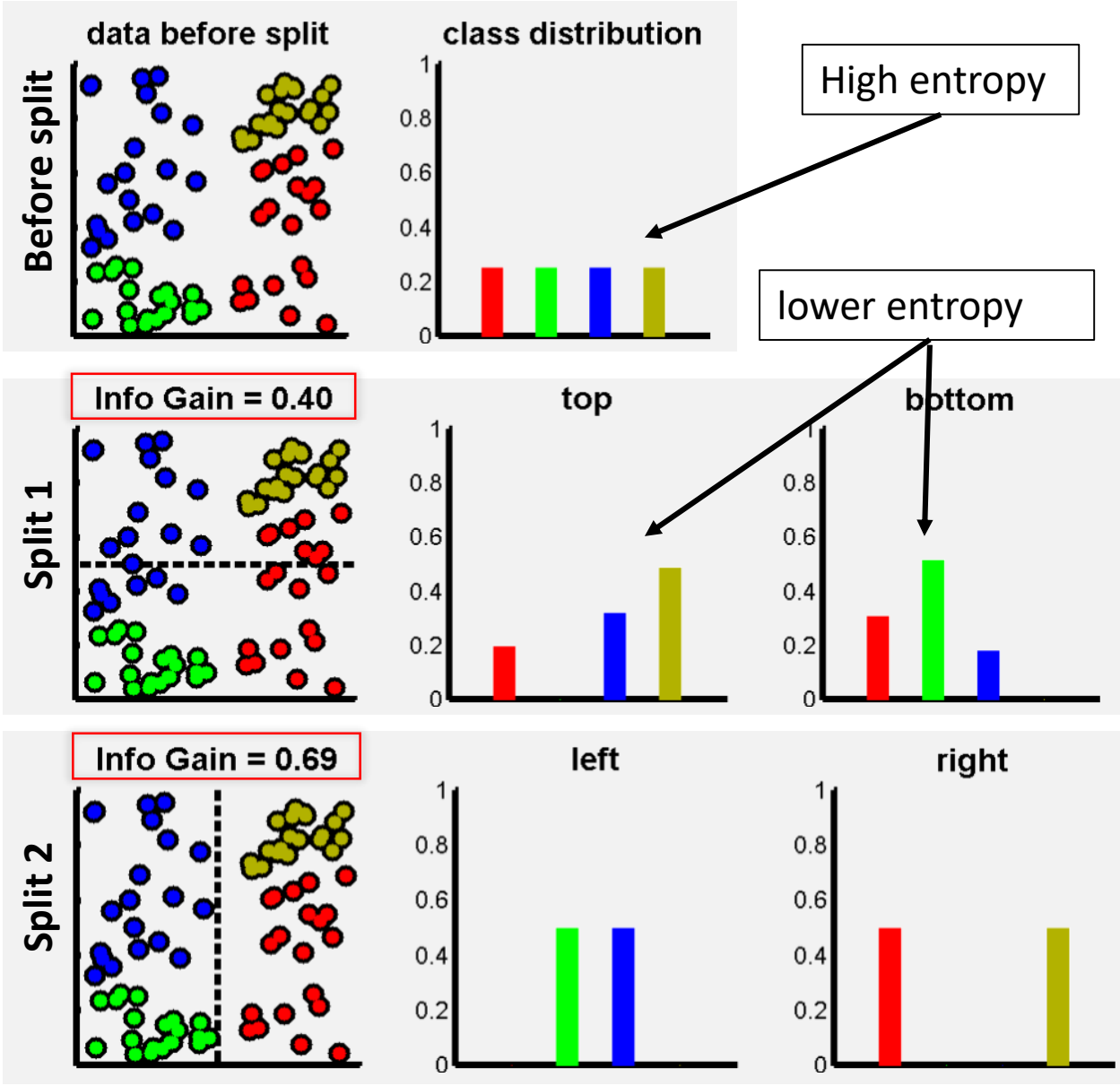
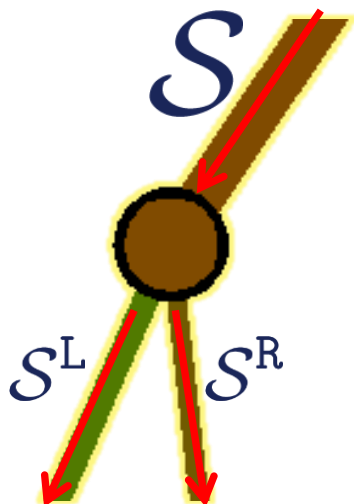
Decision Tree



Decision Tree



Decision Tree – Split Criteria



Maximize Information gain

$$I(S, \theta) = H(S) - \sum_{i \in \{L, R\}} \frac{|S^i|}{|S|} H(S^i)$$

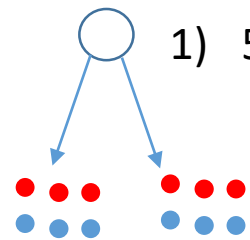
Shannon's entropy

$$H(S) = - \sum_{c \in \mathcal{C}} p(c) \log(p(c))$$

Think of minimizing Entropy

Decision Tree – Split Criteria

- We have $|S| = 12$
- In S we have 6 red and 6 blue points (2 classes)
- $H(S) = -(0.5 \log(0.5) + 0.5 \log(0.5)) = 1$
- We look at two possible splits
(in both cases we happen to have $|S^L| = 6$ and $|S^R| = 6$, but could be different)

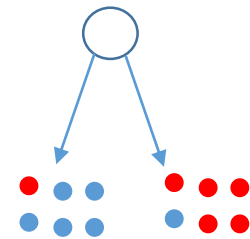


- 1) 50%-50% class-split (each side (S^L and S^R) gets 3 red and 3 blue)

$$H(S^L) = -(0.5 \log(0.5) + 0.5 \log(0.5)) = 1$$

$$H(S^R) = -(0.5 \log(0.5) + 0.5 \log(0.5)) = 1$$

$$I(S) = H(S) - (0.5 + 0.5) = H(S) - 1 = 0 \quad \text{(Lower information gain)}$$



- 2) 16%-84% class-split (right side has 5 red and 1 blue, left side has 5 blue and 1 red)

$$H(S^L) = -\left(\frac{1}{6} \log\left(\frac{1}{6}\right) + \frac{5}{6} \log\left(\frac{5}{6}\right)\right) = 0.64$$

$$H(S^R) = -\left(\frac{1}{6} \log\left(\frac{1}{6}\right) + \frac{5}{6} \log\left(\frac{5}{6}\right)\right) = 0.64$$

$$I(S) = H(S) - (0.5 * 0.64 + 0.5 * 0.64) = H(S) - 0.64 = 0.36$$

(Higher information gain)

Generalization

Training Data:



eatable



not
eatable



eatable



not
eatable

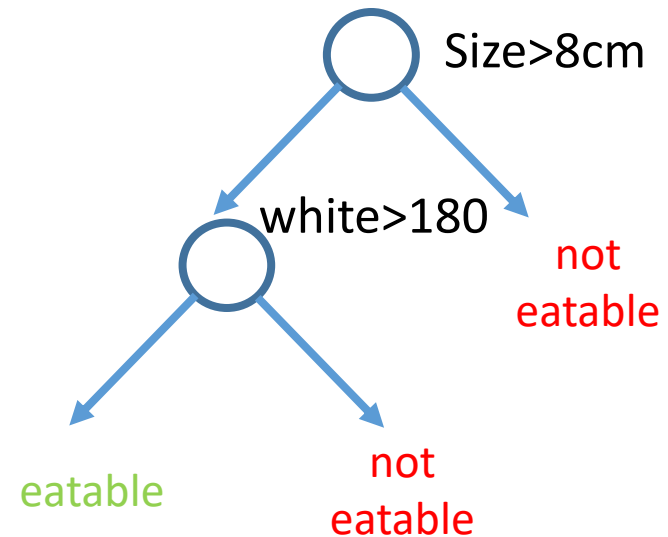


not
eatable



not
eatable

Test Data:



System is optimal!

Generalization

Training Data:



eatable



not
eatable



eatable



not
eatable

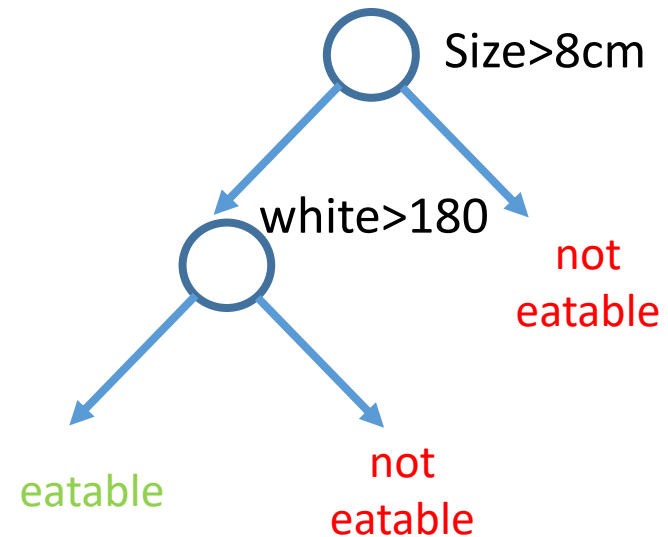


not
eatable



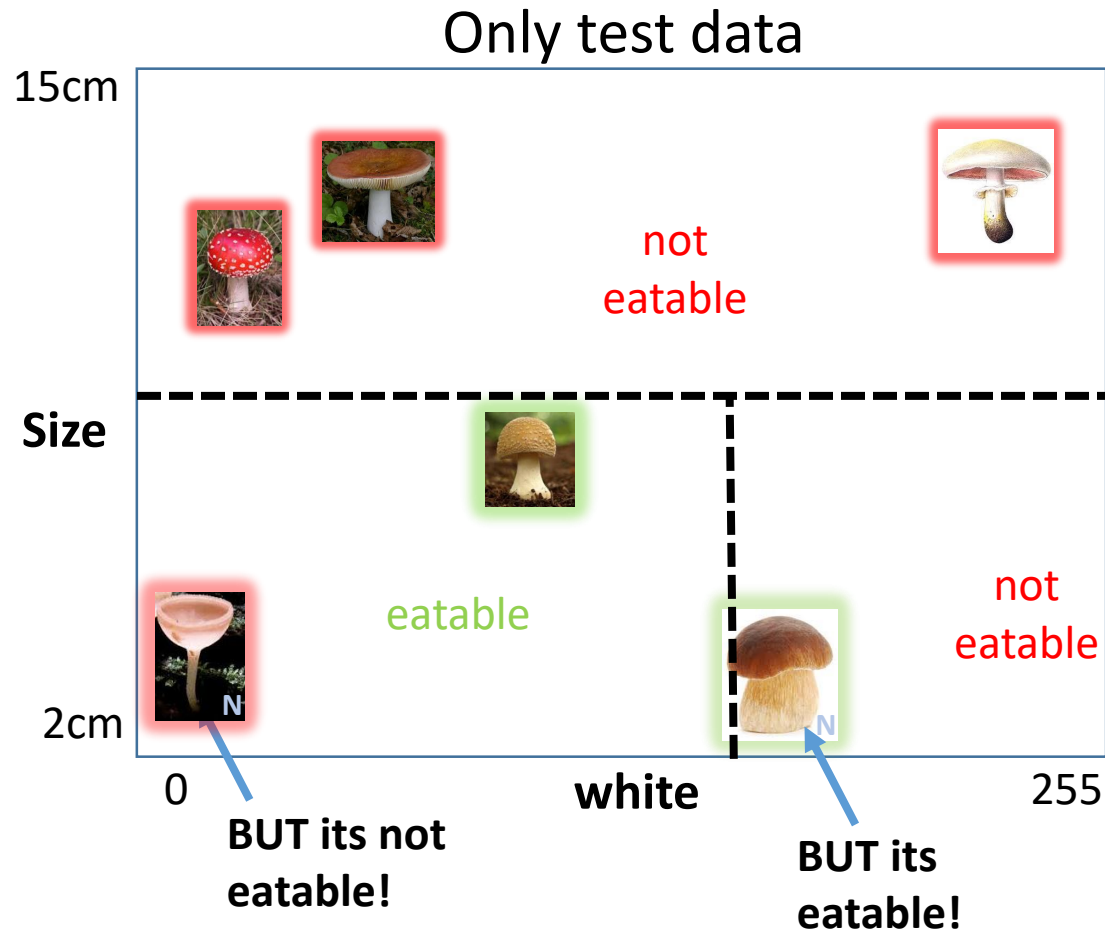
not
eatable

Test Data:



Is the System optimal?

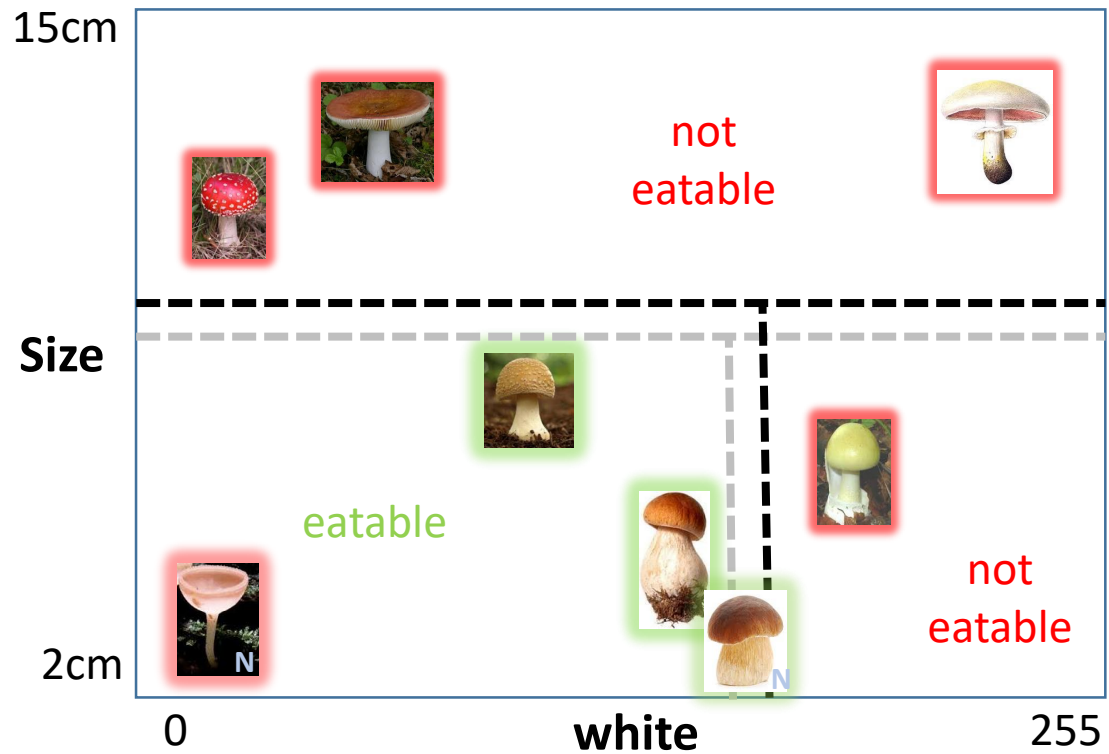
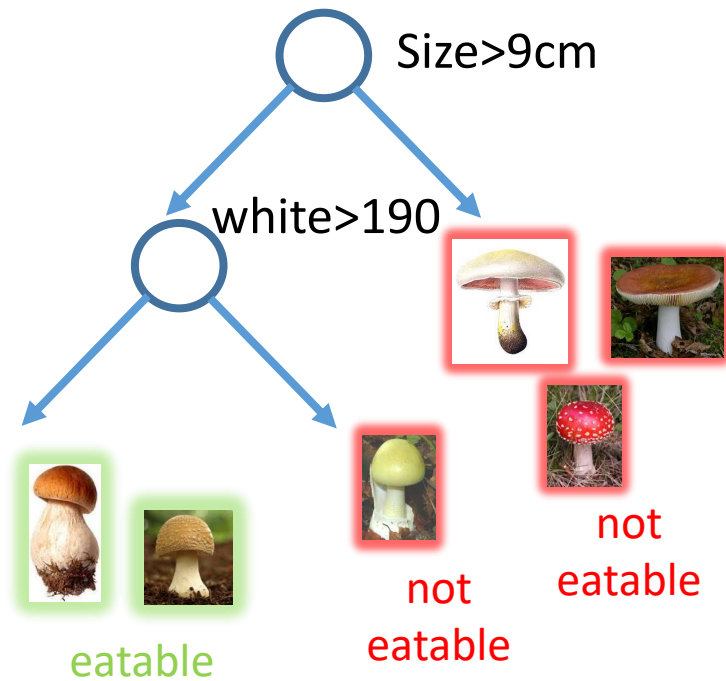
Is the System optimal?



Definition

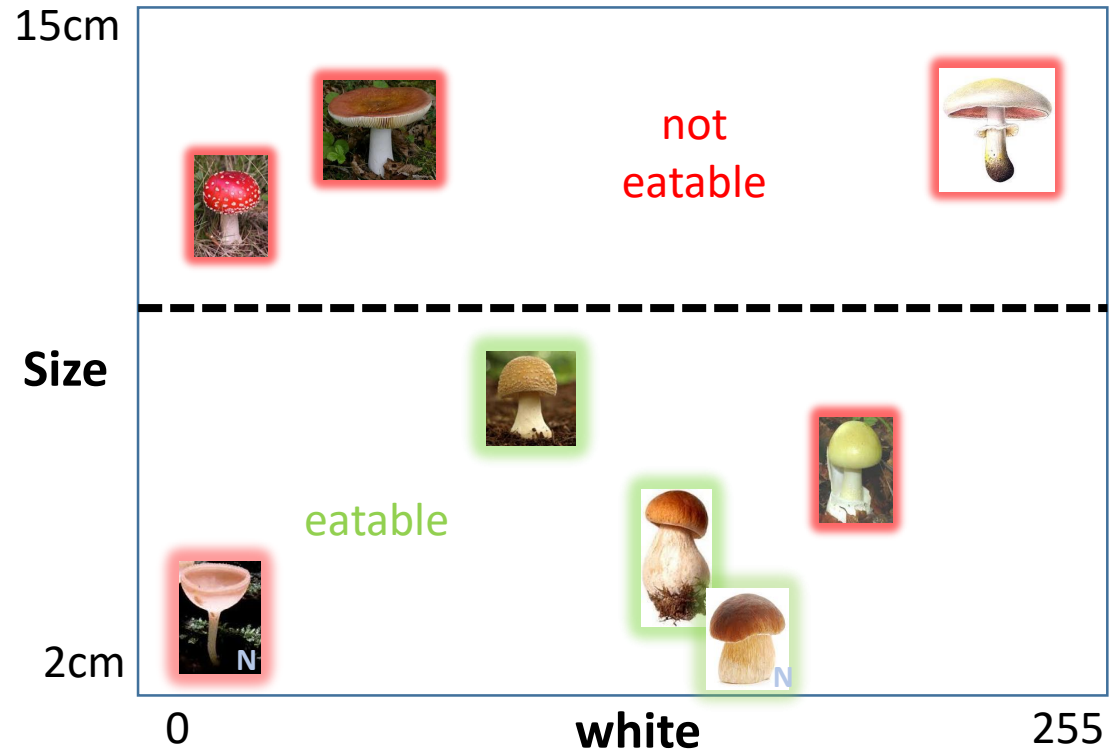
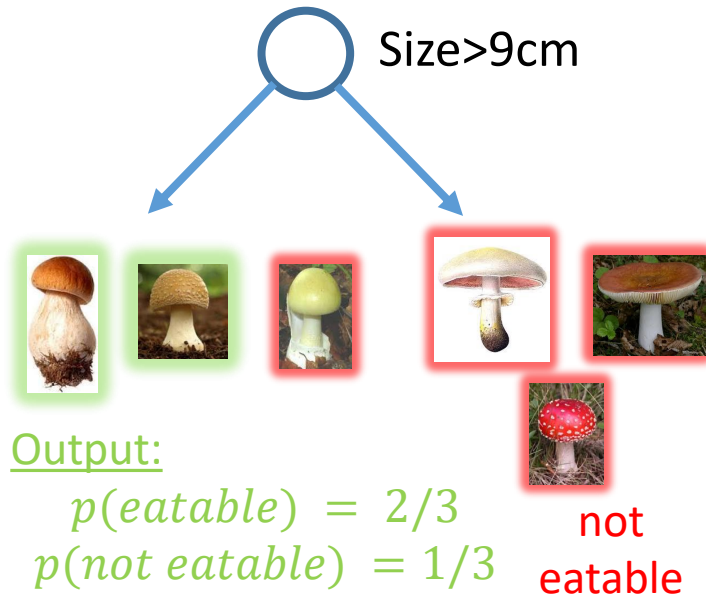
- **Over-fitting:** Is the effect that the model perfectly memorizes the training data, but does not perform well on test data
- **Generalization:** One of the most important aspects of a model is its ability to generalize. That means that new (unseen) test data is correctly classified. A model which overfits does not generalize well.
- **How to avoid over-fitting?**
 - Idea 1: Where to place the decision boundary?
 - Idea 2: Do not make the tree too deep

Decision Boundary

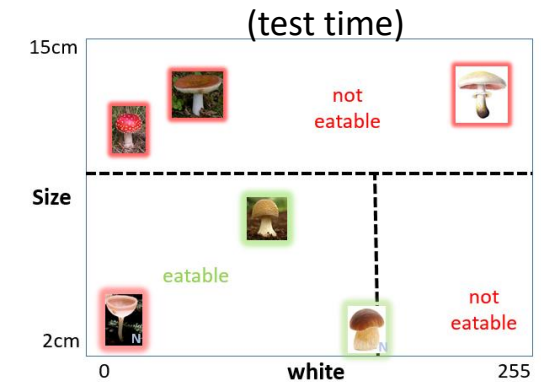
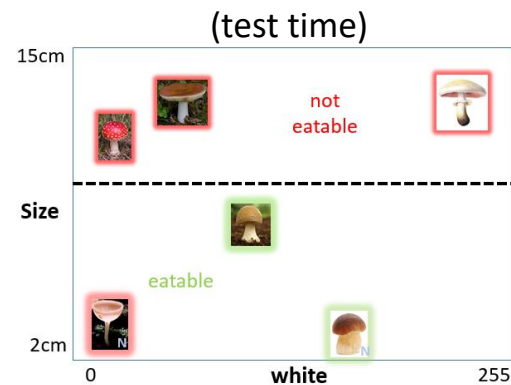
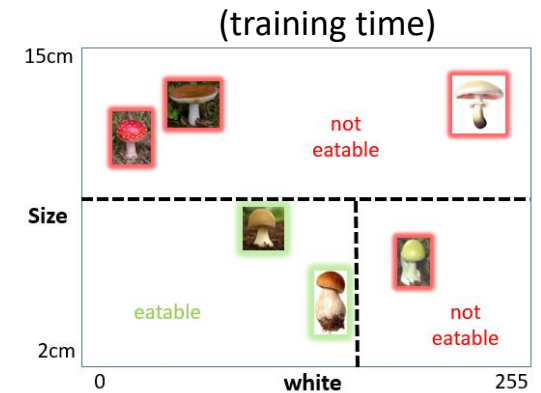
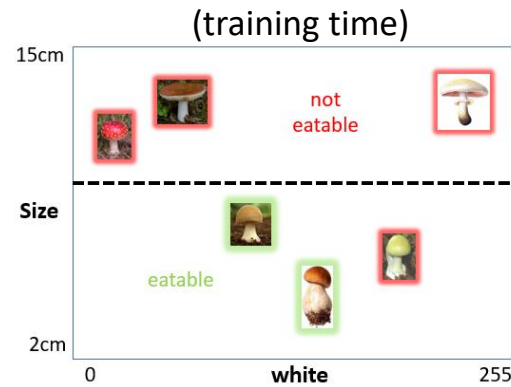


Place “decision boundary” such that the distance to all (some) training examples is maximized!

Tree Depth



Tree depth

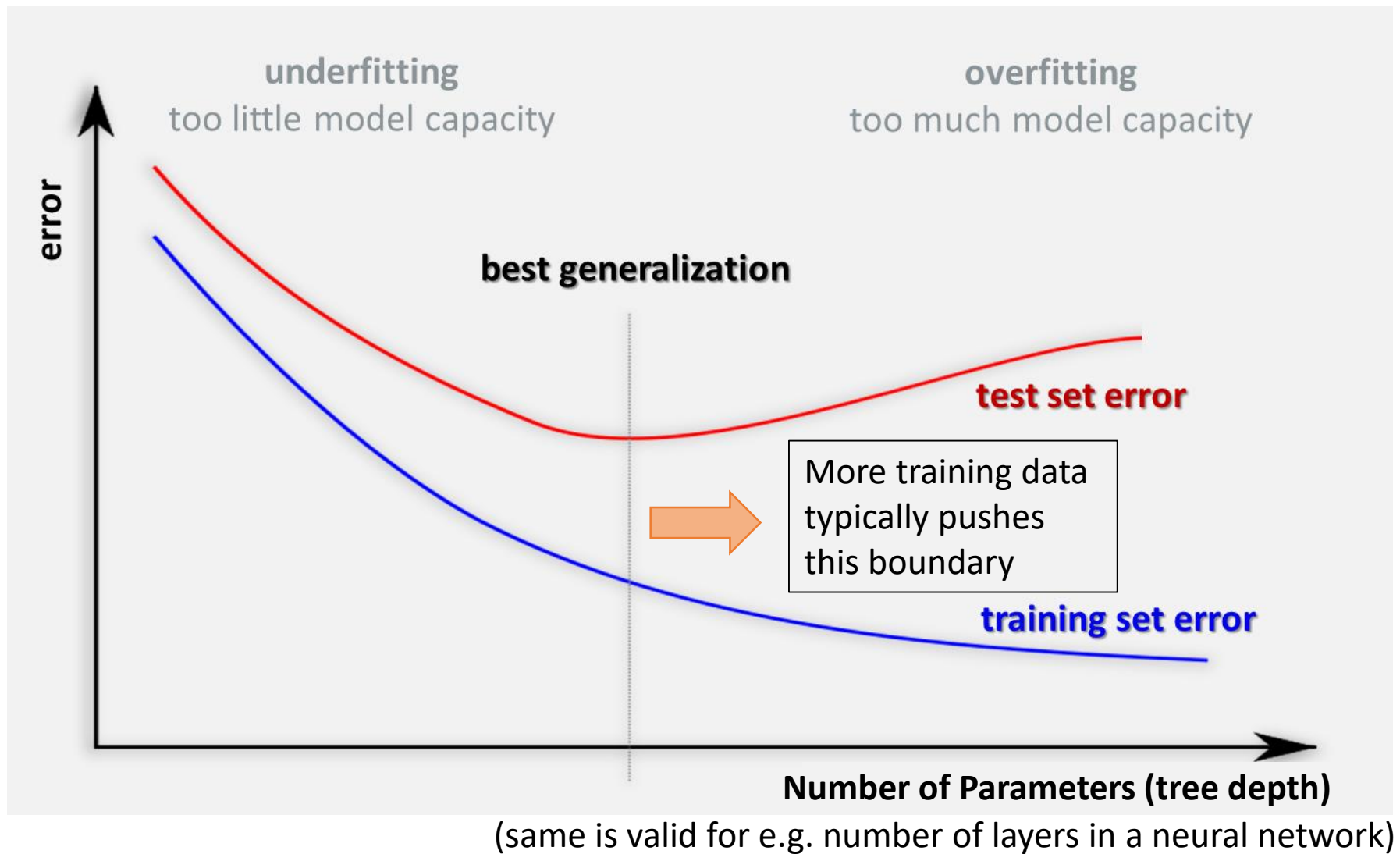


- **System with 1 split-node** makes:
1 mistake training; 1 mistake testing
- **System with 2 split-nodes** makes:
0 mistake training; 1 mistake testing
- **Conclusion:** A system which makes less mistakes during training may not be a better system at test time!

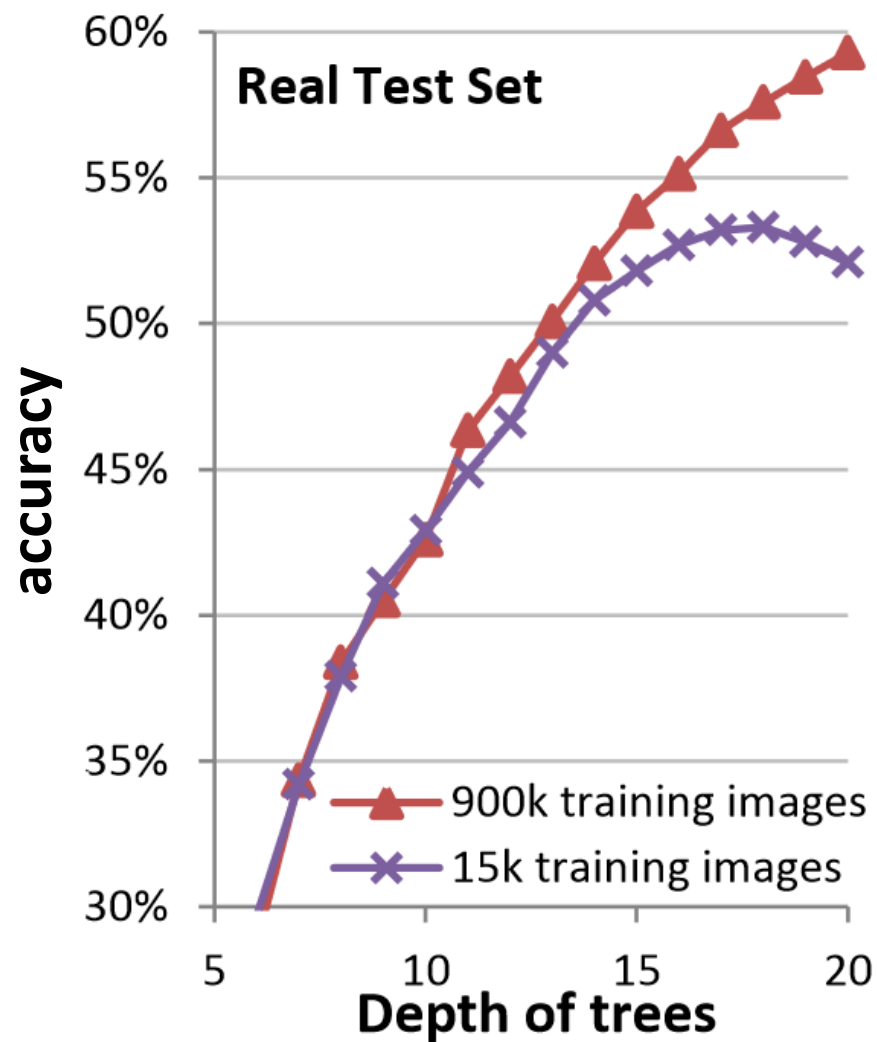
Test data:



Tree Depth



Real world example



Xbox people tracking system

Roadmap

- Types of Machine Learning Problems
- Decision Trees
- Deep Learning: Discriminative Models
 - Neural Networks
 - Convolutional Neural Networks
- Image Classification: ResNet
- Understanding Image Classification Networks
- Semantic Segmentation: U-Net
- Active and Semi-Supervised Learning

Large Number of Models:

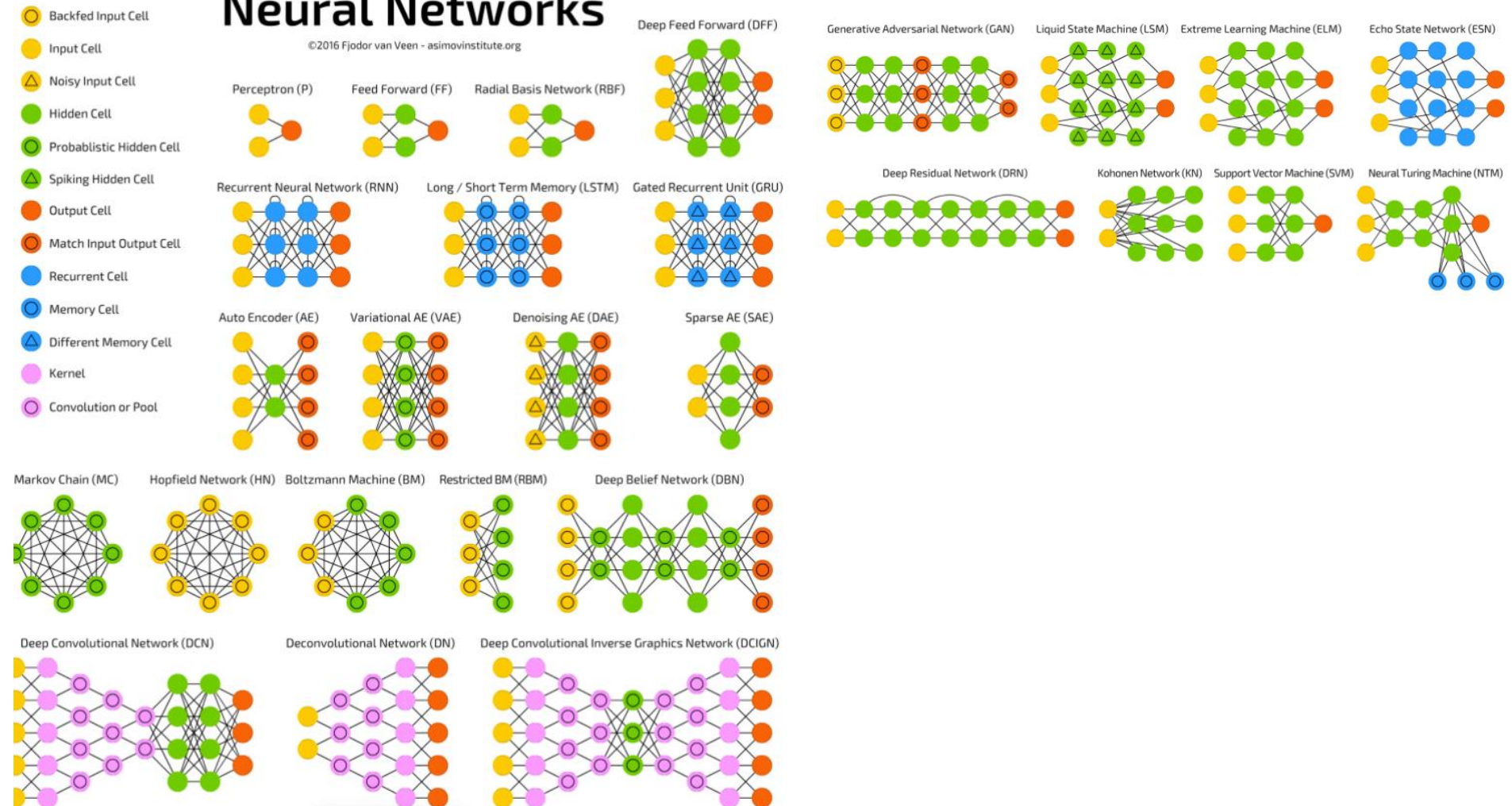
- Perceptron
- Multi-layer Perceptron (MLP) (also called Feed Forward Neural Network)
- Convolutional Neural Network (CNN)
- (Variational) Auto-Encoder (VAE)
- Generative Adversarial Neural Network (GAN)
- Transformers
- Diffusion Models
- Normalizing Flows (Invertible Neural network, INN)
- Recurrent Neural Network (RNN)
- Long Short-Term Memory (LSTM)
- Radial Basis Functional Neural Network

....

Even more types of Network Types

Neural Networks

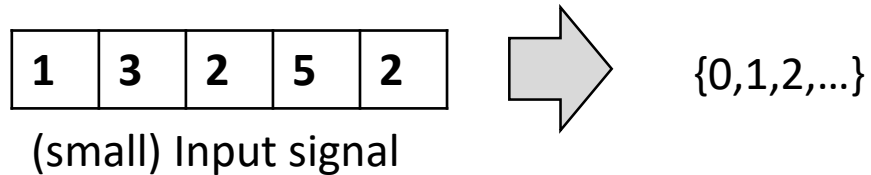
©2016 Fjodor van Veen - asimovinstitute.org



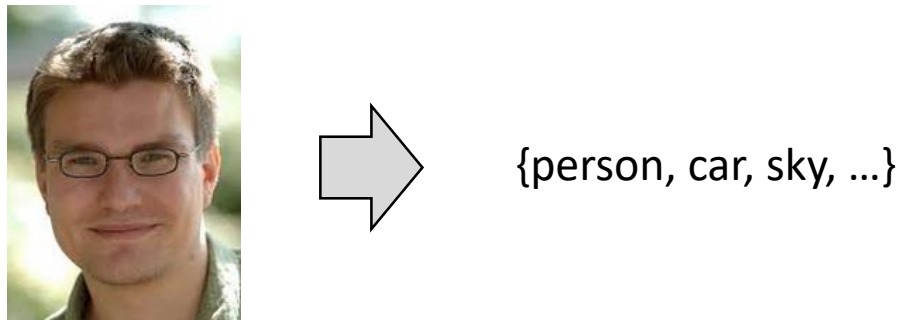
<https://towardsdatascience.com/the-mostly-complete-chart-of-neural-networks-explained-3fb6f2367464>

Different Tasks

- Neural Network (small input)



- Convolutional Neural Network (large input)



Different Tasks

- Convolutional Neural Network (classification)



{person, car, sky, ...}

- Convolutional Neural Network (regression)

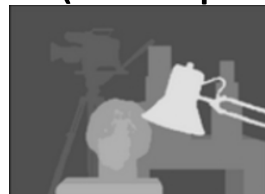


$\mathbb{R}$ (How dark is the hair?)

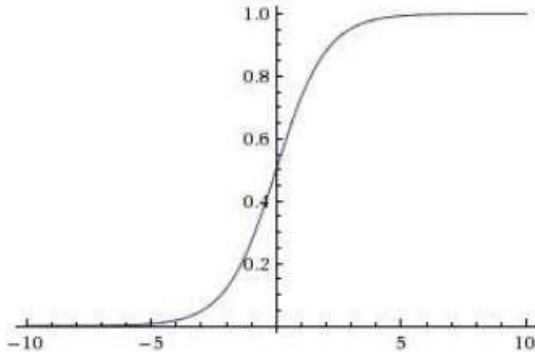
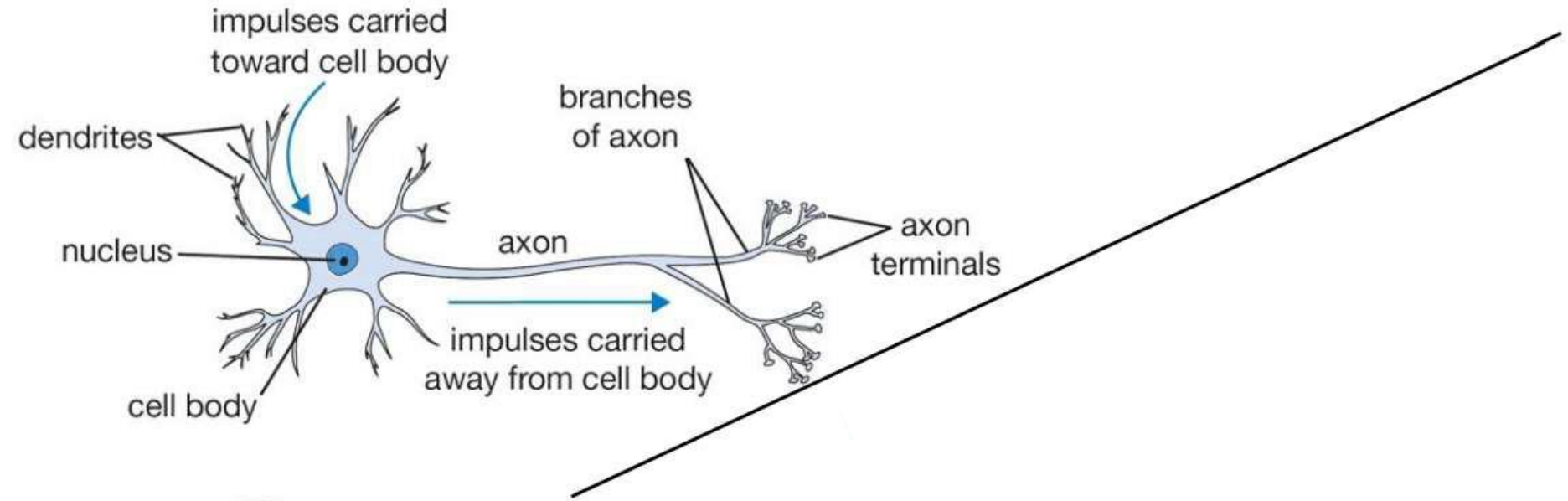
- Convolutional Neural Network (multiple variable classification)



- Convolutional Neural Network (multiple variable regression)



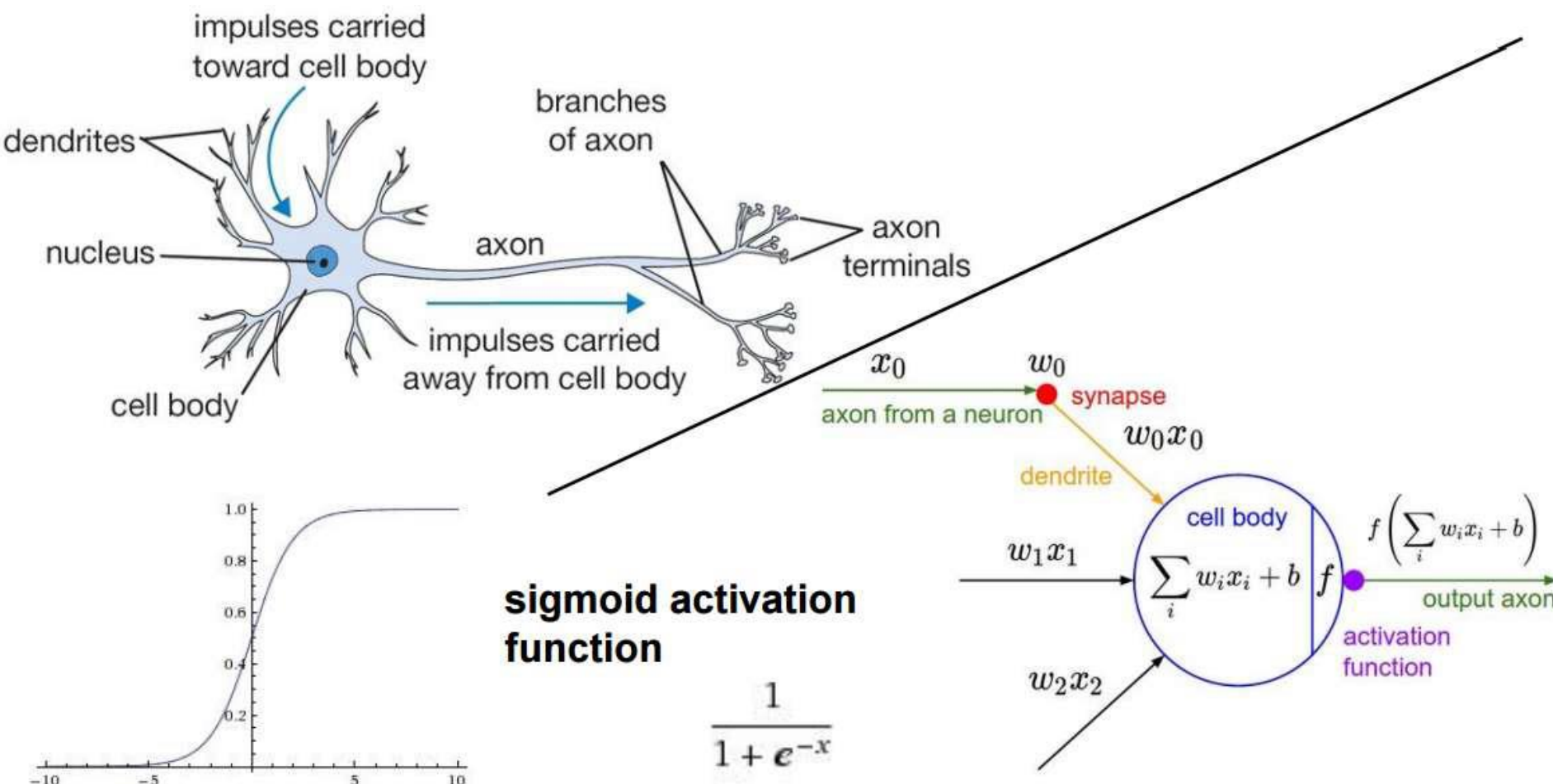
Neurons – artificial and real



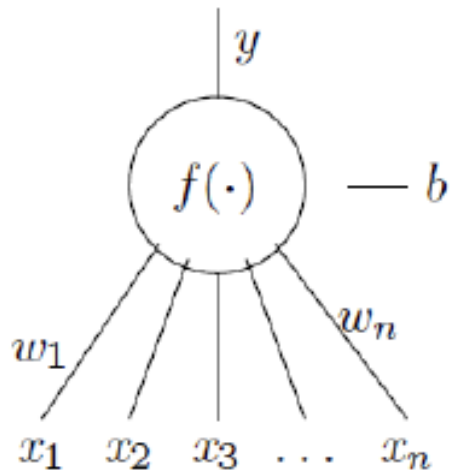
**sigmoid activation
function**

$$\frac{1}{1 + e^{-x}}$$

Neurons – artificial and real



Single Neuron (aka Perceptron)



Input: $\mathbf{x} \in \mathbb{R}^n$

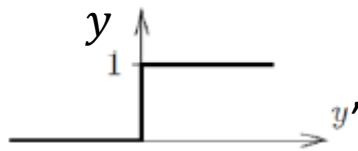
Weights: $\mathbf{w} \in \mathbb{R}^n$

Bias: $b \in \mathbb{R}$

Activation function: $f: \mathbb{R} \rightarrow \mathbb{R}$

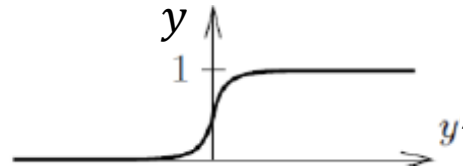
Output: $y = f(\mathbf{w}^t \mathbf{x} + b)$

Two possible activation functions:



$$f(y') = \begin{cases} 1 & \text{if } y' > 0 \\ 0 & \text{otherwise} \end{cases}$$

Step function

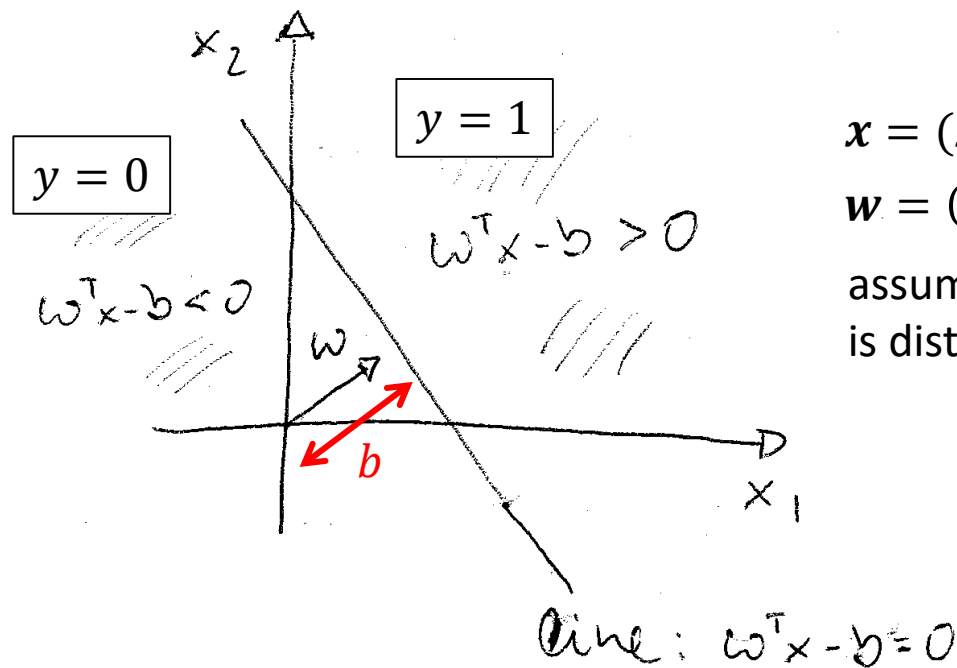


$$f(y') = \frac{1}{1 + \exp(-y')}$$

Sigmoid function

$y \in [0,1]$ can be seen as probability

Geometric interpretation



$$x = (x_1, x_2)$$

$$w = (w_1, w_2)$$

assume $\|w\| = 1$ then b
is distance of line to origin

A single neuron implements a linear classifier

Multilayer Perceptron (MLP)

Input layer

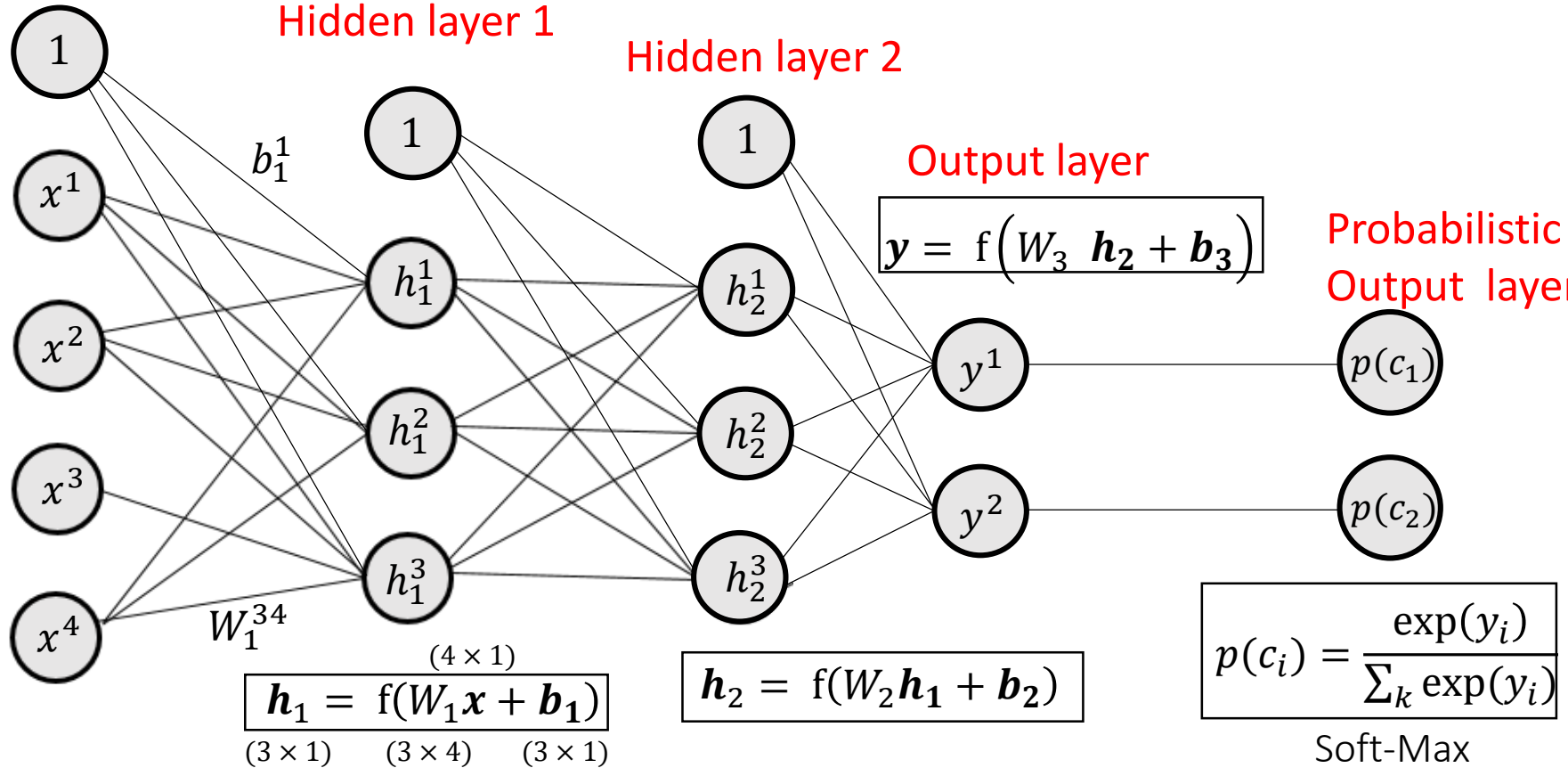
Often referred to as „Feed Forward Neural network“

Hidden layer 1

Hidden layer 2

Output layer

Probabilistic
Output layer

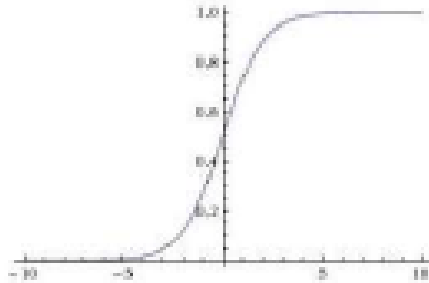


In compact form:

$$\mathbf{y} = f(W_3(f(W_2(f(W_1 \mathbf{x} + \mathbf{b}_1)) + \mathbf{b}_2)) + \mathbf{b}_3)$$

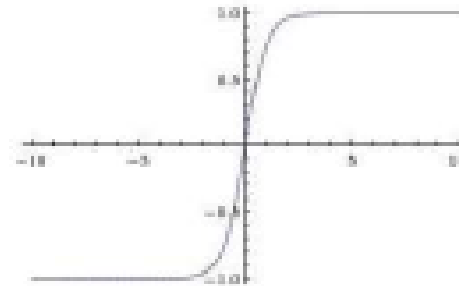
Activation functions

The function $f(W_i \mathbf{x} + \mathbf{b}_i)$ operates on each scalar input:

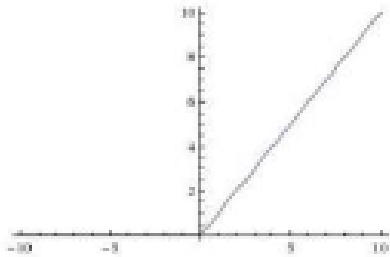


$$f(y) = 1/(1 + e^{-y})$$

Sigmoid

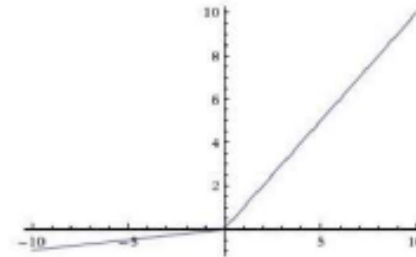


$$f(y) = \tanh(y)$$



$$f(y) = \max(0, y)$$

Rectified Linear Unit (ReLU)



$$f(y) = \max(0.1y, y)$$

Leaky ReLU

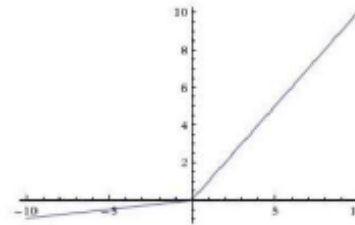
*(most used, avoids
vanishing gradients)*

Question

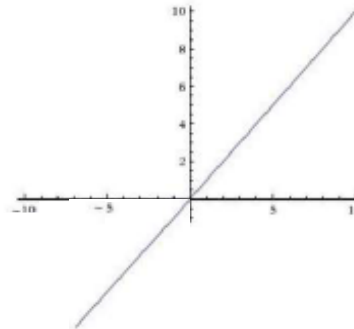
Given a network $\mathbf{y} = f(W_3(f(W_2(f(W_1\mathbf{x} + \mathbf{b}_1)) + \mathbf{b}_2)) + \mathbf{b}_3)$.

Which network is more “powerful” (e.g. can classify better)

1) With $f(y) = \max(0.1y, y)$



2) With $f(y) = y$



3) I don't know

Understanding the Decision Boundary

Construct an “AND” neuron:

Input: $h_1^1, h_1^2 \in \{0,1\}$

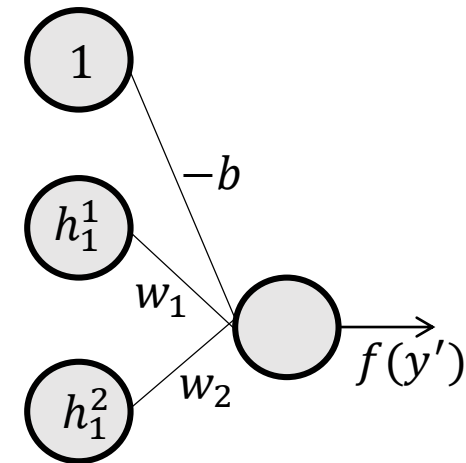
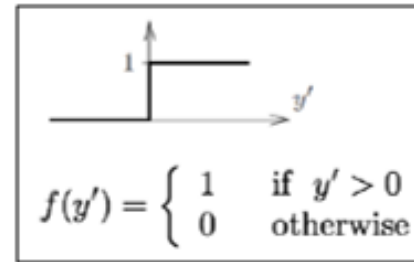
Weights: w_1, w_2

Bias: b

Step-Activation function: f

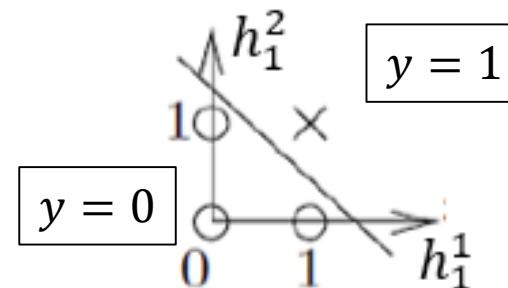
Output: $y = f(w_1 h_1^1 + w_2 h_1^2 - b)$

Goal: $y = h_1^1 \& h_1^2$



h_1^1	h_1^2	y
0	0	0
0	1	0
1	0	0
1	1	1

Solution: $w_1 = \frac{1}{\sqrt{2}}, w_2 = \frac{1}{\sqrt{2}}, b = 1$



Question

How would you construct an “OR” neuron?

Answers:

- 1) $w_1 = \frac{1}{\sqrt{2}}, w_2 = \frac{1}{\sqrt{2}}, b = 0.2$
- 2) $w_1 = -\frac{1}{\sqrt{2}}, w_2 = \frac{1}{\sqrt{2}}, b = 0.2$
- 3) It's not possible
- 4) I don't know

Construct an “AND” neuron:

Input: $h_1^1, h_1^2 \in \{0,1\}$

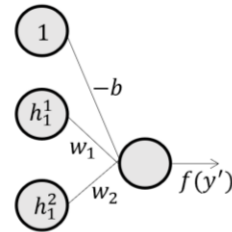
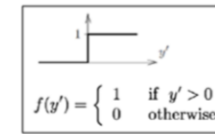
Weights: w_1, w_2

Bias: b

Step-Activation function: f

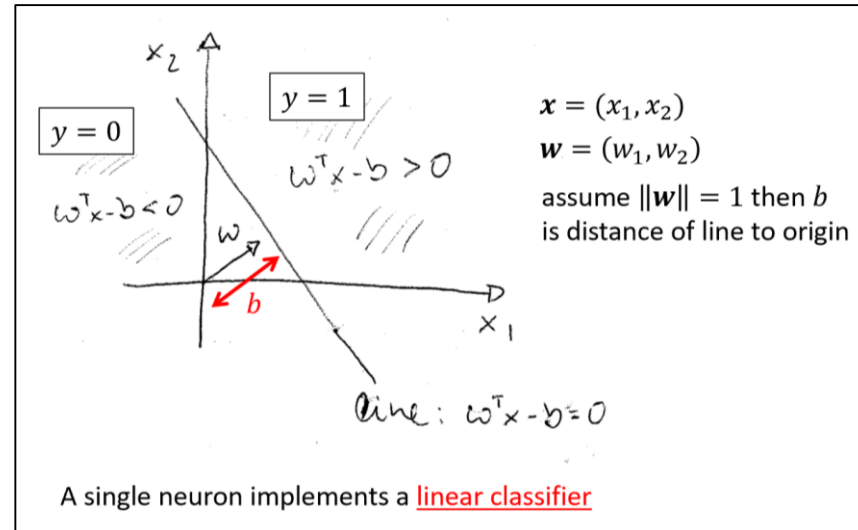
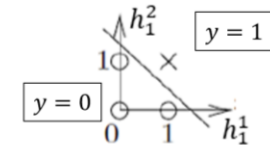
Output: $y = f(w_1 h_1^1 + w_2 h_1^2 - b)$

Goal: $y = h_1^1 \& h_1^2$

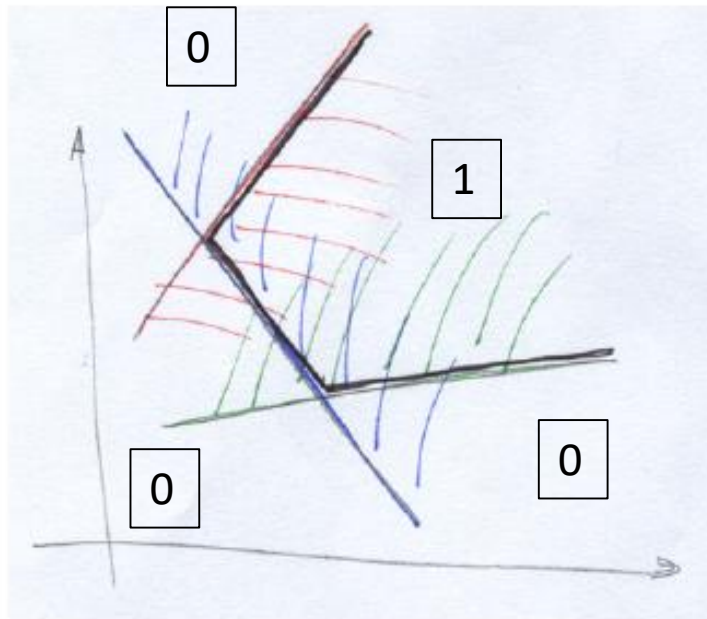


h_1^1	h_1^2	y
0	0	0
0	1	0
1	0	0
1	1	1

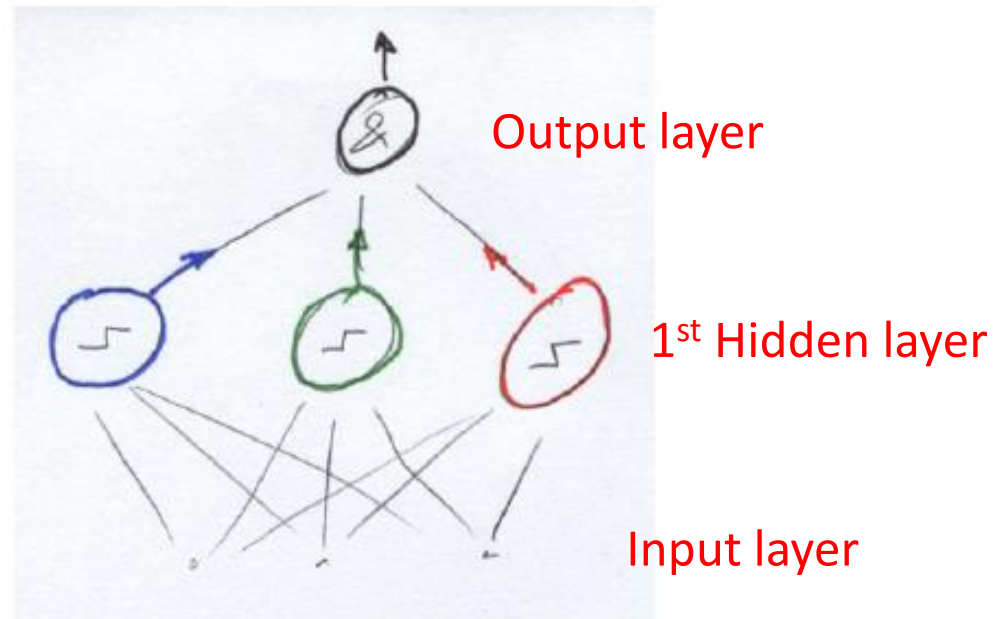
Solution: $w_1 = \frac{1}{\sqrt{2}}, w_2 = \frac{1}{\sqrt{2}}, b = 1$



Understanding Decision Boundary

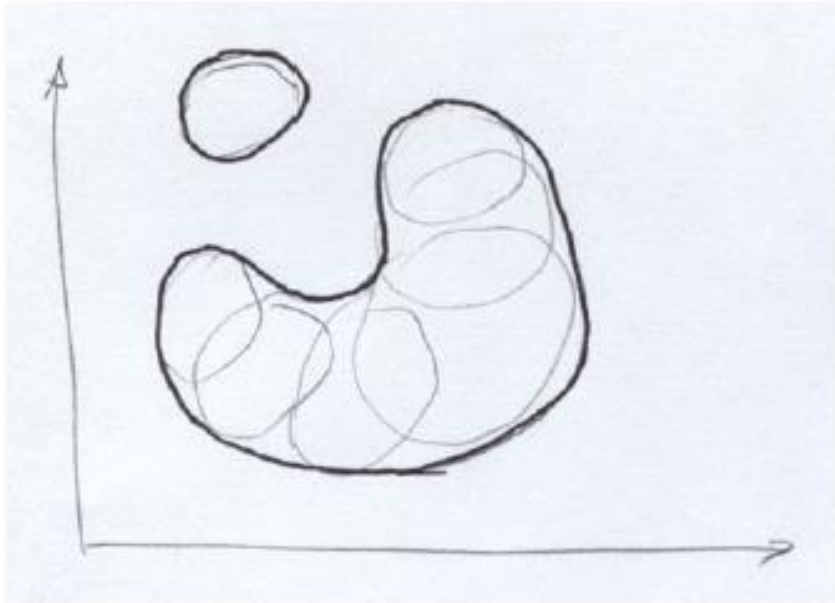


Any convex subspace can be implemented with an arbitrary precision

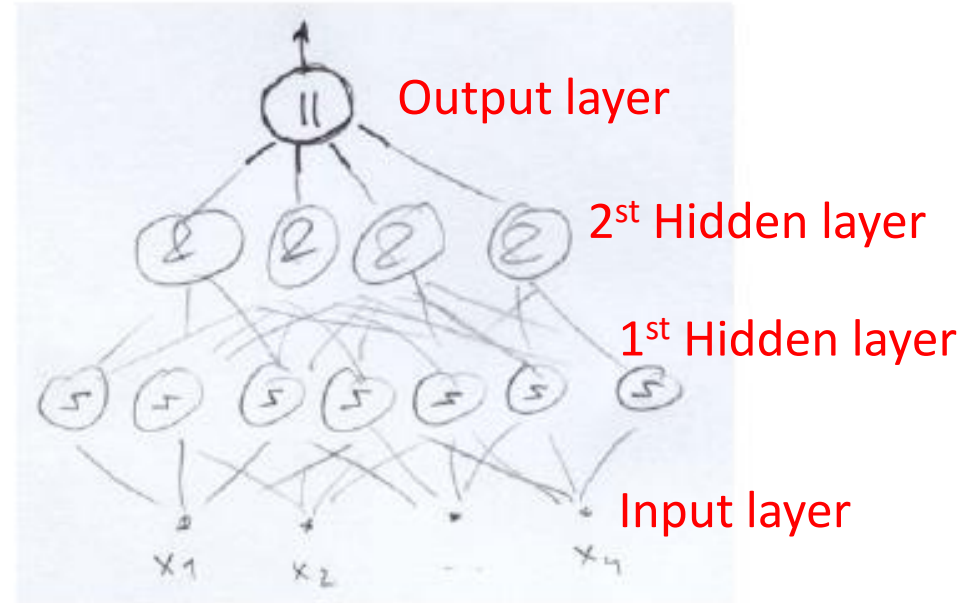


One hidden layers with unlimited number of “step” neurons and one “AND” neuron at output layer

Understanding Decision Boundary



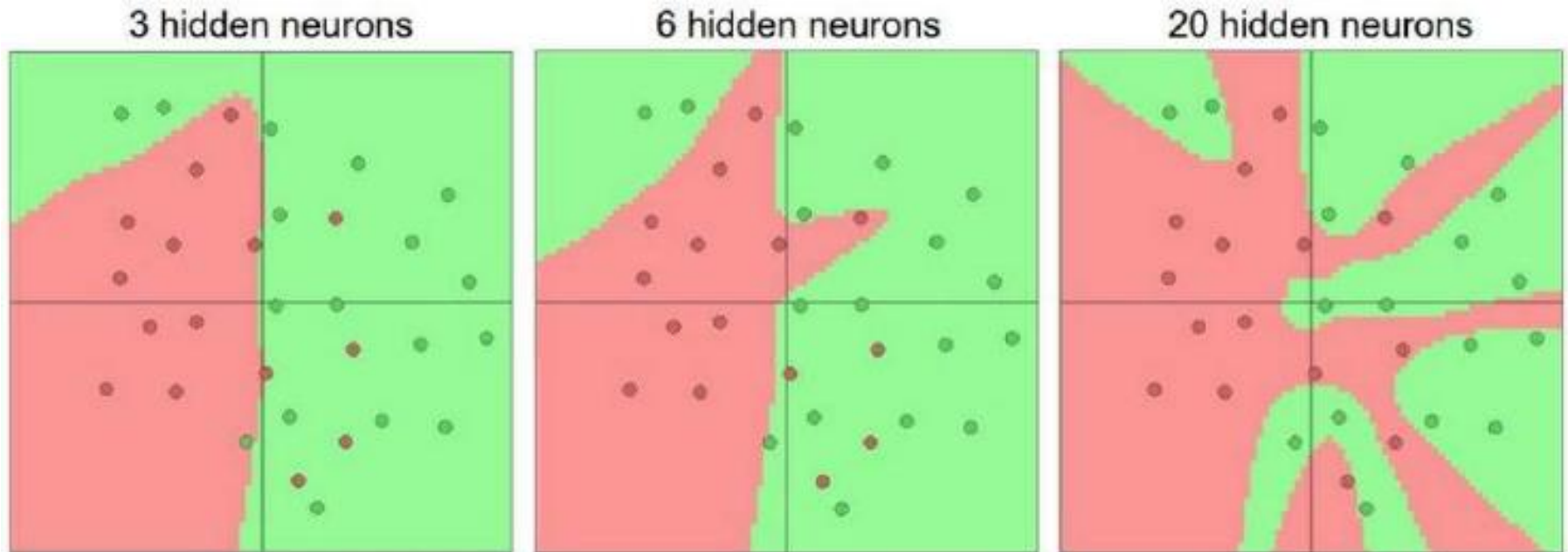
Any subspace can be implemented as a union of convex subspaces



Two hidden layers with unlimited number of “step” neurons and “AND” neurons, and one “OR” neuron at output layer

In practice one hidden layer is already powerful

Example



Network with one hidden layer
(remember: too many neurons may overfit the training data)

Learning Neural Networks

Derivative Rules:

$$\frac{\partial f(x) + g(x)}{\partial x} = \frac{\partial f(x)}{\partial x} + \frac{\partial g(x)}{\partial x}$$

$$\frac{\partial f(x) \cdot g(x)}{\partial x} = \frac{\partial f(x)}{\partial x} \cdot g(x) + f(x) \cdot \frac{\partial g(x)}{\partial x}$$

$$\frac{\partial f(g(x))}{\partial x} = \frac{\partial f(y)}{\partial y} \cdot \frac{\partial g(x)}{\partial x} \quad y = g(x) \quad \text{So-called „chain rule“}$$

Special Derivatives:

$$\frac{\partial e^{ax}}{\partial x} = a e^{ax}$$

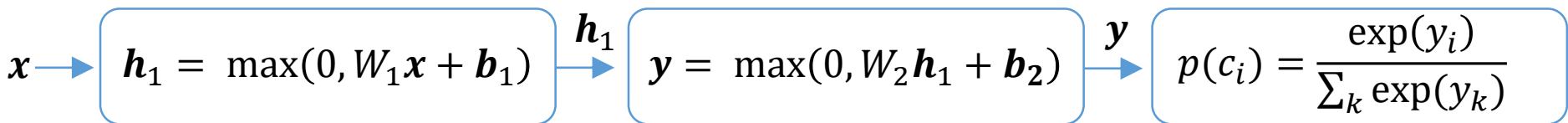
$$\frac{\partial \log x}{\partial x} = \frac{1}{x}$$

$$\frac{\partial x^{-1}}{\partial x} = -x^{-2}$$

Example $y = x^2 - 1$

$$\begin{aligned} \frac{\partial f(x)}{\partial x} &= \frac{\partial \ln(x^2 - 1)}{\partial x} = \frac{\partial \ln(y)}{\partial y} \cdot \frac{\partial y}{\partial x} \\ &= \frac{1}{y} \cdot 2x = \frac{1}{(x^2 - 1)} \cdot 2x \end{aligned}$$

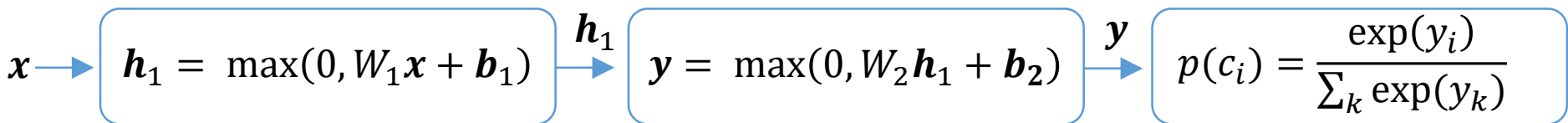
Learning Neural Networks



- x is the input, $p(c)$ is the output
- h_i is the i -th hidden layer output
- W_i, b_i are the parameters of the i -th layer
- all weights will be denoted by $\mathbf{w} = \{W_i, b_i\}$

- We want to estimate the parameters and hyper-parameters (e.g., number of layers, number of units) such that we do good prediction
- Collect a training set of input-output pairs $\{\mathbf{x}, \mathbf{t}\}$
- Encode the output with 1-K encoding $\mathbf{t} = [0, \dots, 1, \dots, 0]$

Learning Neural Networks



- x is the input, $p(\mathbf{c})$ is the output
- $\mathbf{h}_i$ is the i -th hidden layer output
- $W_i, \mathbf{b}_i$ are the parameters of the i -th layer
- all weights will be denoted by $\mathbf{w} = \{W_i, \mathbf{b}_i\}$

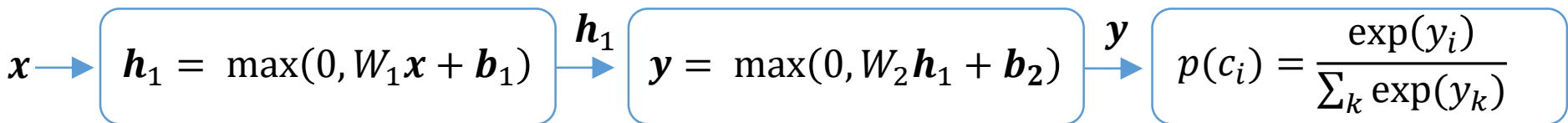
- We want to estimate the parameters and hyper-parameters (e.g., number of layers, number of units) such that we do good prediction
- Collect a training set of input-output pairs $\{\mathbf{x}, \mathbf{t}\}$
- Encode the output with 1-K encoding $\mathbf{t} = [0, \dots, 1, \dots, 0]$
- Define a loss ℓ per training example and the overall loss $\mathcal{L}$

$$\mathcal{L}(\mathbf{w}) = \frac{1}{N} \sum_{n=1}^N \ell(\mathbf{w}, \mathbf{x}^n, \mathbf{t}^n) + \mathcal{R}(\mathbf{w})$$

with N number of examples, $\mathcal{R}$ a regularizer, and $\mathbf{w}$ are all parameters

- **Training Objective:** minimize $\mathcal{L}(\mathbf{w})$: $\mathbf{w}^* = \underset{\mathbf{w}}{\operatorname{argmin}} \mathcal{L}(\mathbf{w})$

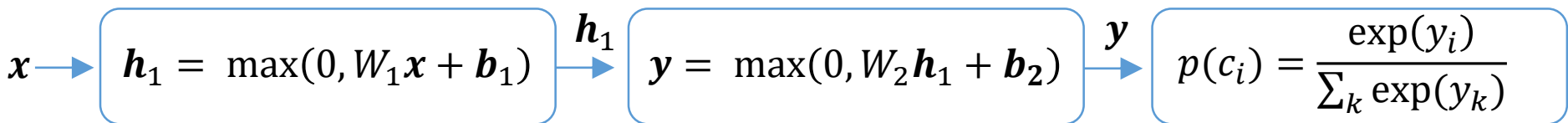
Learning Neural Networks



$$\mathcal{L}(\mathbf{w}) = \frac{1}{N} \sum_{n=1}^N \ell(\mathbf{w}, \mathbf{x}^n, \mathbf{t}^n) + \mathcal{R}(\mathbf{w})$$

- $\mathbf{x}$ is the input, $p(\mathbf{c})$ is the output
- $\mathbf{h}_i$ is the i -th hidden layer output
- $W_i, \mathbf{b}_i$ are the parameters of the i -th layer
- all weights will be denoted by $\mathbf{w} = \{W_i, \mathbf{b}_i\}$

Learning Neural Networks

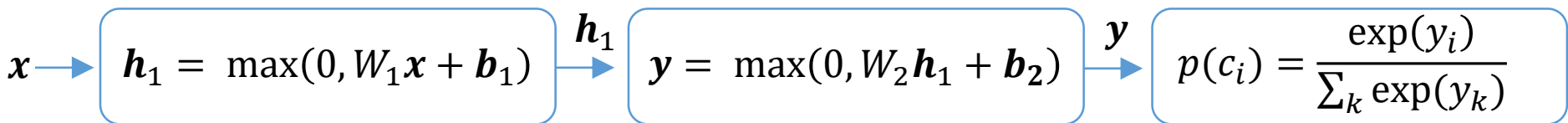


$$\mathcal{L}(\mathbf{w}) = \frac{1}{N} \sum_{n=1}^N \ell(\mathbf{w}, \mathbf{x}^n, \mathbf{t}^n) + \mathcal{R}(\mathbf{w})$$

- x is the input, $p(\mathbf{c})$ is the output
- $\mathbf{h}_i$ is the i -th hidden layer output
- $W_i, \mathbf{b}_i$ are the parameters of the i -th layer
- all weights will be denoted by $\mathbf{w} = \{W_i, \mathbf{b}_i\}$

- The task-loss is often too difficult to compute, so one uses a so-called surrogate loss that is typically convex wrt output

Learning Neural Networks

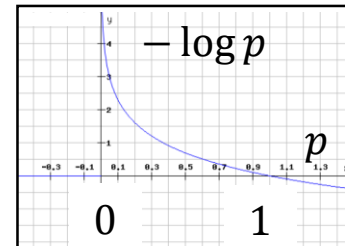


$$\mathcal{L}(\mathbf{w}) = \frac{1}{N} \sum_{n=1}^N \ell(\mathbf{w}, \mathbf{x}^n, \mathbf{t}^n) + \mathcal{R}(\mathbf{w})$$

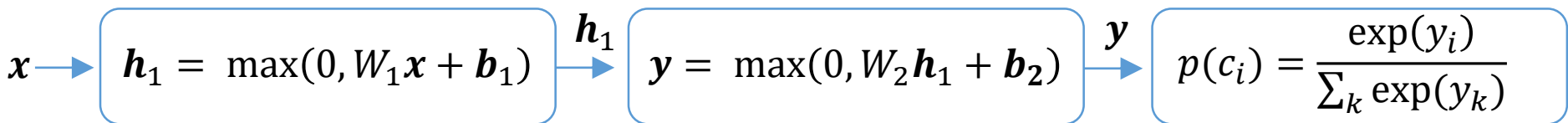
- x is the input, $p(\mathbf{c})$ is the output
- $\mathbf{h}_i$ is the i -th hidden layer output
- $W_i, \mathbf{b}_i$ are the parameters of the i -th layer
- all weights will be denoted by $\mathbf{w} = \{W_i, \mathbf{b}_i\}$

- The task-loss is often too difficult to compute, so one uses a so-called surrogate loss that is typically convex wrt output
- Cross entropy is the most used loss-function for classification

$$\ell(\mathbf{x}, \mathbf{t}, \mathbf{w}) = - \sum_j t_j \log p(c_j | \mathbf{x}) \text{ with } p(c_j = 1 | \mathbf{x}) = \frac{\exp(y_j)}{\sum_{k=1}^C \exp(y_k)}$$



Learning Neural Networks



$$\mathcal{L}(\mathbf{w}) = \frac{1}{N} \sum_{n=1}^N \ell(\mathbf{w}, \mathbf{x}^n, \mathbf{t}^n) + \mathcal{R}(\mathbf{w})$$

- x is the input, $p(\mathbf{c})$ is the output
- $\mathbf{h}_i$ is the i -th hidden layer output
- $W_i, \mathbf{b}_i$ are the parameters of the i -th layer
- all weights will be denoted by $\mathbf{w} = \{W_i, \mathbf{b}_i\}$

- The task-loss is often too difficult to compute, so one uses a so-called surrogate loss that is typically convex wrt output
- Cross entropy is the most used loss-function for classification

$$\ell(\mathbf{x}, \mathbf{t}, \mathbf{w}) = - \sum_j t_j \log p(c_j | \mathbf{x}) \text{ with } p(c_j = 1 | \mathbf{x}) = \frac{\exp(y_j)}{\sum_{k=1}^C \exp(y_k)}$$

Low (good) value when p is high for right example:

0	1	0	0	0
---	---	---	---	---

 $\mathbf{t}$

0.1	0.7	0	0.1	0.1
-----	-----	---	-----	-----

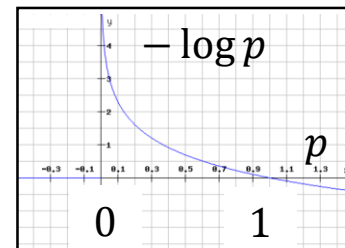
 $\mathbf{p}$
 $\ell(\mathbf{x}, \mathbf{t}, \mathbf{w}) = -\log 0.7 = 0.15$

0	1	0	0	0
---	---	---	---	---

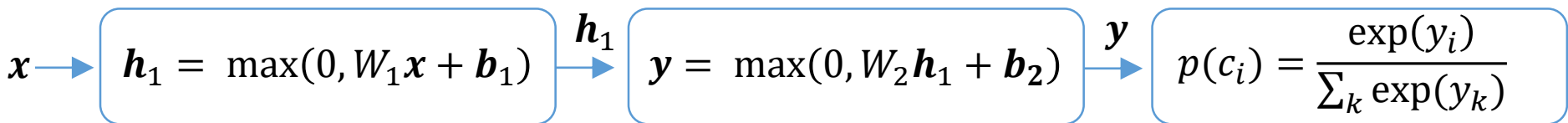
 $\mathbf{t}$

0.3	0.3	0.2	0.1	0.1
-----	-----	-----	-----	-----

 $\mathbf{p}$
 $\ell(\mathbf{x}, \mathbf{t}, \mathbf{w}) = -\log 0.3 = 0.52$



Learning Neural Networks



$$\mathcal{L}(\mathbf{w}) = \frac{1}{N} \sum_{n=1}^N \ell(\mathbf{w}, \mathbf{x}^n, \mathbf{t}^n) + \mathcal{R}(\mathbf{w})$$

- x is the input, $p(\mathbf{c})$ is the output
- $\mathbf{h}_i$ is the i -th hidden layer output
- $W_i, \mathbf{b}_i$ are the parameters of the i -th layer
- all weights will be denoted by $\mathbf{w} = \{W_i, \mathbf{b}_i\}$

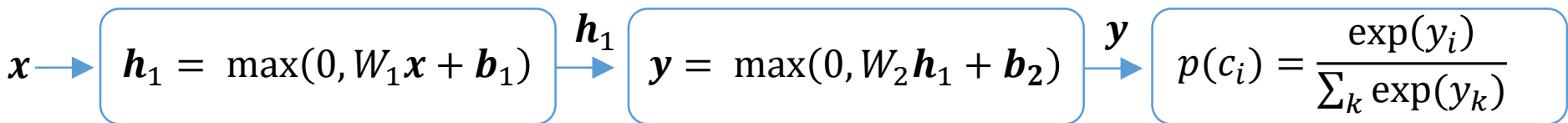
Algorithm (Gradient Descent)

1. Initialize weights $\mathbf{w}_0$ (small random noise)
2. Iterate K -times
 3. For each example n
 2. Forward pass to compute $\mathbf{h}_i, \mathbf{y}, p(\mathbf{c})$
 3. Backward pass to compute the gradient $\nabla_{\mathbf{w}} \ell(\mathbf{w}, \mathbf{x}^n, \mathbf{t}^n)$
 4. Average the gradient over all examples:

$$\nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w}_k) = \frac{1}{N} \sum_{n=1}^N \nabla_{\mathbf{w}} \ell(\mathbf{w}, \mathbf{x}^n, \mathbf{t}^n) + \nabla_{\mathbf{w}} \mathcal{R}(\mathbf{w})$$

5. Apply Gradient descent step $\mathbf{w}_{k+1} = \mathbf{w}_k - \gamma_k \nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w}_k)$

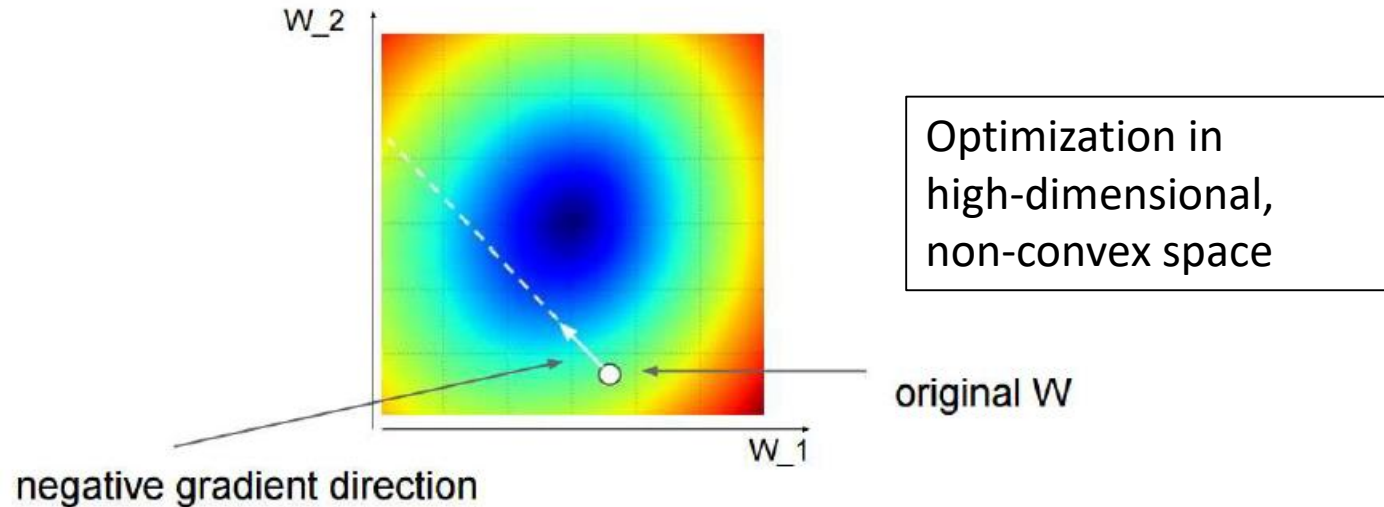
Learning Neural Networks



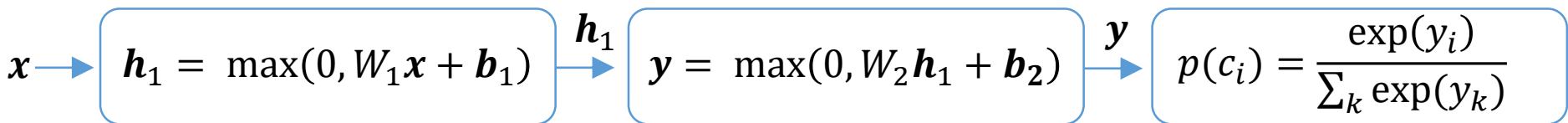
$$\mathcal{L}(\mathbf{w}) = \frac{1}{N} \sum_{n=1}^N \ell(\mathbf{w}, \mathbf{x}^n, \mathbf{t}^n) + \mathcal{R}(\mathbf{w})$$

- x is the input, $p(\mathbf{c})$ is the output
- $\mathbf{h}_i$ is the i -th hidden layer output
- $W_i, \mathbf{b}_i$ are the parameters of the i -th layer
- all weights will be denoted by $\mathbf{w} = \{W_i, \mathbf{b}_i\}$

Algorithm (Gradient Descent)



Learning Neural Networks



$$\mathcal{L}(\mathbf{w}) = \frac{1}{N} \sum_{n=1}^N \ell(\mathbf{w}, \mathbf{x}^n, \mathbf{t}^n) + \mathcal{R}(\mathbf{w})$$

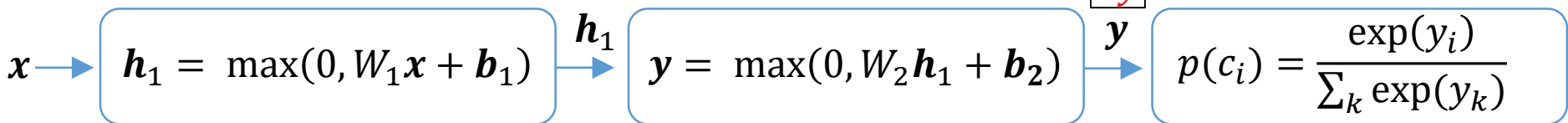
- $\mathbf{x}$ is the input, $p(\mathbf{c})$ is the output
- $\mathbf{h}_i$ is the i -th hidden layer output
- $W_i, \mathbf{b}_i$ are the parameters of the i -th layer
- all weights will be denoted by $\mathbf{w} = \{W_i, \mathbf{b}_i\}$

Algorithm (Gradient Descent)

1. Initialize weights $\mathbf{w}_0$ (small random noise)
2. Iterate K -times
 3. For each example n
 2. Forward pass to compute $\mathbf{h}_i, \mathbf{y}, p(\mathbf{c})$
 3. Backward pass to compute the gradient $\nabla_{\mathbf{w}} \ell(\mathbf{w}, \mathbf{x}^n, \mathbf{t}^n)$
 4. Average the gradient over all examples:

$$\nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w}_k) = \frac{1}{N} \sum_{n=1}^N \nabla_{\mathbf{w}} \ell(\mathbf{w}, \mathbf{x}^n, \mathbf{t}^n) + \nabla_{\mathbf{w}} \mathcal{R}(\mathbf{w})$$

5. Apply Gradient descent step $\mathbf{w}_{k+1} = \mathbf{w}_k - \gamma_k \nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w}_k)$



$$\mathcal{L}(\mathbf{w}) = \frac{1}{N} \sum_{n=1}^N \ell(\mathbf{w}, \mathbf{x}^n, \mathbf{t}^n) + \mathcal{R}(\mathbf{w})$$

- x is the input, $p(\mathbf{c})$ is the output
- h_i is the i -th hidden layer output
- W_i, b_i are the parameters of the i -th layer
- all weights will be denoted by $\mathbf{w} = \{W_i, b_i\}$

- Efficient computation of the gradients by applying the chain rule
- Compute the derivative of loss w.r.t the output:
 - $\ell(\mathbf{x}, \mathbf{t}, \mathbf{w}) = -\sum_j t_j \log p(c_j | \mathbf{x})$
 - $p(c_j | \mathbf{x}) = \frac{\exp(y_j)}{\sum_{k=1}^C \exp(y_k)}$
 - $\frac{\partial \ell}{\partial y} = p(\mathbf{c} | \mathbf{x}) - \mathbf{t}$
- Note that the **forward pass** is necessary to compute $p(\mathbf{c} | \mathbf{x})$

Learning Neural Networks – side calculation

$$\ell(x, t, w) = - \sum_i t^i \log p(c; |x)$$

$$p(c; |x) = \frac{\exp(y_i)}{\sum_{k=1}^K \exp(y_k)}$$

$$\frac{\partial \ell(x, t, w)}{\partial y} = \frac{\partial - \sum_i t^i \log p(c; |x)}{\partial y} \quad (k\text{-dim vector})$$

take y_1 :

Insert p

$$\log(a/b) = \log(a) - \log(b)$$

$$\frac{\partial - \sum_i t^i \log p(c; |x)}{\partial y_1} = \frac{\partial - \sum_i (t^i y_i - t^i \log(\sum_{k=1}^K \exp(y_k)))}{\partial y_1}$$

Deriv. y_1

$$= -t^1 + \frac{\partial \sum_i t^i \log(\sum_{k=1}^K \exp(y_k))}{\partial y_1}$$

Log() indep. of i

$$= -t^1 + \frac{\partial \log(\sum_{k=1}^K \exp(y_k)) \cdot (\sum_i t^i)}{\partial y_1}$$

Chain rule. Deriv. $\log(x) = 1/x$. Deriv. $\exp(x) = \exp(x)$

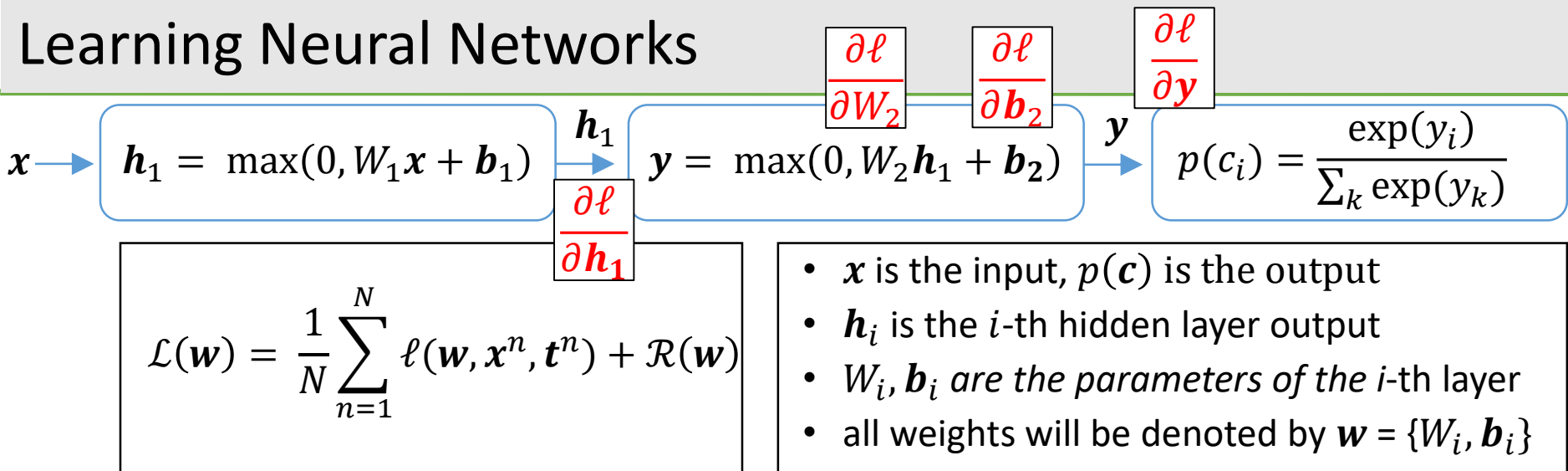
$$= -t^1 + \frac{1}{\sum_{k=1}^K \exp(y_k)} \cdot \exp(y_1) = p(c_1 | x) - t^1$$

See def. $p(x)$

$\leadsto$

$$\frac{\partial \ell(x, t, w)}{\partial y} = p(c | x) - t \quad (k\text{-dim vector})$$

Learning Neural Networks



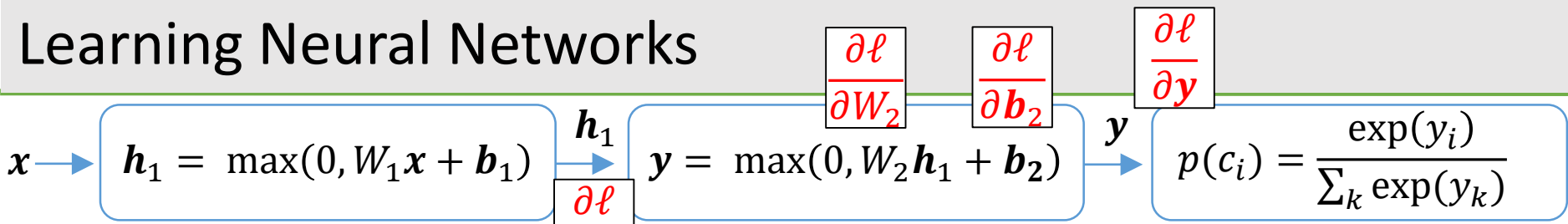
- We want:

- $\frac{\partial \ell}{\partial W_2}$ and $\frac{\partial \ell}{\partial b_2}$ are necessary derivatives
- $\frac{\partial \ell}{\partial h_1}$ is needed for the derivatives wrt W_1 and b_1

- We need:

- $\mathbf{h}_1$ from forward pass
- $\frac{\partial \ell}{\partial y}$ is sufficient since we use Chain rule.

Learning Neural Networks



$$\mathcal{L}(\mathbf{w}) = \frac{1}{N} \sum_{n=1}^N \ell(\mathbf{w}, \mathbf{x}^n, \mathbf{t}^n) + \mathcal{R}(\mathbf{w})$$

- x is the input, $p(\mathbf{c})$ is the output
- $\mathbf{h}_i$ is the i -th hidden layer output
- $W_i, \mathbf{b}_i$ are the parameters of the i -th layer
- all weights will be denoted by $\mathbf{w} = \{W_i, \mathbf{b}_i\}$

$$\ell(\mathbf{y}) = - \sum_j t_j \log \left[\frac{\exp(y_j)}{\sum_{k=1}^C \exp(y_k)} \right]$$

$$\frac{\partial f(g(x))}{\partial x} = \frac{\partial f(y)}{\partial y} \cdot \frac{\partial g(x)}{\partial x} \quad y = g(x)$$

$$\mathbf{y} = \max(0, W_2 \mathbf{h}_1 + \mathbf{b}_2)$$

$$\ell(\mathbf{h}_1, \mathbf{b}_2, W_2) = - \sum_j t_j \log \left[\frac{\exp([\max(0, W_2 \mathbf{h}_1 + \mathbf{b}_2)]_j)}{\sum_{k=1}^C \exp([\max(0, W_2 \mathbf{h}_1 + \mathbf{b}_2)]_k)} \right]$$

Example derivative wrt $\mathbf{h}_1$:

$$\frac{\partial \ell(\mathbf{h}_1, \mathbf{b}_2, W_2)}{\partial \mathbf{h}_1} = \frac{\partial \ell(\mathbf{y})}{\partial \mathbf{y}} * \frac{\partial \mathbf{y}}{\partial \mathbf{h}_1}$$

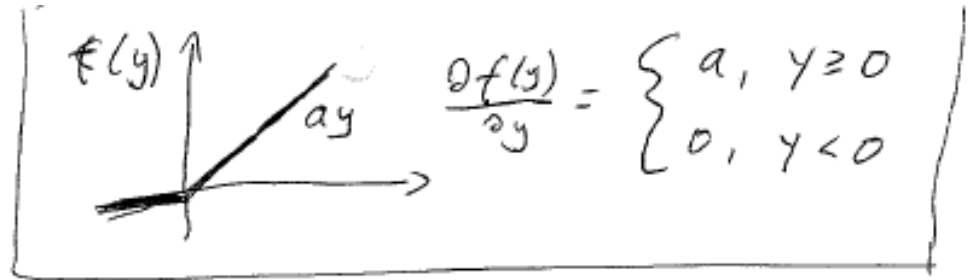
$$= (p(\mathbf{c}|\mathbf{x}) - \mathbf{t}) * \frac{\partial \max(0, W_2 \mathbf{h}_1 + \mathbf{b}_2)}{\partial \mathbf{h}_1}$$

Learning Neural Networks

$$1) \frac{\partial \ell}{\partial y} = p(c|x) - t$$

vector

$$y = \max(0, w_2 h_1 + b_2)$$



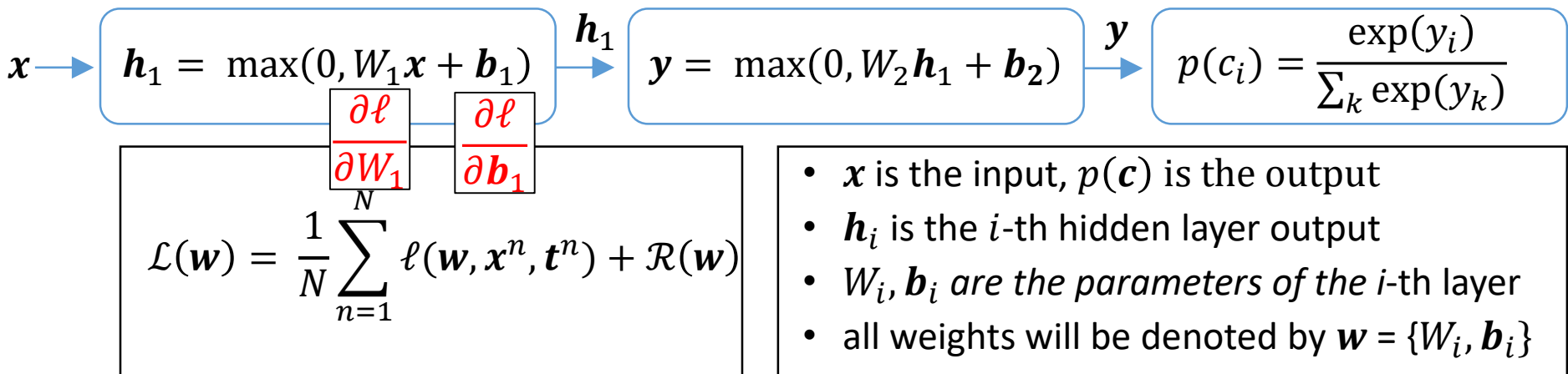
$$2) \underbrace{\frac{\partial \ell}{\partial b_2}}_{\text{vector}} = \underbrace{\frac{\partial \ell(y)}{\partial y}}_{\text{vector}} \cdot \underbrace{\frac{\partial y}{\partial b_2}}_{\text{matrix with potentially many 0's}} = p(c|x) - t \cdot \frac{\partial (\max(0, w_2 h_1 + b_2))}{\partial b_2}$$

chain rule

$$3) \underbrace{\frac{\partial \ell}{\partial w_2}}_{\text{matrix}} = \underbrace{\frac{\partial \ell(y)}{\partial y}}_{\text{vector}} \cdot \underbrace{\frac{\partial y}{\partial w_2}}_{\substack{\text{3D tensor with 0's}}} = p(c|x) - t \cdot \frac{\partial (\max(0, w_2 h_1 + b_2))}{\partial w_2}$$

$$4) \underbrace{\frac{\partial \ell}{\partial h_1}}_{\text{vector}} = \underbrace{\frac{\partial \ell(y)}{\partial y}}_{\text{vector}} \cdot \underbrace{\frac{\partial y}{\partial h_1}}_{\text{matrix with 0's}} = p(c|x) - t \cdot \frac{\partial (\max(0, w_2 h_1 + b_2))}{\partial h_1}$$

Learning Neural Networks



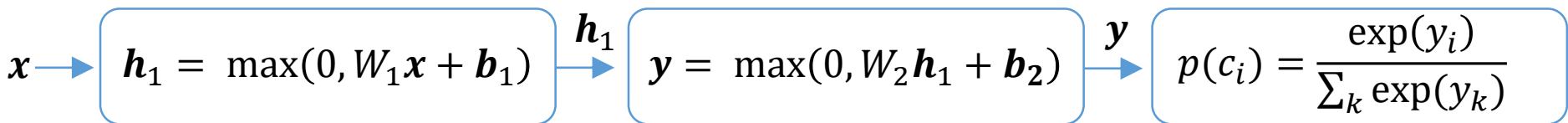
- We want:
 - $\frac{\partial \ell}{\partial W_1}$ and $\frac{\partial \ell}{\partial b_1}$ are necessary derivatives
- We need:
 - Input x
 - $\frac{\partial \ell}{\partial h_1}$ is sufficient since we use Chain rule

Learning Neural Networks

$$1) \underbrace{\frac{\partial \mathcal{L}}{\partial w_1}}_{\text{matrix}} = \frac{\partial \mathcal{L}}{\partial h_1} \cdot \frac{\partial h_1}{\partial w_1} = \underbrace{\frac{\partial \mathcal{L}}{\partial h_1}}_{\text{vector}} \cdot \underbrace{\frac{\partial (\max(0, w_1 x + b_1))}{\partial w_1}}_{\text{3D Ten sor with 0's}}$$

$$2) \underbrace{\frac{\partial \mathcal{L}}{\partial b_1}}_{\text{vector}} = \frac{\partial \mathcal{L}}{\partial h_1} \cdot \frac{\partial h_1}{\partial b_1} = \underbrace{\frac{\partial \mathcal{L}}{\partial h_1}}_{\text{vector}} \cdot \underbrace{\frac{\partial (\max(0, w_1 x + b_1))}{\partial b_1}}_{\text{matrix}}$$

Learning Neural Networks



$$\mathcal{L}(\mathbf{w}) = \frac{1}{N} \sum_{n=1}^N \ell(\mathbf{w}, \mathbf{x}^n, \mathbf{t}^n) + \mathcal{R}(\mathbf{w})$$

- x is the input, $p(\mathbf{c})$ is the output
- $\mathbf{h}_i$ is the i -th hidden layer output
- $W_i, \mathbf{b}_i$ are the parameters of the i -th layer
- all weights will be denoted by $\mathbf{w} = \{W_i, \mathbf{b}_i\}$

Algorithm (Gradient Descent)

1. Initialize weights $\mathbf{w}_0$ (small random noise)
2. Iterate K -times
 3. For each example n
 2. Forward pass to compute $\mathbf{h}_i, \mathbf{y}, p(\mathbf{c})$
 3. Backward pass to compute the gradient $\nabla_{\mathbf{w}} \ell(\mathbf{w}, \mathbf{x}^n, \mathbf{t}^n)$
 4. Average the gradient over all examples:

$$\nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w}_k) = \frac{1}{N} \sum_{n=1}^N \nabla_{\mathbf{w}} \ell(\mathbf{w}, \mathbf{x}^n, \mathbf{t}^n) + \nabla_{\mathbf{w}} \mathcal{R}(\mathbf{w})$$

5. Apply Gradient descent step $\mathbf{w}_{k+1} = \mathbf{w}_k - \gamma_k \nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w}_k)$

Learning Neural Networks

- We need to compute at each iteration

$$\mathbf{w}_{k+1} = \mathbf{w}_k - \gamma_k \nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w}_k)$$

with

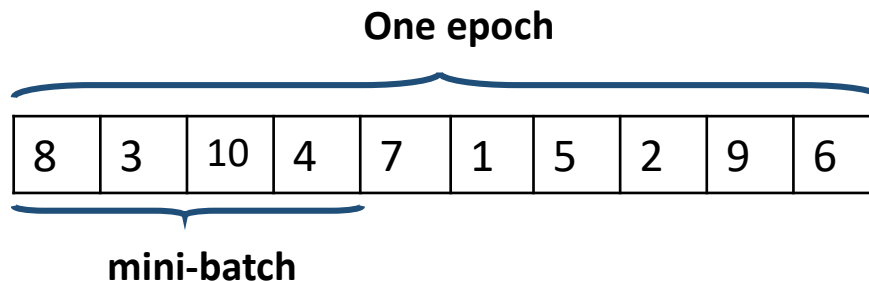
$$\nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w}_k) = \frac{1}{N} \sum_{n=1}^N \nabla_{\mathbf{w}} \ell^n(\mathbf{w}, \mathbf{x}^n, \mathbf{t}^n) + \nabla_{\mathbf{w}} \mathcal{R}(\mathbf{w})$$

- Too expensive when having millions of examples
- Instead, we approximate the gradient for a subset of examples (mini-batch).

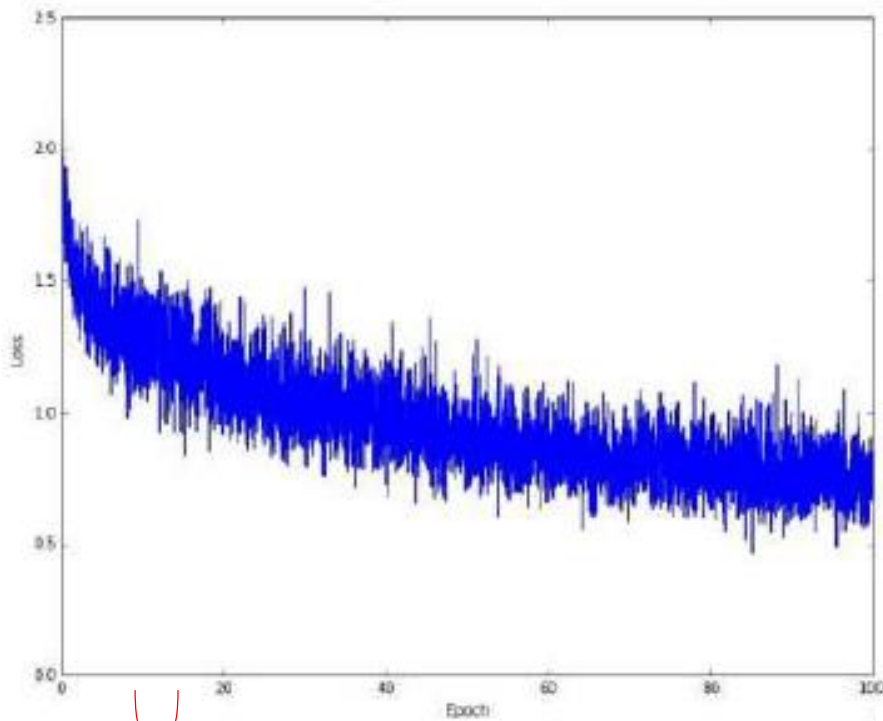
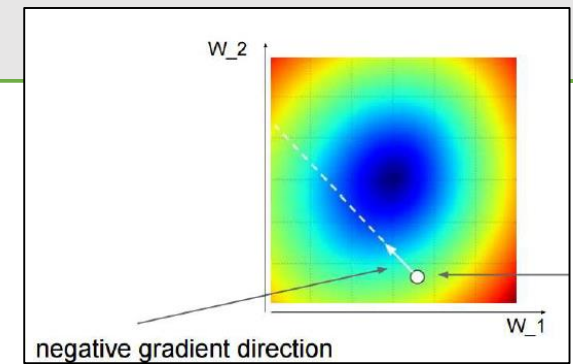
This is called **Stochastic Gradient Descent (SGD)**

$$\frac{1}{N} \sum_{n=1}^N \nabla_{\mathbf{w}} \ell^n(\mathbf{w}, \mathbf{x}^n, \mathbf{t}^n) \approx 1/|S| \sum_{n \in S} \nabla_{\mathbf{w}} \ell^n(\mathbf{w}, \mathbf{x}^n, \mathbf{t}^n)$$

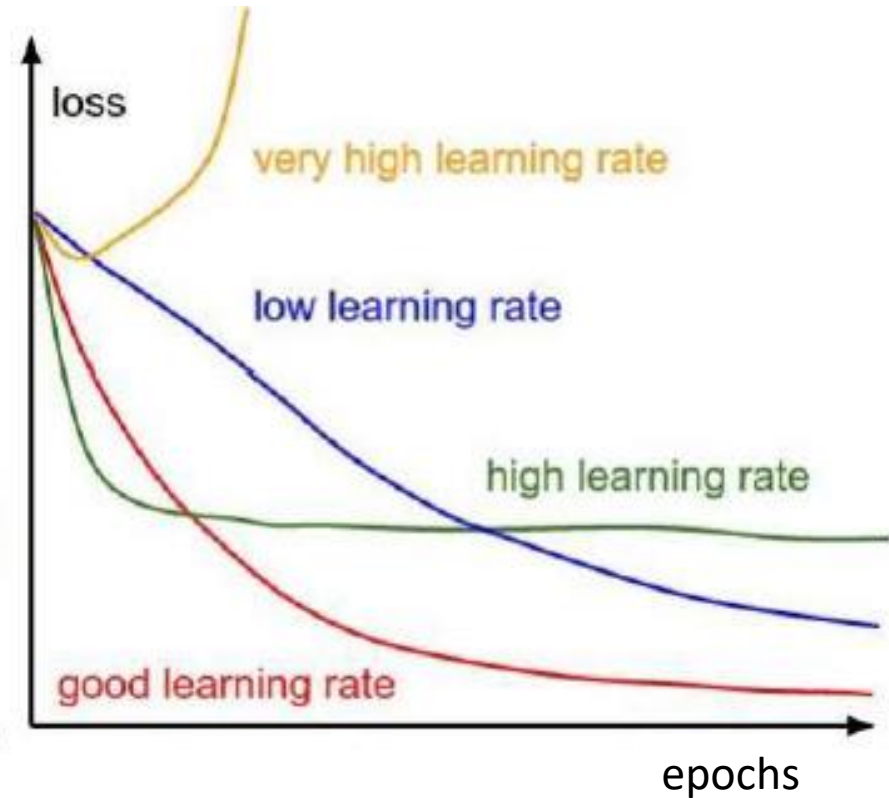
- Randomly shuffle the order of training examples:



Learning Neural Networks



One epoch, i.e. one pass
over all training examples N



The effect of step size (or „learning rate“)

Regularization

Weight Regularization

$$\mathcal{L}(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^N \ell(\mathbf{w}, \mathbf{x}^i, t^i) + \mathcal{R}(\mathbf{w})$$

In common use:

L2 regularization

$$R(W) = \sum_k \sum_l W_{k,l}^2$$

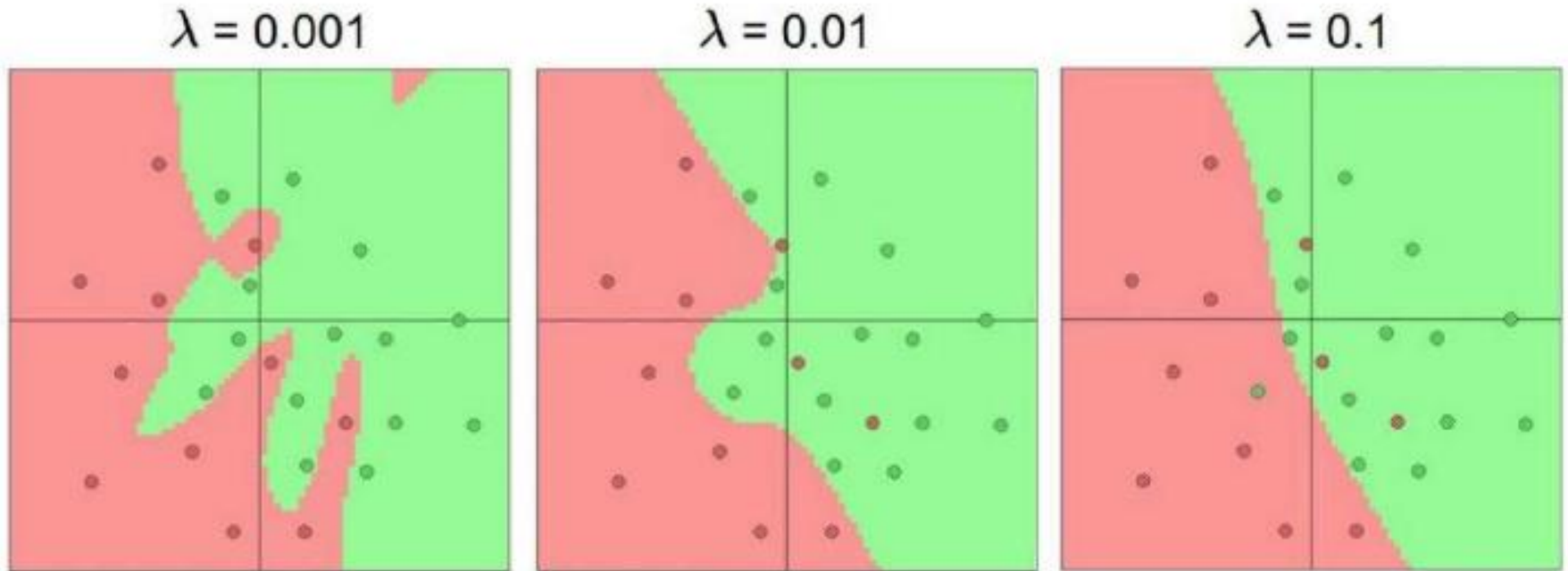
L1 regularization

$$R(W) = \sum_k \sum_l |W_{k,l}|$$

Elastic net (L1 + L2)

$$R(W) = \sum_k \sum_l \beta W_{k,l}^2 + |W_{k,l}|$$

Regularization



$$\mathcal{L}(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^N \ell(\mathbf{w}, \mathbf{x}^i, t^i) + \lambda \sum_k \mathbf{w}_k^2$$

Regularization

Weight Regularization

$$\mathcal{L}(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^N \ell(\mathbf{w}, \mathbf{x}^i, t^i) + \mathcal{R}(\mathbf{w})$$

In common use:

L2 regularization

$$R(W) = \sum_k \sum_l W_{k,l}^2$$

L1 regularization

$$R(W) = \sum_k \sum_l |W_{k,l}|$$

Elastic net (L1 + L2)

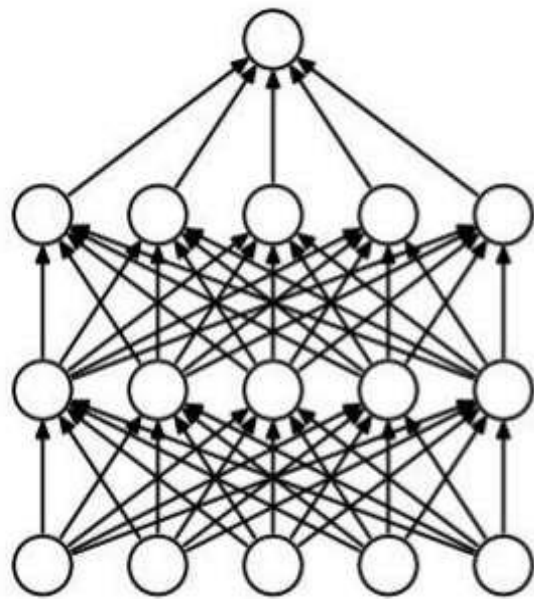
$$R(W) = \sum_k \sum_l \beta W_{k,l}^2 + |W_{k,l}|$$

In general, we have different options to get good generalization:
(remember decision boundaries)

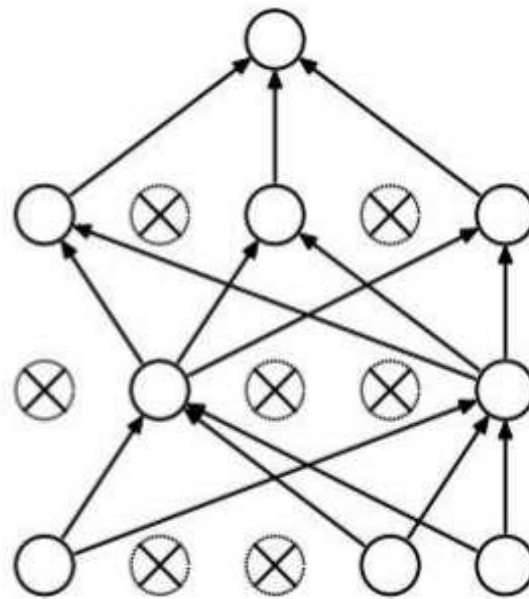
1. Control complexity of model
(e.g. number of layers; depth of decision tree)
2. Weight regularization
3. Ensemble methods
(e.g. decision forest, ensemble of neural networks)
4. Dropout

Regularization with Dropout during training

“randomly set some neurons to zero in the forward pass”



(a) Standard Neural Net



(b) After applying dropout.

[Srivastava et al., 2014]

Other options:

- Weight Dropout
- Layer Dropout

Regularization with Dropout

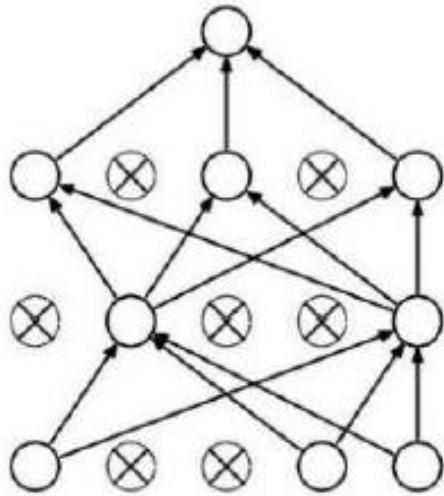
Waaaaait a second...

How could this possibly be a good idea?

Regularization with Dropout

Waaaaait a second...

How could this possibly be a good idea?



Forces the network to have a redundant representation.

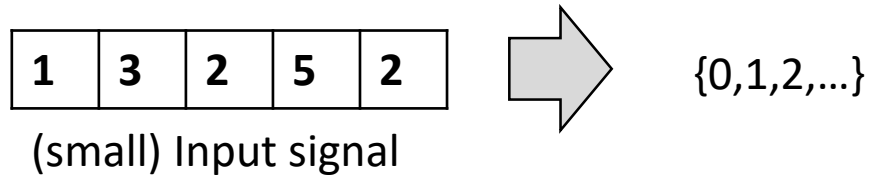


Roadmap

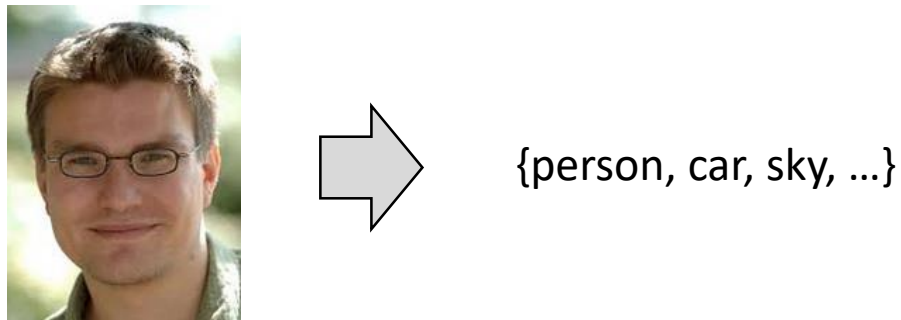
- Types of Machine Learning Problems
- Decision Trees
- Deep Learning: Discriminative Models
 - Neural Networks
 - Convolutional Neural Networks
- Image Classification: ResNet
- Understanding Image Classification Networks
- Semantic Segmentation: U-Net
- Active and Semi-Supervised Learning

Different Tasks

- Neural Network (small input)

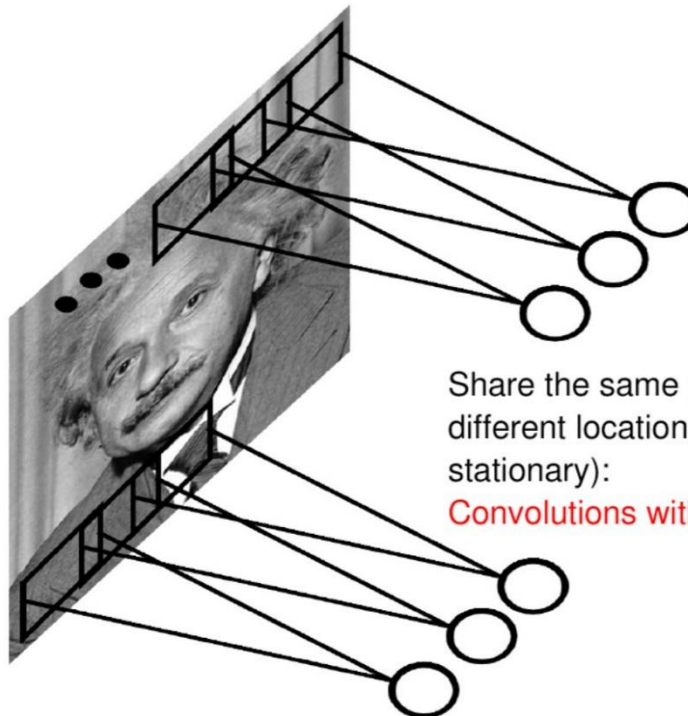
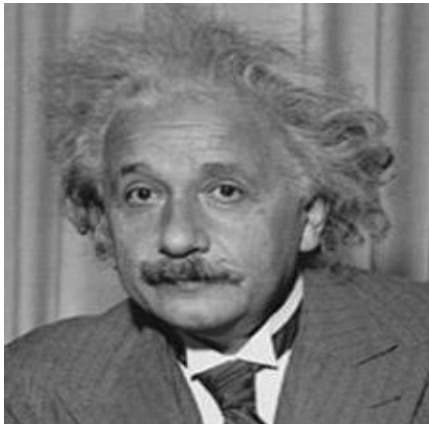


- Convolutional Neural Network (large input)



How to deal with large Input Spaces?

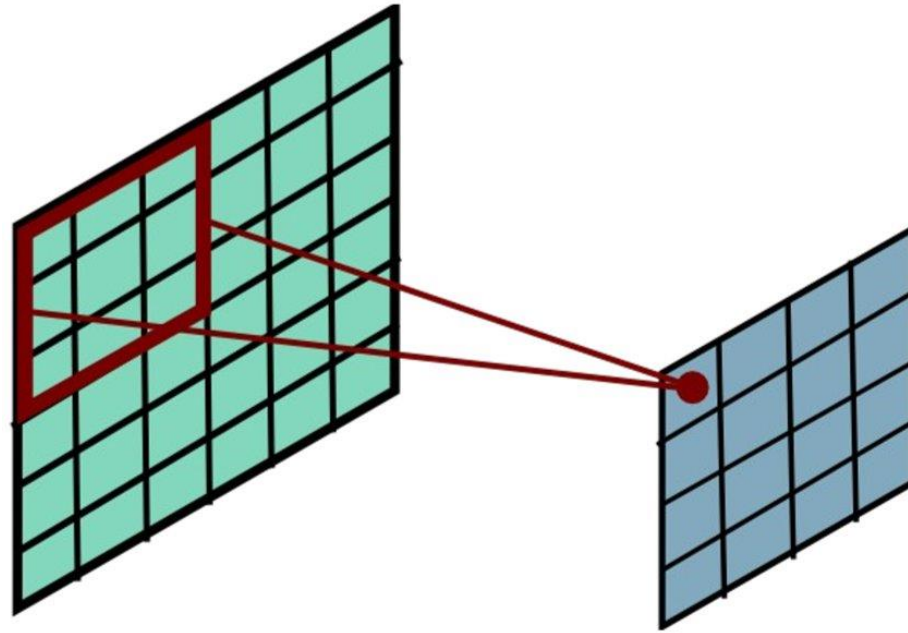
- Images can have millions of pixels, i.e. x is very high dimensional
 - Prohibitive to have fully-connected layer
- **Idea:** Statistics are similar at different locations (LeCun 1998)
 - We can use a **locally connected layer (via a patch-filter)**
 - The weights of the patch-filter are the same across image space
 - This is called a convolution layer and the network is a **convolutional network**



Share the same parameters across different locations (assuming input is stationary):

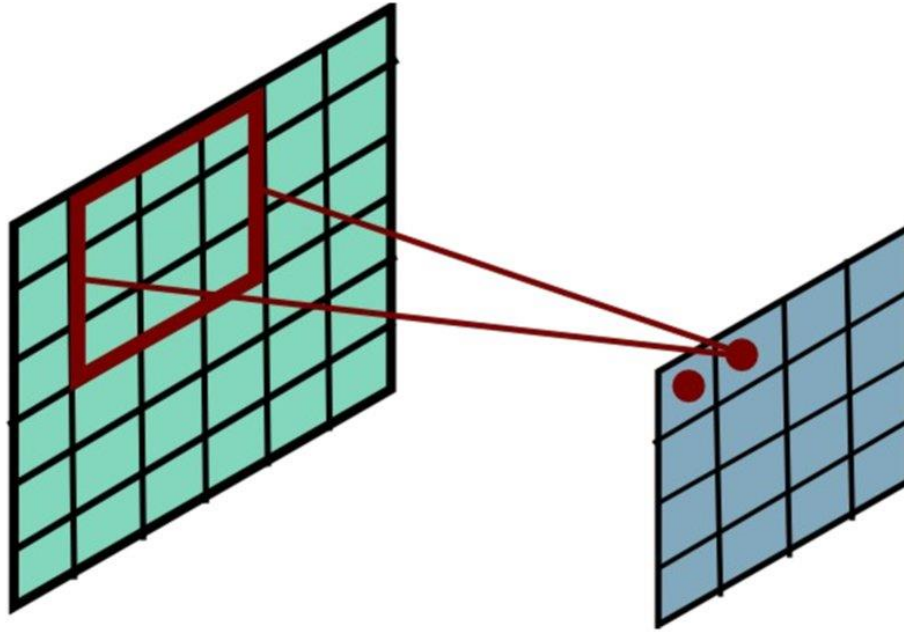
Convolutions with learned kernels

Convolutional Layer



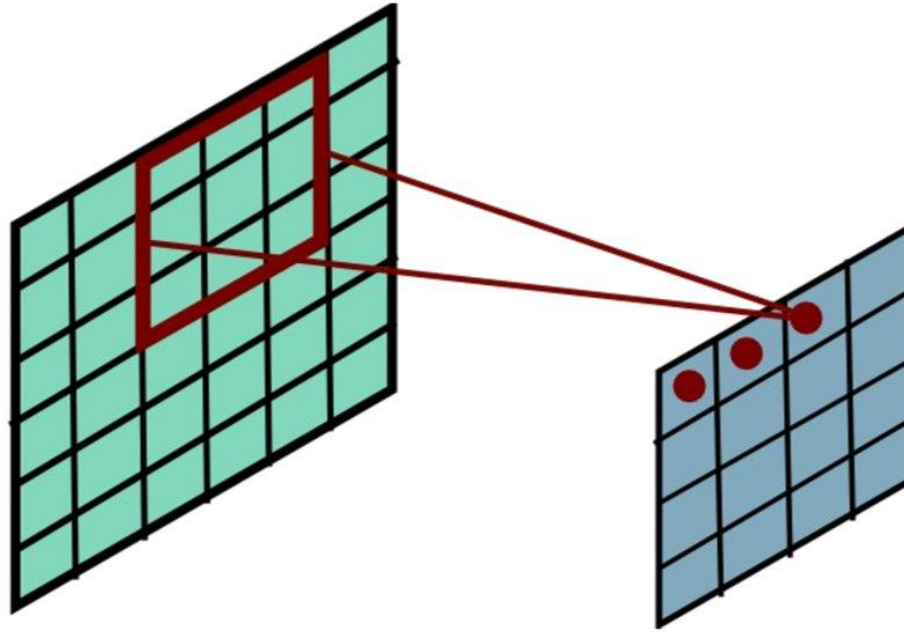
$$h_j^n = \max(0, \sum_{k=1}^K h_k^{n-1} * w_{jk}^n)$$

Convolutional Layer



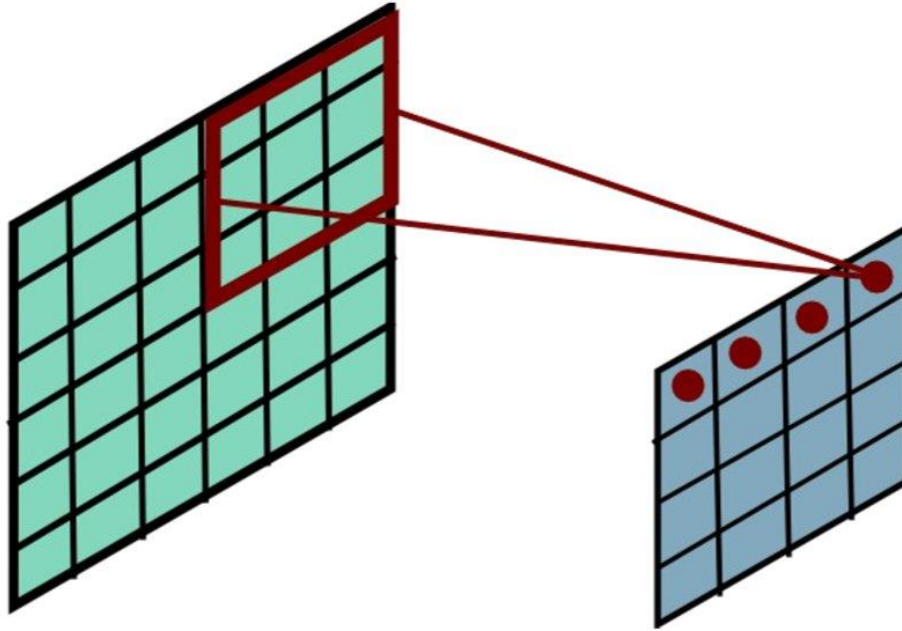
$$h_j^n = \max(0, \sum_{k=1}^K h_k^{n-1} * w_{jk}^n)$$

Convolutional Layer



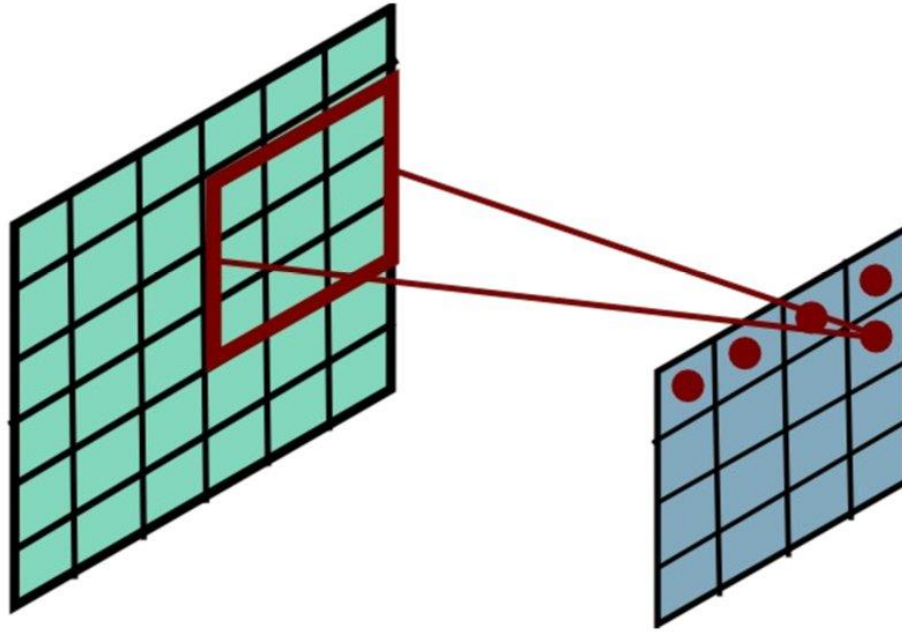
$$h_j^n = \max(0, \sum_{k=1}^K h_k^{n-1} * w_{jk}^n)$$

Convolutional Layer



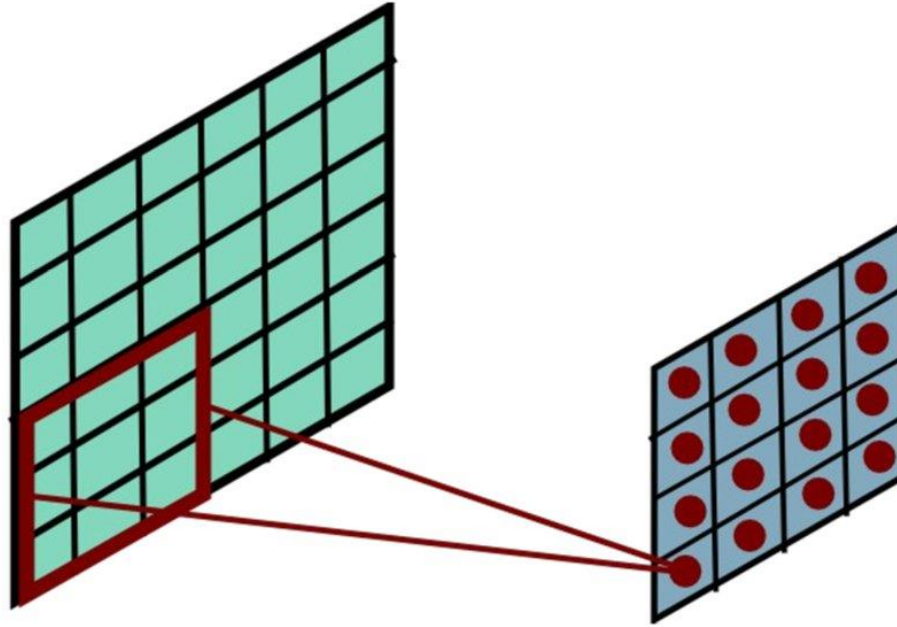
$$h_j^n = \max(0, \sum_{k=1}^K h_k^{n-1} * w_{jk}^n)$$

Convolutional Layer



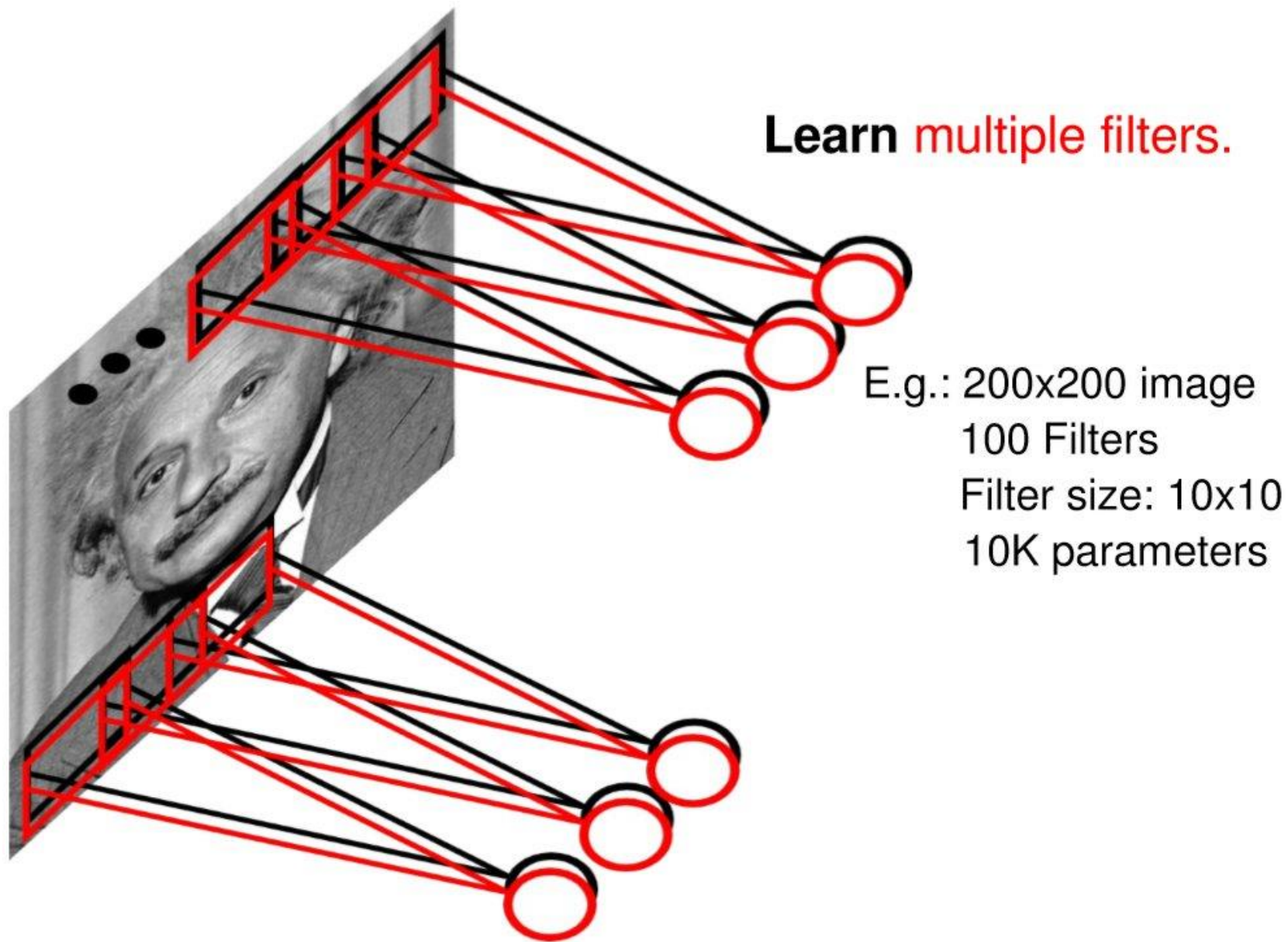
$$h_j^n = \max(0, \sum_{k=1}^K h_k^{n-1} * w_{jk}^n)$$

Convolutional Layer



$$h_j^n = \max(0, \sum_{k=1}^K h_k^{n-1} * w_{jk}^n)$$

Convolutional Layer



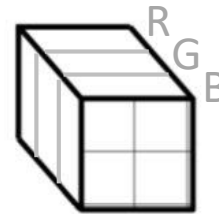
Convolutional Neural Networks (CNN)

- Compare to Image Processing Basics

Input "image"



Filter



Convolutional Neural Networks (CNN)

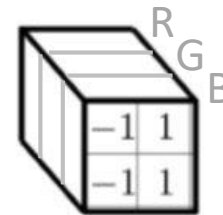
- If our filter was $[-1, 1]$ for each channel, we got a vertical edge detector

Input "image"

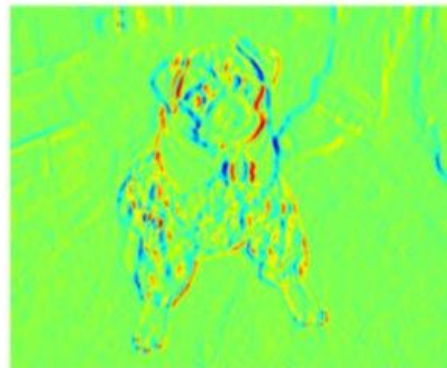


$224 \times 180 \times 3$

Filter



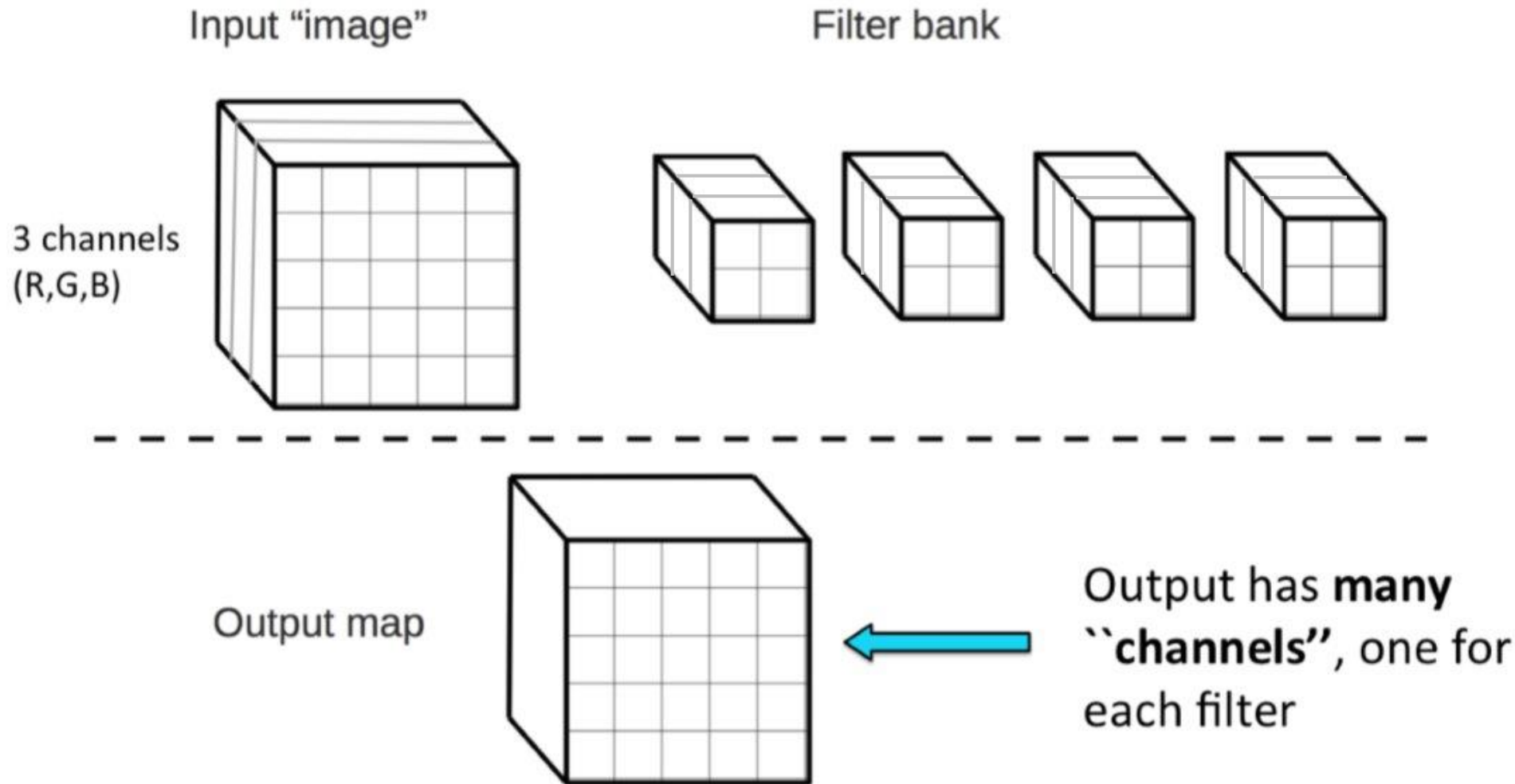
Output map



$224 \times 180 \times 1$

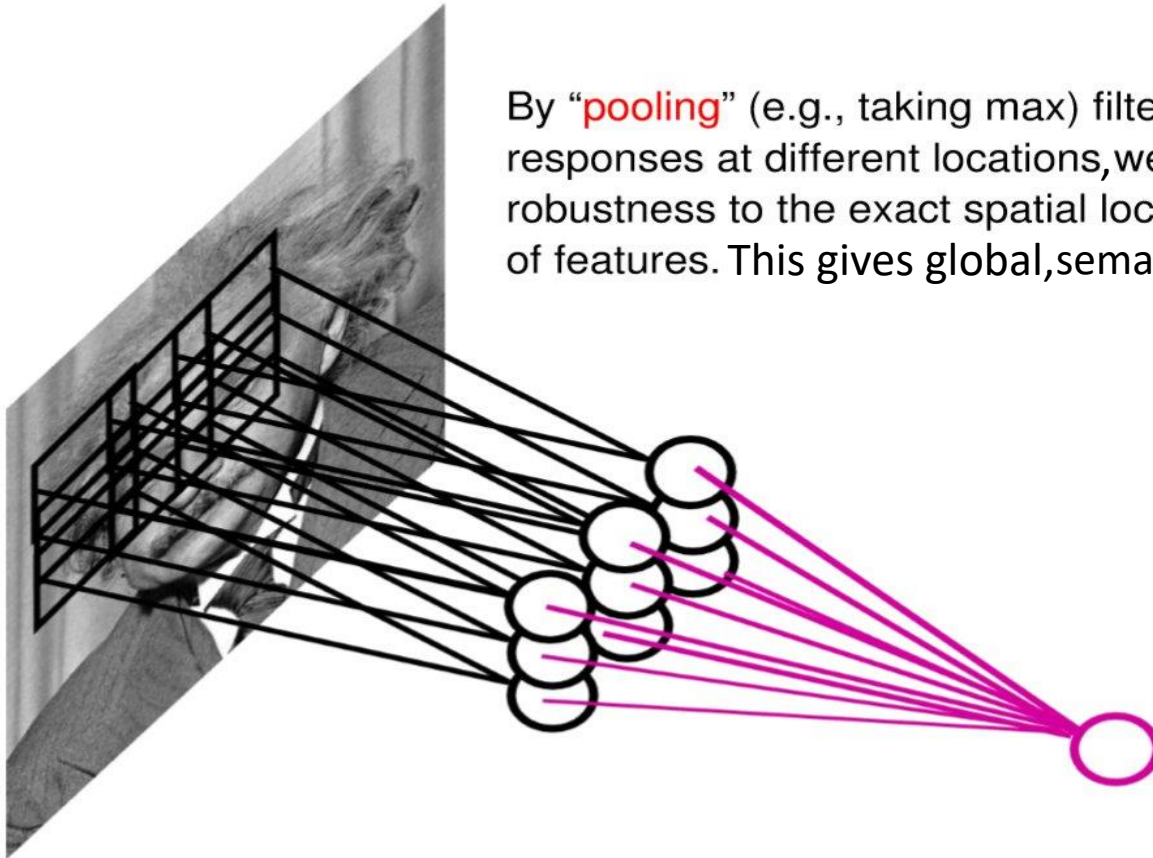
Convolutional Neural Networks (CNN)

- Now a **filterbank** (e.g., vertical, horizontal, corners, one for dots).



Pooling Layer

By “pooling” (e.g., taking max) filter responses at different locations, we gain robustness to the exact spatial location of features. This gives global, semantic reasoning

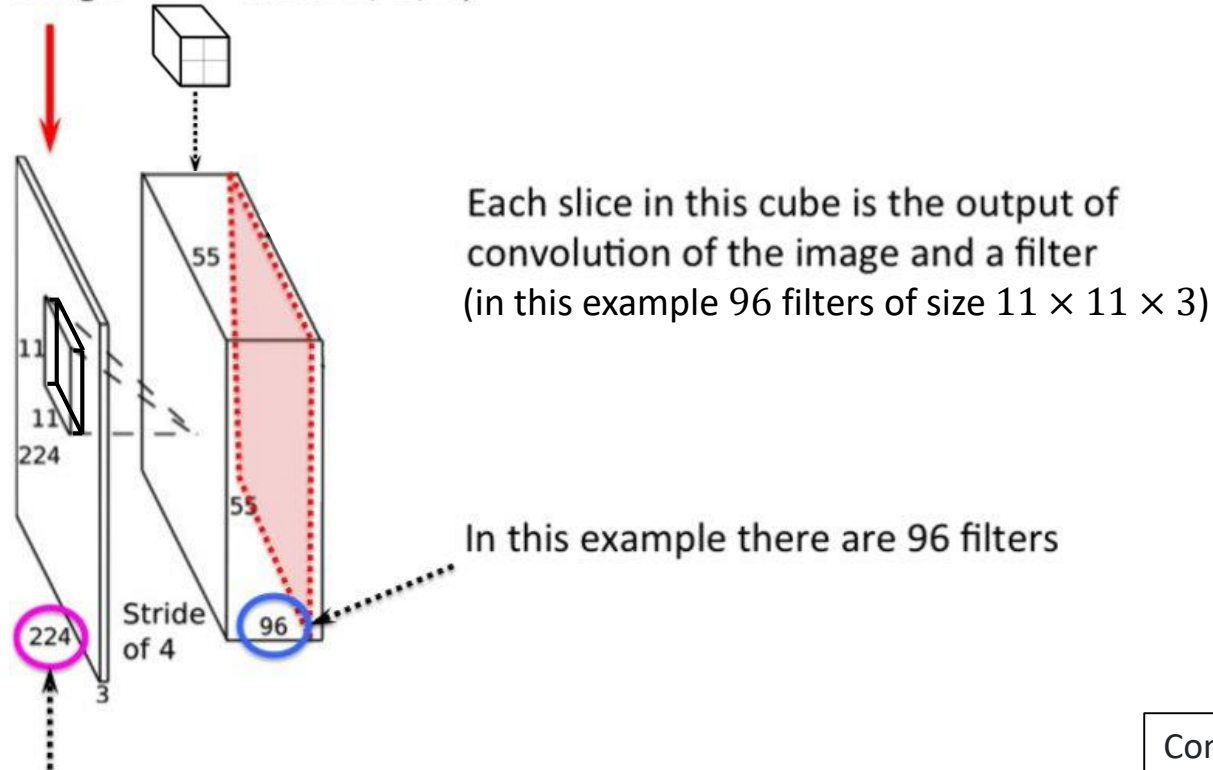


Pooling Options:

- **Max Pooling:** return the maximal argument
- **Average Pooling:** return the average of the arguments
- Other types of pooling exist

Convolutional Neural Networks (CNN)

image (3 channels: R, G, B)

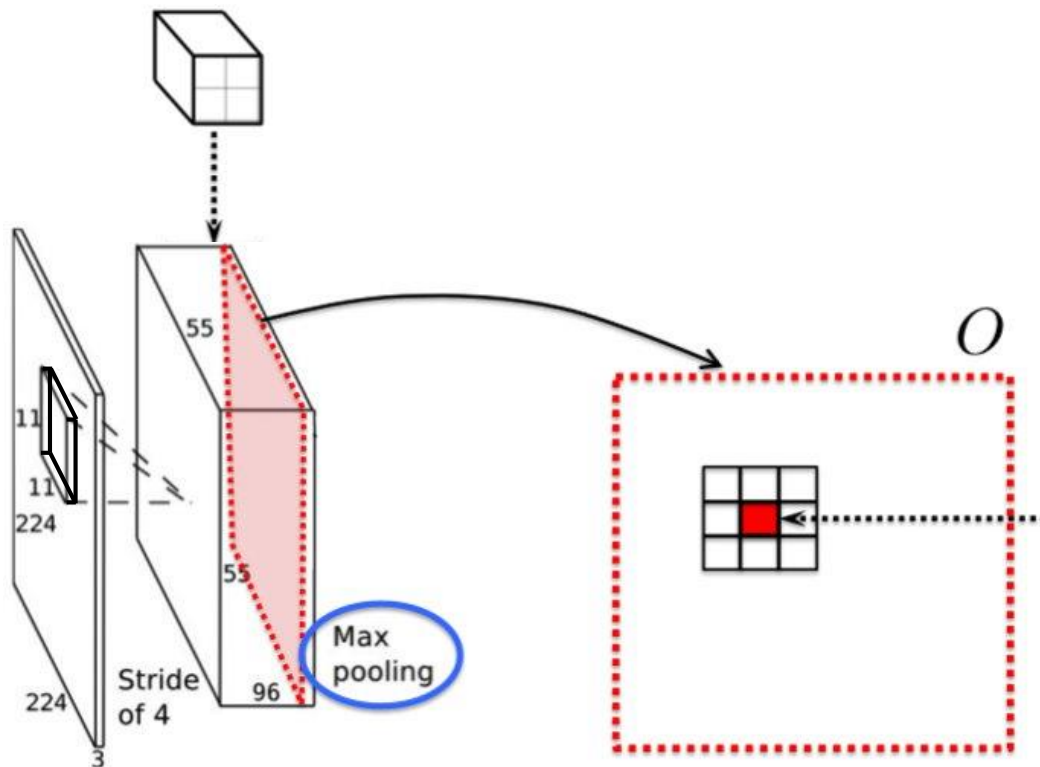


In this example our network will always expect a 224x224x3 image.

Comment. We are now building the famous **AlexNet** [A. Krizhevsky, I. Sutskever, G. E. Hinton. ImageNet Classification with Deep Convolutional Neural Networks. NIPS 2012]

[Slide Credit: Sanja Fidler, Pic adopted from: A. Krizhevsky]

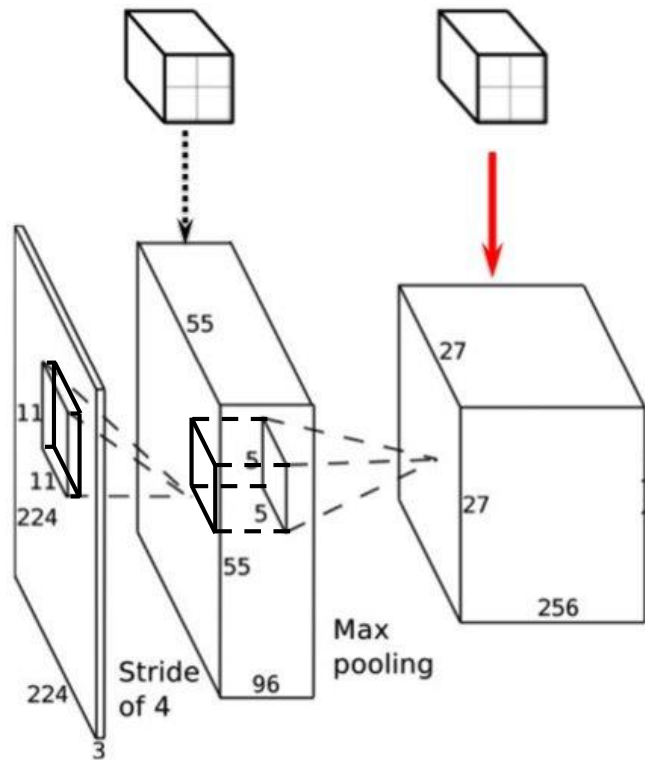
Convolutional Neural Networks (CNN)



$$O(i, j) = \max_{\substack{k \in \{i-1, i, i+1\} \\ l \in \{j-1, j, j+1\}}} O(k, l)$$

Take each slice in the output cube, and in each pixel compute a max over a small patch around it. This is called **max pooling**.

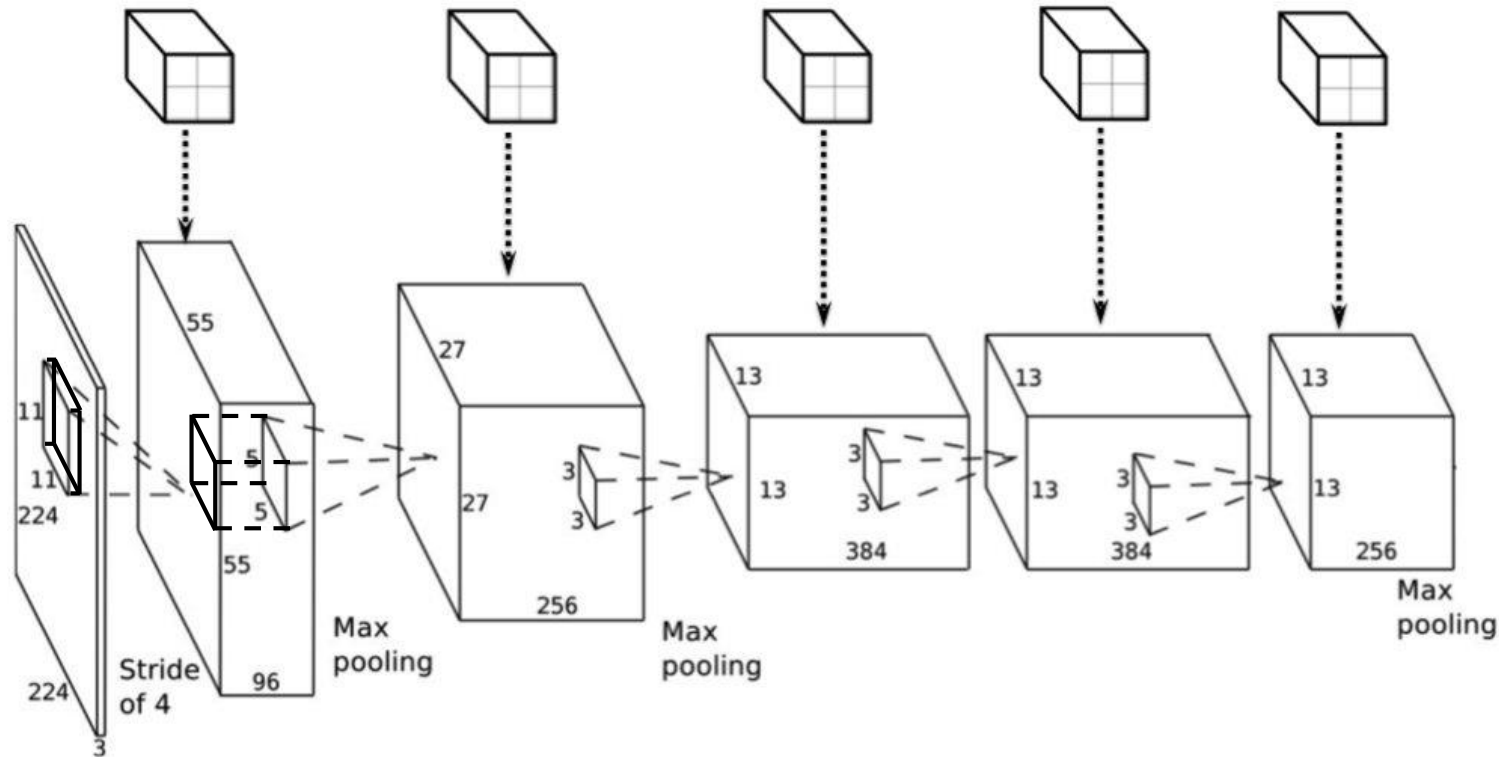
Convolutional Neural Networks (CNN)



Add one more layer of filters

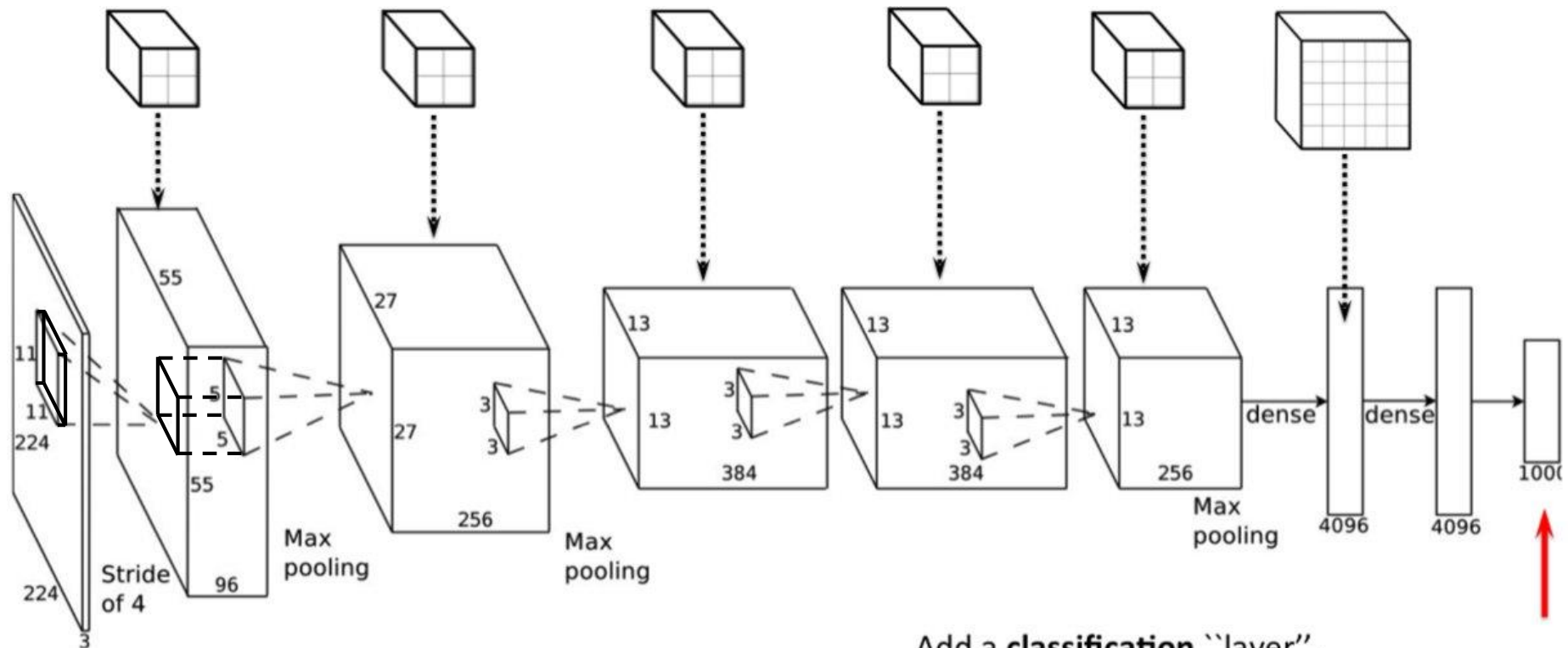
These filters are convolved with the output of the previous layer. The results of each convolution is again a slice in the cube on the right.

Convolutional Neural Networks (CNN)



Let's make it **deep**: repeat many layers

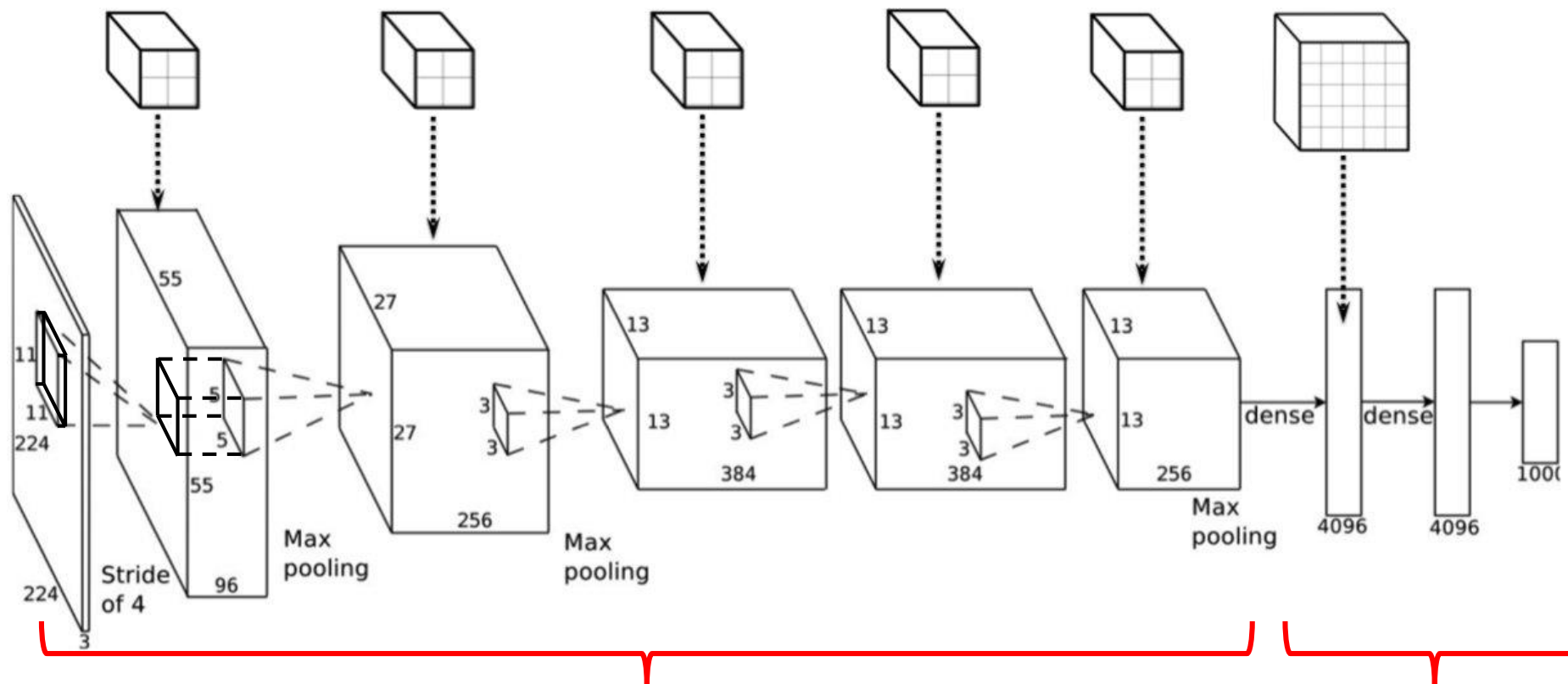
Convolutional Neural Networks (CNN)



Add a **classification** "layer".

For an input image, the value in a particular dimension of this vector tells you the probability of the corresponding object class.

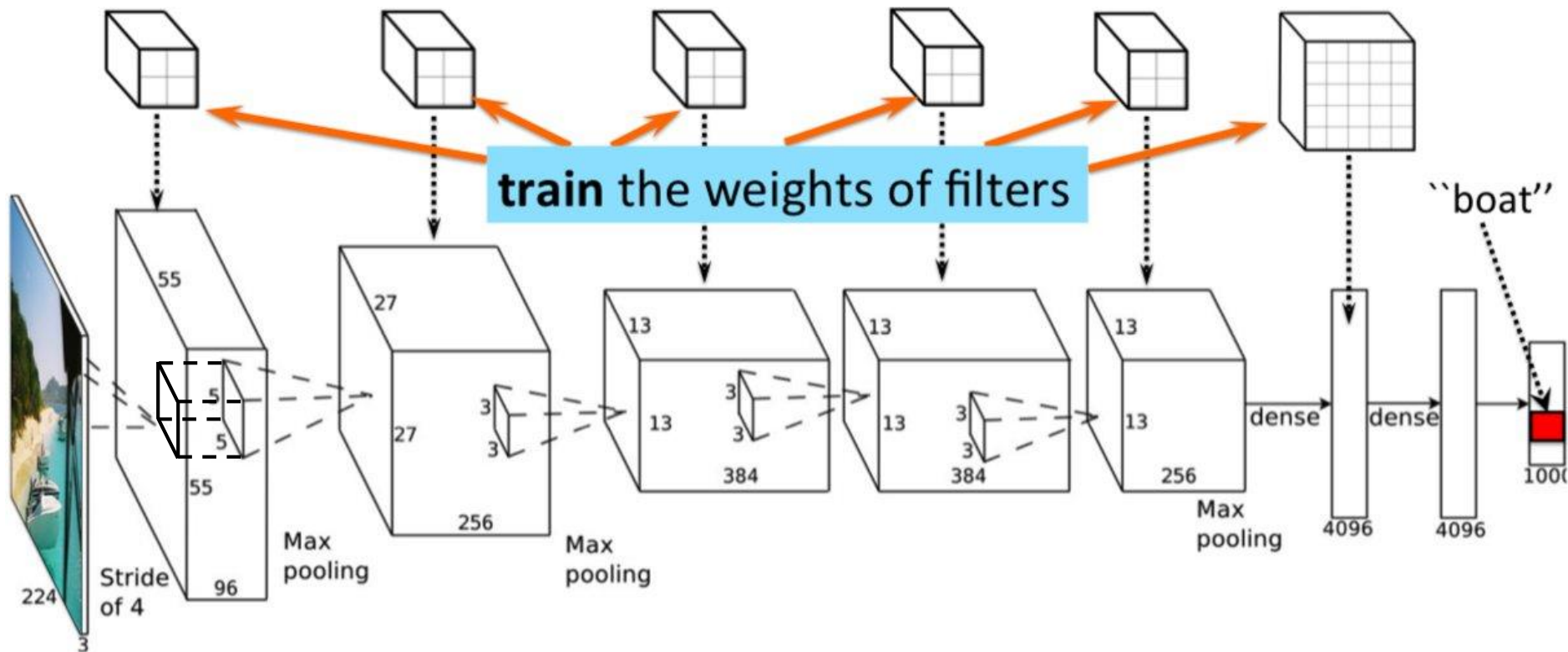
Convolutional Neural Networks (CNN)



Represents a "deep Filterbank"

Represents a classifier

Convolutional Neural Networks (CNN)

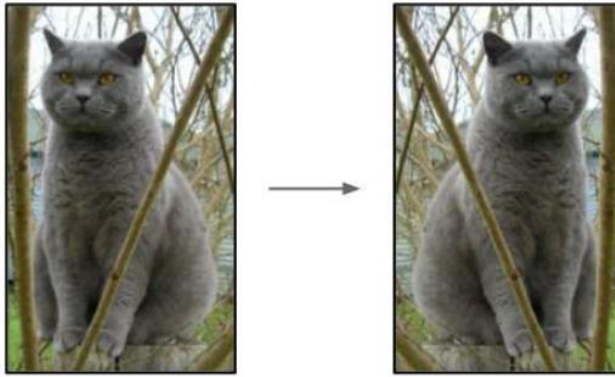


In practice:

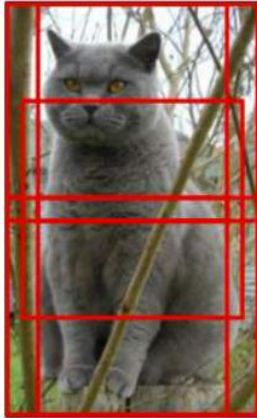
- 1) Use millions of Training Data
- 2) Enrich your Training Data with **Data Augmentation**
- 3) Cut-off the fully-connected layers and re-train for your problem (so-called **pre-training / pre-text tasks**)

Simple Data Augmentation

- Flip



- Random Crop and Scale



- Colour adjustments, etc.

Machine Learning - Variants

Two Types of Models:

1. Discriminative Models: $y = f_w(x)$ or $p(y|x)$
2. Generative Models: $p(x)$, $p(x|y)$, or $p(y, x)$

Variants of Machine Learning Models:

1. *Structured models versus not structured models*
2. ...

Three Classes of Machine Learning Problems:

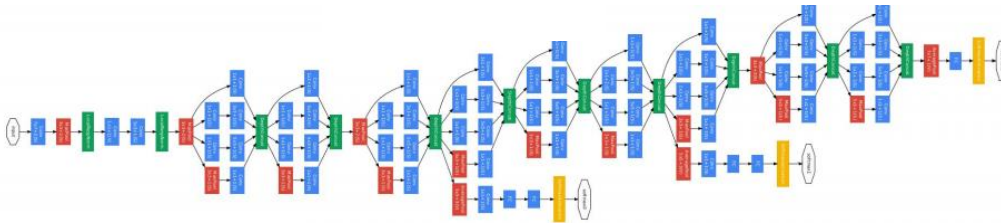
1. **Supervised learning:** fully labelled (annotated) dataset
2. **Unsupervised learning:** no annotations at all (e.g. clustering)
3. **Reinforcement learning:** there is a reward for correct action
(we focus on supervised learning)

Lost of variants of Machine Learning Problems:

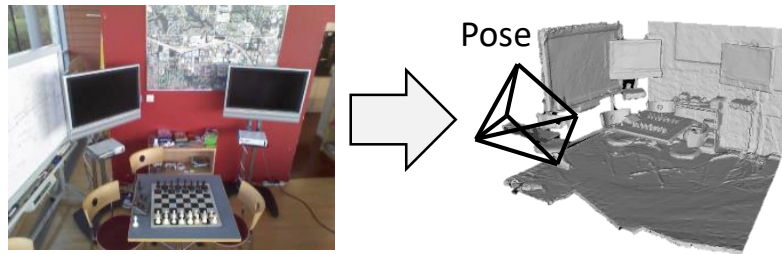
1. On-line learning: Training samples are not available at once
2. Continual Learning: incrementally learning/adapting your model
3. **Self-Supervised:** Use the data itself for learning
4. **Domain adaption:** Train in Domain A and test in Domain B
5. **Pre-Text Tasks:** Train for Task A and fine-tune for task B
6. **Active learning:** What unlabeled data should I label next?
7. **Semi-Supervised learning:** Dataset is partially labelled
8.

Pre-text Tasks: Pre-Training for a new task

1) Pre-train GoogleNet for ImageNet Classification:



2) Fine-tune GoogleNet (with different last layers) for 6D camera Pose estimation:



Research Field: What Pre-Text Task is good for what final Task?

[A. Kendall, M. Grimes and R. Cipolla "PoseNet: A Convolutional Network for Real-Time 6-DOF Camera Relocalization." ICCV 2015.]

- Types of Machine Learning Problems
- Decision Trees
- Deep Learning: Discriminative Models
 - Neural Networks
 - Convolutional Neural Networks
- Image Classification: ResNet
- Understanding Image Classification Networks
- Semantic Segmentation: U-Net
- Active and Semi-Supervised Learning

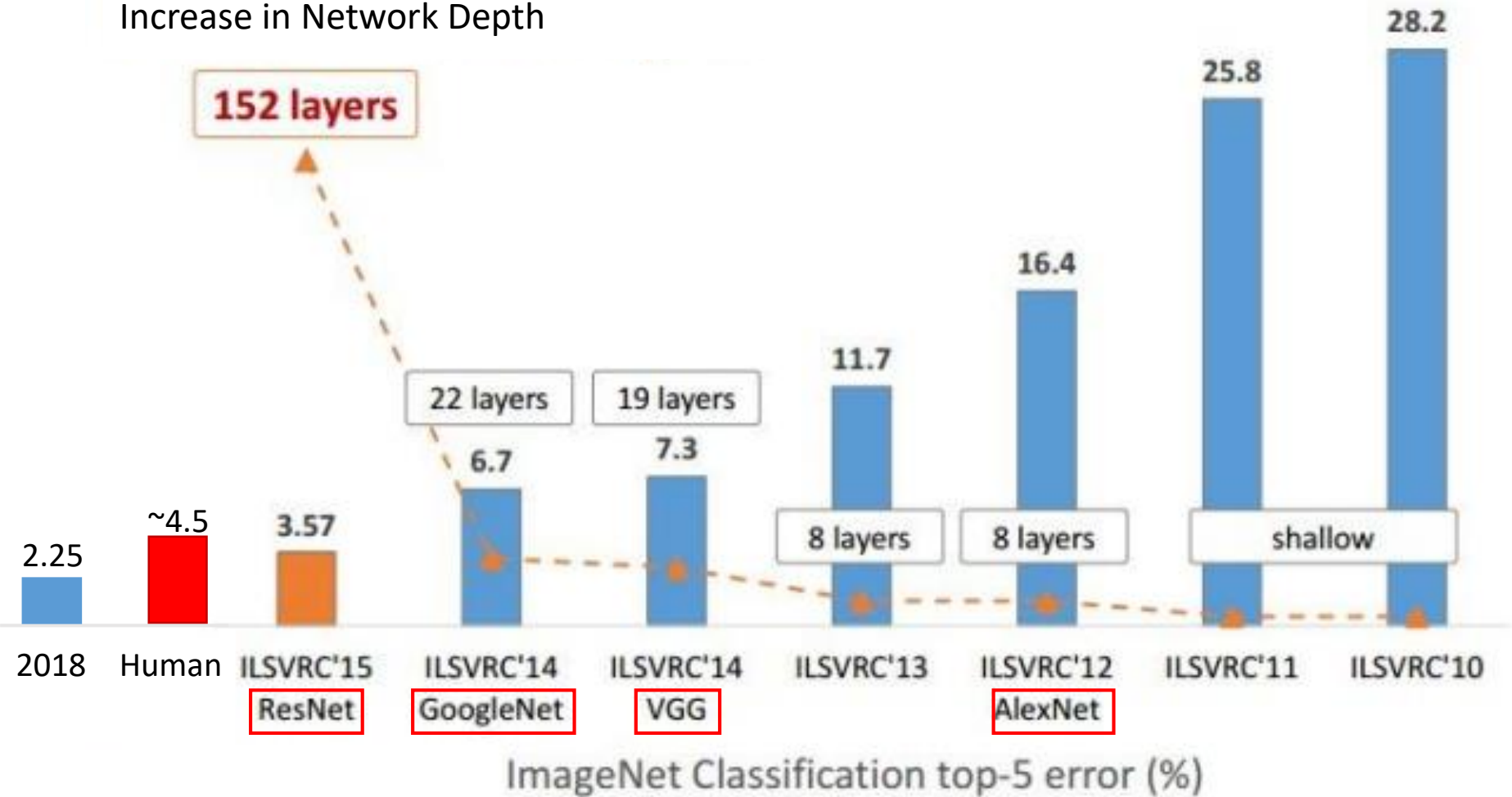
Classification Performance

- ImageNet - main challenge for object classification: <http://image-net.org/>
- 1000 classes, 1.2M training images, 150K for test



ImageNet Challenge Development

Increase in Network Depth

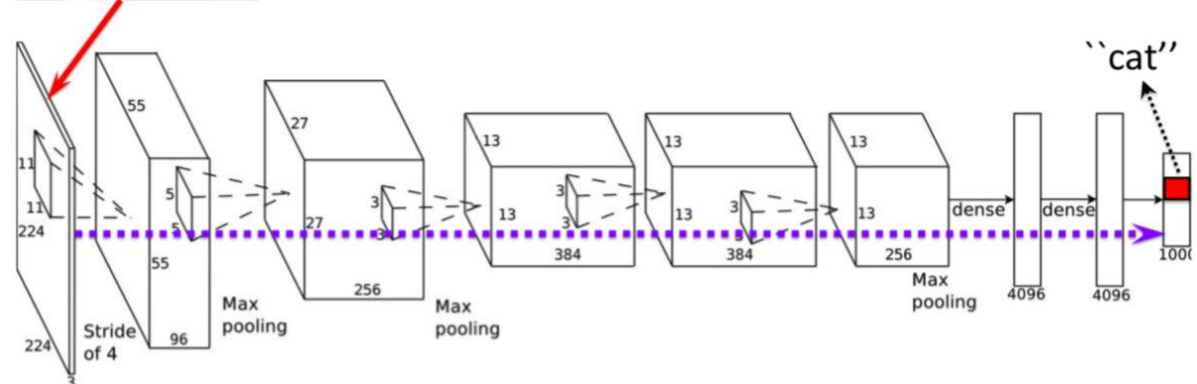


(slide credits: Kaiming He, ResNet 2015)

AlexNet



What's the class of this object?



Full (simplified) AlexNet architecture:

[227x227x3] INPUT

[55x55x96] CONV1: 96 11x11 filters at stride 4, pad 0

[27x27x96] MAX POOL1: 3x3 filters at stride 2

[27x27x96] NORM1: Normalization layer

[27x27x256] CONV2: 256 5x5 filters at stride 1, pad 2

[13x13x256] MAX POOL2: 3x3 filters at stride 2

[13x13x256] NORM2: Normalization layer

[13x13x384] CONV3: 384 3x3 filters at stride 1, pad 1

[13x13x384] CONV4: 384 3x3 filters at stride 1, pad 1

[13x13x256] CONV5: 256 3x3 filters at stride 1, pad 1

[6x6x256] MAX POOL3: 3x3 filters at stride 2

[4096] FC6: 4096 neurons

[4096] FC7: 4096 neurons

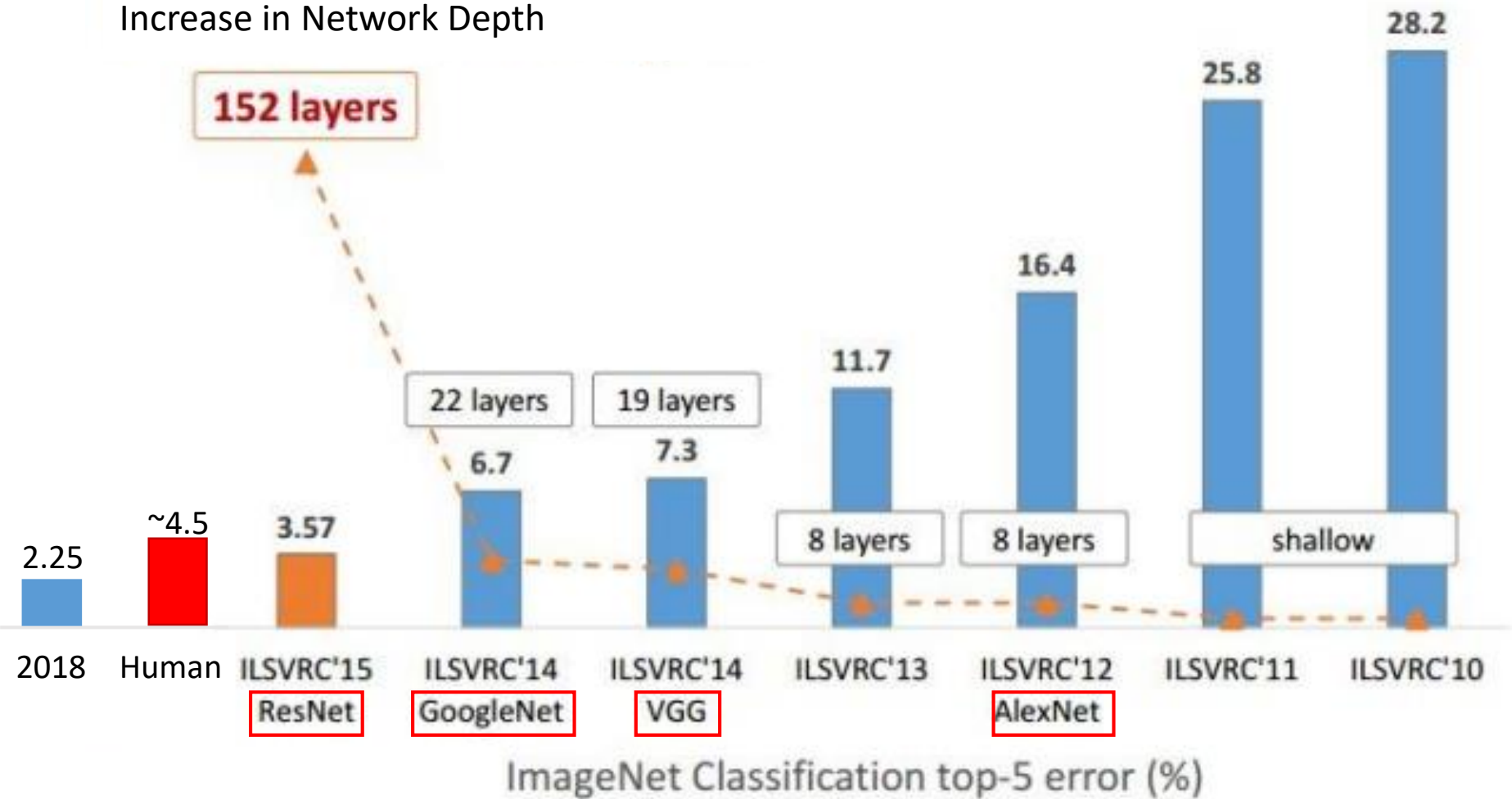
[1000] FC8: 1000 neurons (class scores)

- Total number of parameters: 60M
- Total number of flops: 832M
- Ensemble 7 CNN:
- ImageNet error 16.4%

[Krizhevsky et. al. NIPS 2012]

ImageNet Challenge Development

Increase in Network Depth



(slide credits: Kaiming He, ResNet 2015)

VGG Net

Only 3x3 CONV stride 1, pad 1
and 2x2 MAX POOL stride 2

best model

- ImageNet error 7.3%

input
↓
output

ConvNet Configuration					
A	A-LRN	B	C	D	E
11 weight layers	11 weight layers	13 weight layers	16 weight layers	16 weight layers	19 weight layers
input (224 × 224 RGB image)					
conv3-64	conv3-64 LRN	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64
maxpool					
conv3-128	conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128
maxpool					
conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256 conv1-256	conv3-256 conv3-256	conv3-256 conv3-256 conv3-256
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512	conv3-512 conv3-512 conv3-512
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512	conv3-512 conv3-512 conv3-512
maxpool					
FC-4096					
FC-4096					
FC-1000					
soft-max					

Table 2: Number of parameters (in millions).

Network	A,A-LRN	B	C	D	E
Number of parameters	133	133	134	138	144

[Simonyan and Zisserman, 2014]

VGG Net

INPUT: [224x224x3] memory: $224*224*3=150K$ params: 0 (not counting biases)

CONV3-64: [224x224x64] memory: $224*224*64=3.2M$ params: $(3*3*3)*64 = 1,728$

CONV3-64: [224x224x64] memory: $224*224*64=3.2M$ params: $(3*3*64)*64 = 36,864$

POOL2: [112x112x64] memory: $112*112*64=800K$ params: 0

CONV3-128: [112x112x128] memory: $112*112*128=1.6M$ params: $(3*3*64)*128 = 73,728$

CONV3-128: [112x112x128] memory: $112*112*128=1.6M$ params: $(3*3*128)*128 = 147,456$

POOL2: [56x56x128] memory: $56*56*128=400K$ params: 0

CONV3-256: [56x56x256] memory: $56*56*256=800K$ params: $(3*3*128)*256 = 294,912$

CONV3-256: [56x56x256] memory: $56*56*256=800K$ params: $(3*3*256)*256 = 589,824$

CONV3-256: [56x56x256] memory: $56*56*256=800K$ params: $(3*3*256)*256 = 589,824$

POOL2: [28x28x256] memory: $28*28*256=200K$ params: 0

CONV3-512: [28x28x512] memory: $28*28*512=400K$ params: $(3*3*256)*512 = 1,179,648$

CONV3-512: [28x28x512] memory: $28*28*512=400K$ params: $(3*3*512)*512 = 2,359,296$

CONV3-512: [28x28x512] memory: $28*28*512=400K$ params: $(3*3*512)*512 = 2,359,296$

POOL2: [14x14x512] memory: $14*14*512=100K$ params: 0

CONV3-512: [14x14x512] memory: $14*14*512=100K$ params: $(3*3*512)*512 = 2,359,296$

CONV3-512: [14x14x512] memory: $14*14*512=100K$ params: $(3*3*512)*512 = 2,359,296$

CONV3-512: [14x14x512] memory: $14*14*512=100K$ params: $(3*3*512)*512 = 2,359,296$

POOL2: [7x7x512] memory: $7*7*512=25K$ params: 0

FC: [1x1x4096] memory: 4096 params: $7*7*512*4096 = 102,760,448$

FC: [1x1x4096] memory: 4096 params: $4096*4096 = 16,777,216$

FC: [1x1x1000] memory: 1000 params: $4096*1000 = 4,096,000$

TOTAL memory: 24M * 4 bytes $\approx$ 93MB / image (only forward! $\sim$ *2 for bwd)

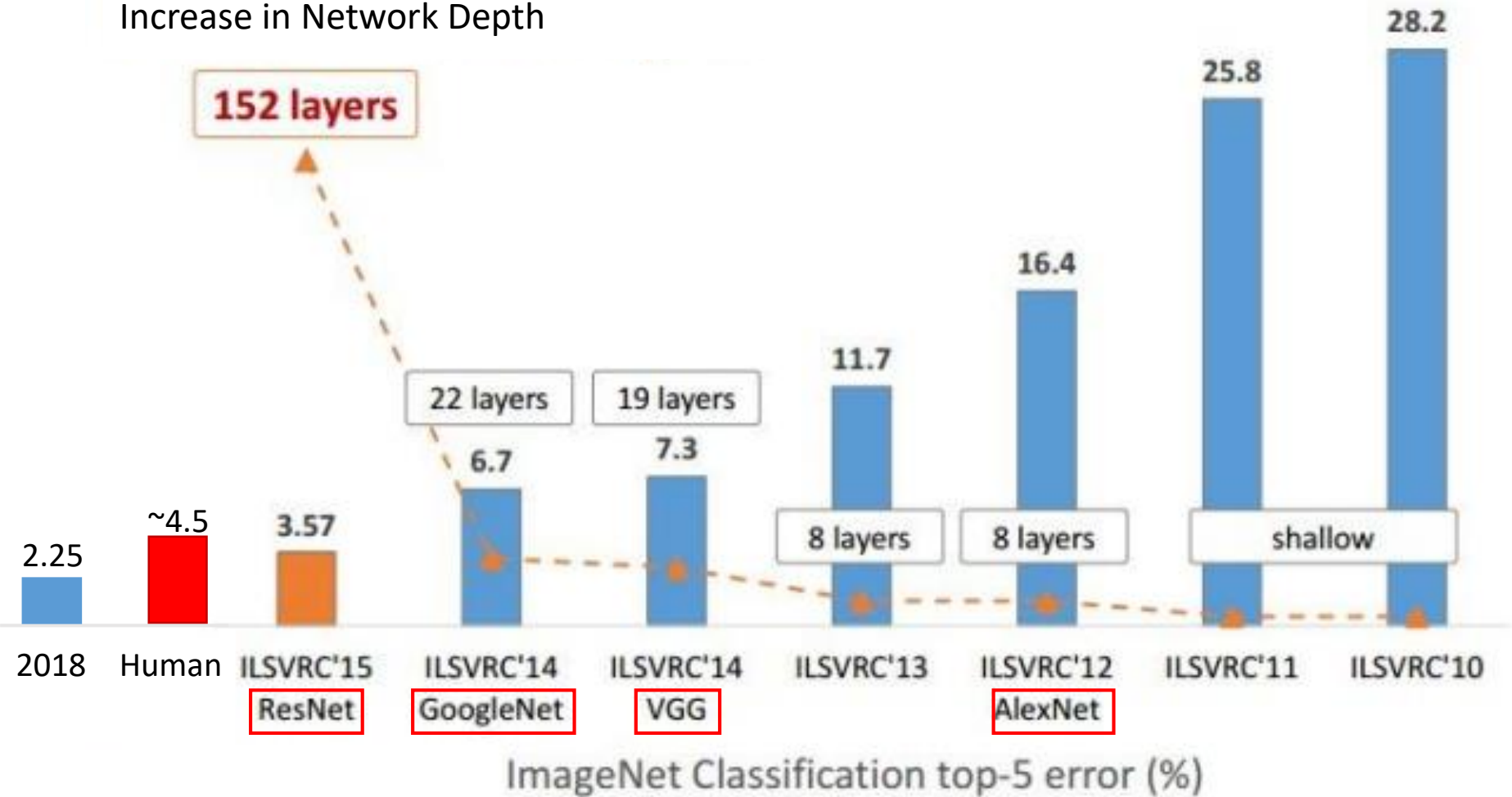
TOTAL params: 138M parameters

ConvNet Configuration			
B	C	D	
13 weight layers	16 weight layers	16 weight layers	19
put (224 × 224 RGB image)			
conv3-64	conv3-64	conv3-64	conv3-64
conv3-64	conv3-64	conv3-64	conv3-64
maxpool			
conv3-128	conv3-128	conv3-128	conv3-128
conv3-128	conv3-128	conv3-128	conv3-128
maxpool			
conv3-256	conv3-256	conv3-256	conv3-256
conv3-256	conv3-256	conv3-256	conv3-256
maxpool			
conv3-512	conv3-512	conv3-512	conv3-512
conv3-512	conv3-512	conv3-512	conv3-512
maxpool			
conv3-512	conv3-512	conv3-512	conv3-512
conv3-512	conv3-512	conv3-512	conv3-512
maxpool			
conv3-512	conv3-512	conv3-512	conv3-512
conv3-512	conv3-512	conv3-512	conv3-512
maxpool			
FC-4096			
FC-4096			
FC-1000			
soft-max			

[Simonyan and Zisserman, 2014]

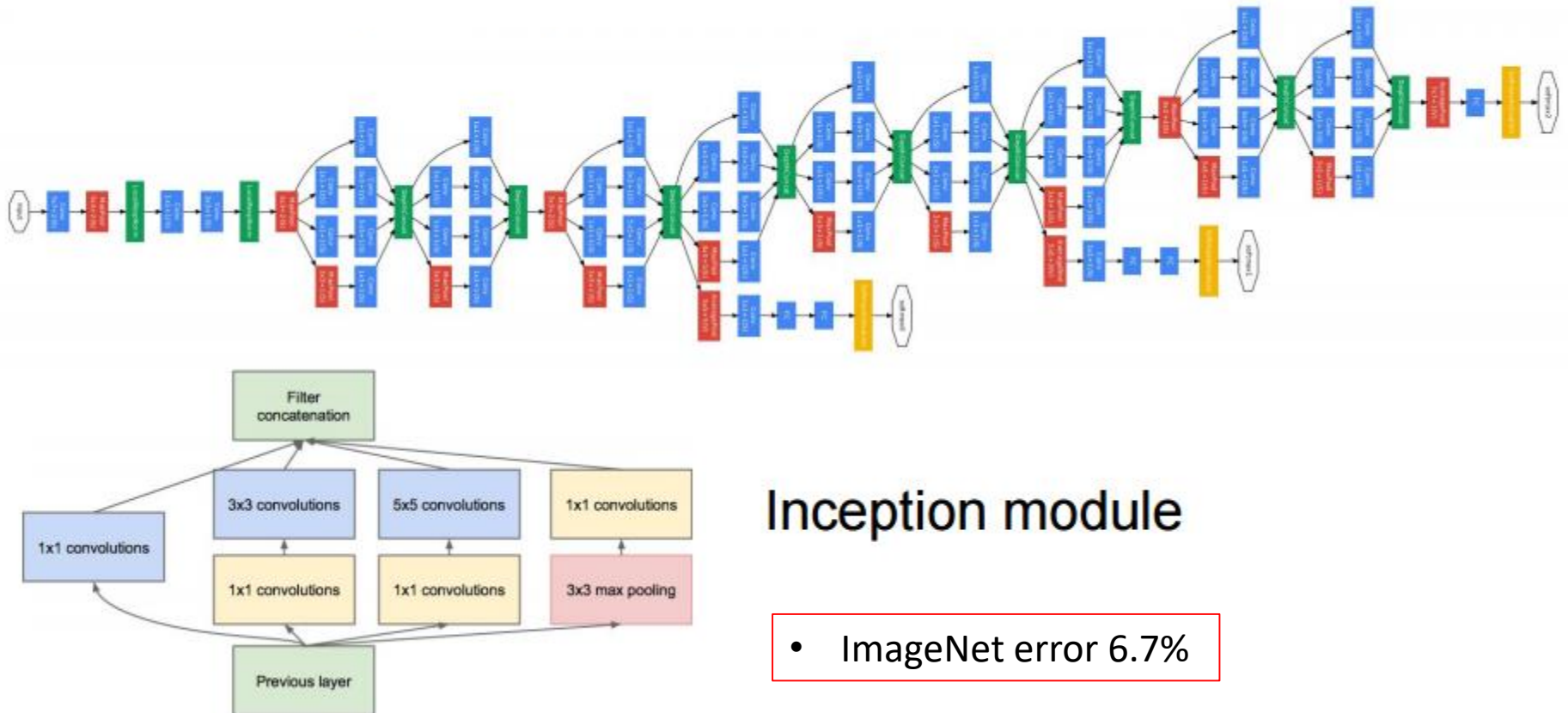
ImageNet Challenge Development

Increase in Network Depth



(slide credits: Kaiming He, ResNet 2015)

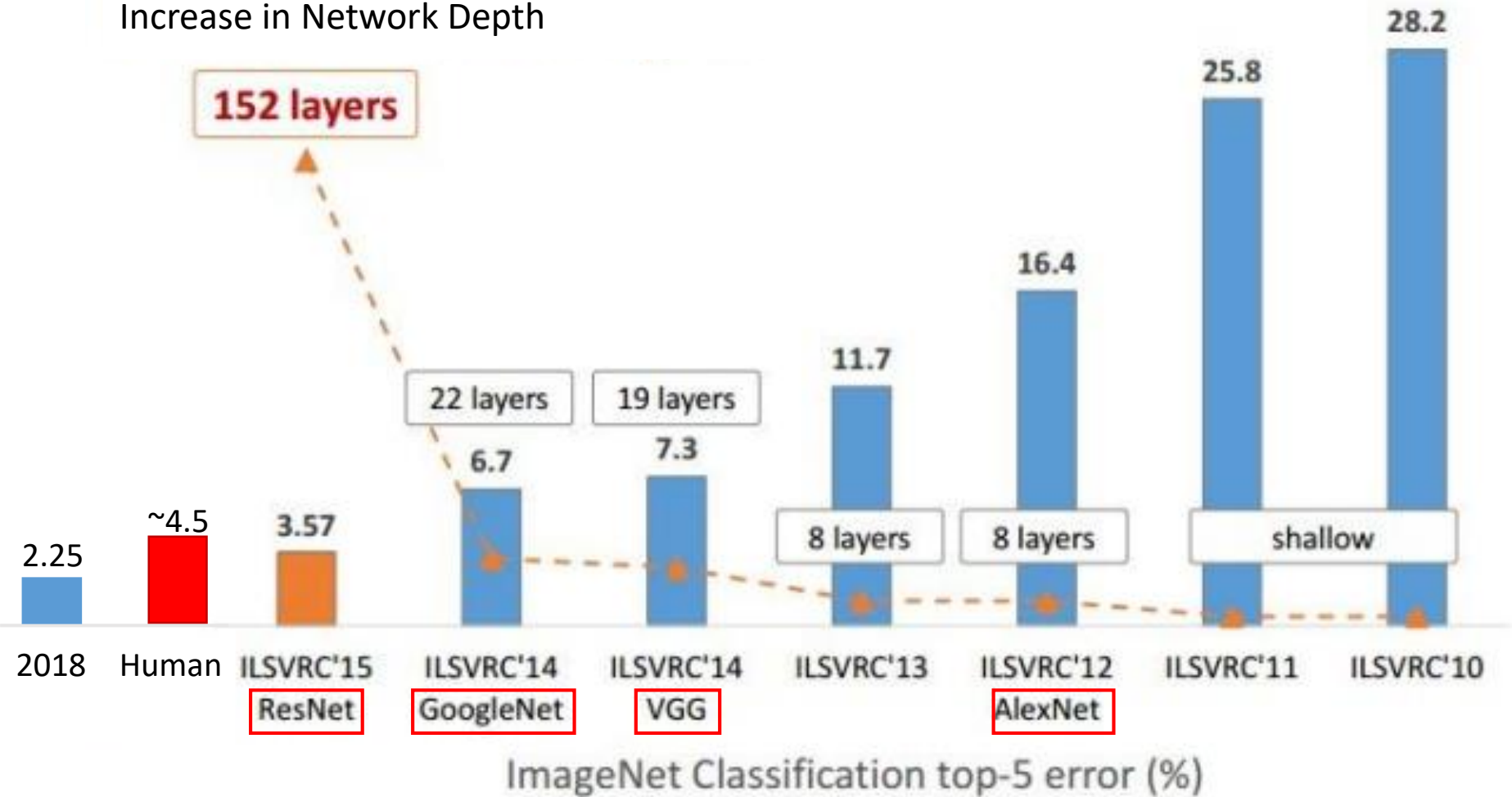
GoogleNet



[Szegedy et al., 2014]

ImageNet Challenge Development

Increase in Network Depth



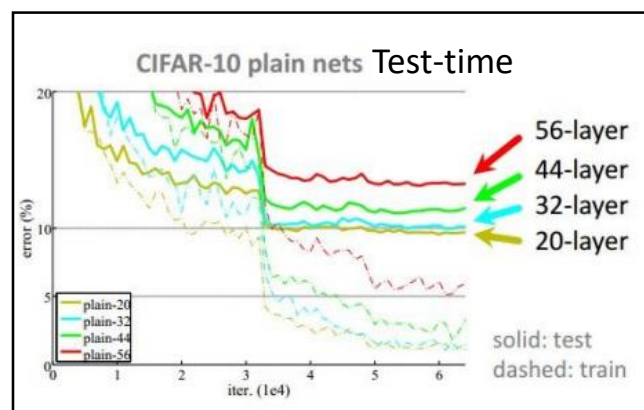
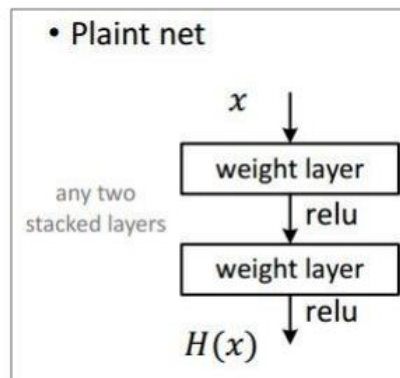
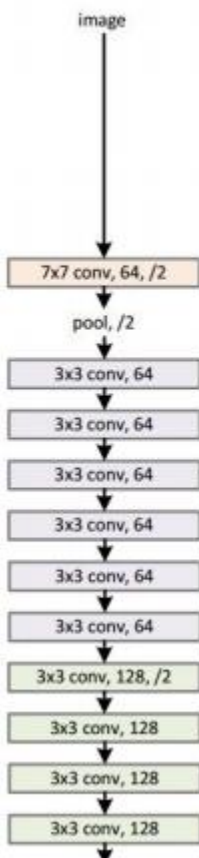
(slide credits: Kaiming He, ResNet 2015)

- **1st places** in all five main tracks

- ImageNet Classification: *“Ultra-deep”* (quote Yann) **152-layer** nets
 - ImageNet Detection: **16%** better than 2nd
 - ImageNet Localization: **27%** better than 2nd
 - COCO Detection: **11%** better than 2nd
 - COCO Segmentation: **12%** better than 2nd
- 2-3 weeks of training on 8 GPUs
 - At runtime faster than VGG (it has 8x more layers but the layers are more shallow and there is no fully connected layer in the end)

ResNet

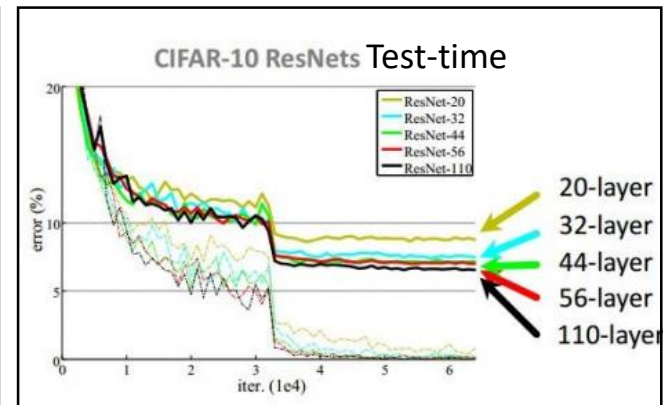
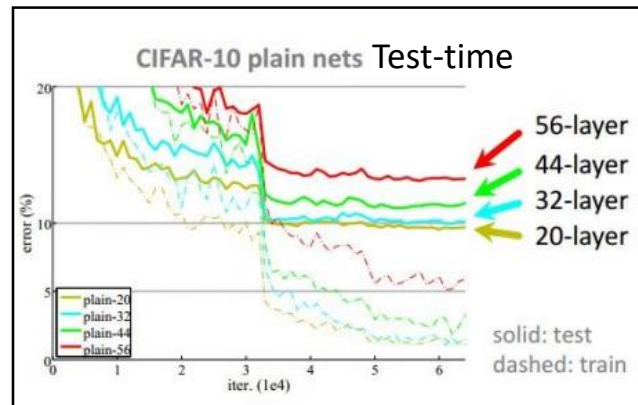
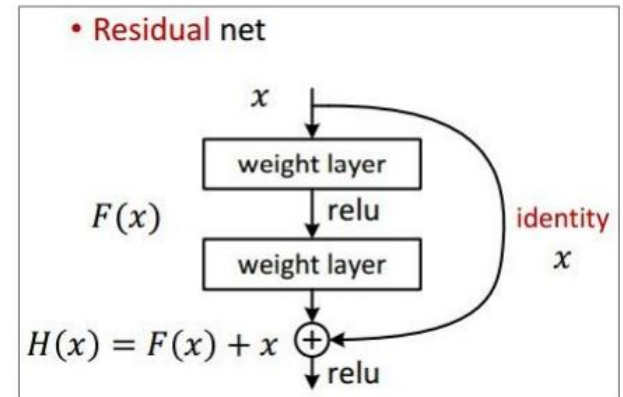
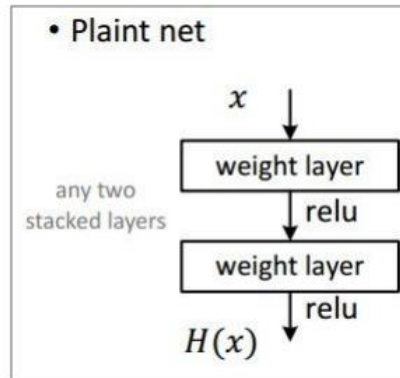
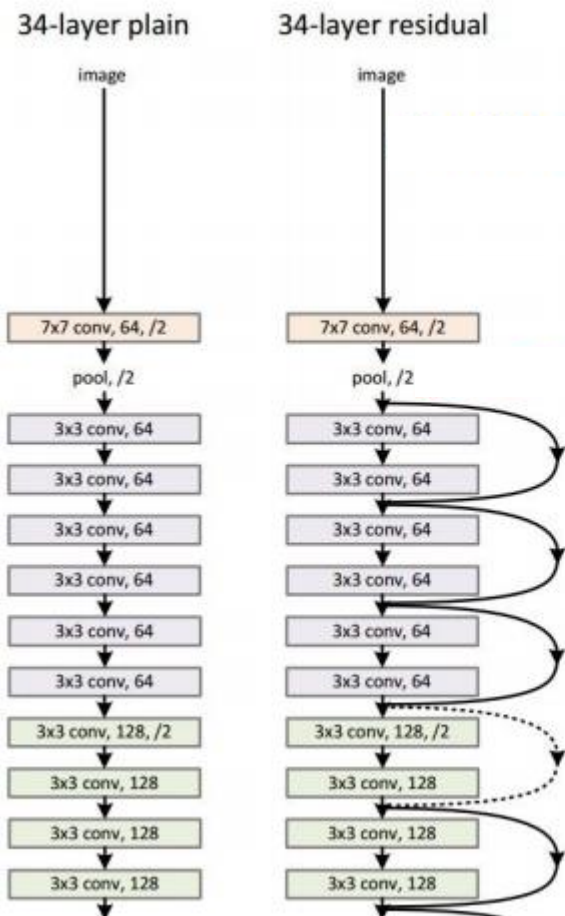
34-layer plain



- More layers in the plain CNN architecture means overfitting

[He et al., 2015]

ResNet



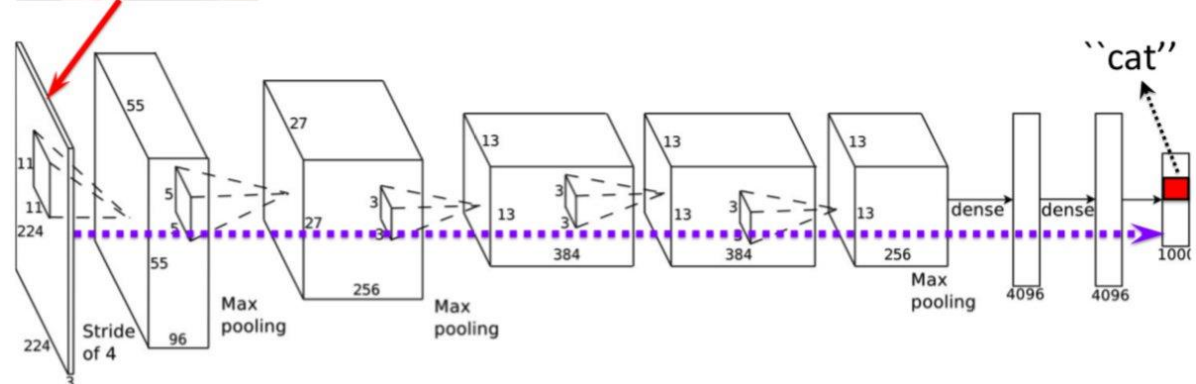
- More layers in the plain CNN architecture means overfitting
 - More layers in the residual CNN architecture gives better performance
 - No vanishing gradients! It allows gradients to go through the network without modification
- [He et al., 2015]

- Types of Machine Learning Problems
- Decision Trees
- Deep Learning: Discriminative Models
 - Neural Networks
 - Convolutional Neural Networks
- Image Classification: ResNet
- Understanding Image Classification Networks
- Semantic Segmentation: U-Net
- Active and Semi-Supervised Learning

AlexNet



What's the class of this object?



Full (simplified) AlexNet architecture:

[227x227x3] INPUT

[55x55x96] CONV1: 96 11x11 filters at stride 4, pad 0

[27x27x96] MAX POOL1: 3x3 filters at stride 2

[27x27x96] NORM1: Normalization layer

[27x27x256] CONV2: 256 5x5 filters at stride 1, pad 2

[13x13x256] MAX POOL2: 3x3 filters at stride 2

[13x13x256] NORM2: Normalization layer

[13x13x384] CONV3: 384 3x3 filters at stride 1, pad 1

[13x13x384] CONV4: 384 3x3 filters at stride 1, pad 1

[13x13x256] CONV5: 256 3x3 filters at stride 1, pad 1

[6x6x256] MAX POOL3: 3x3 filters at stride 2

[4096] FC6: 4096 neurons

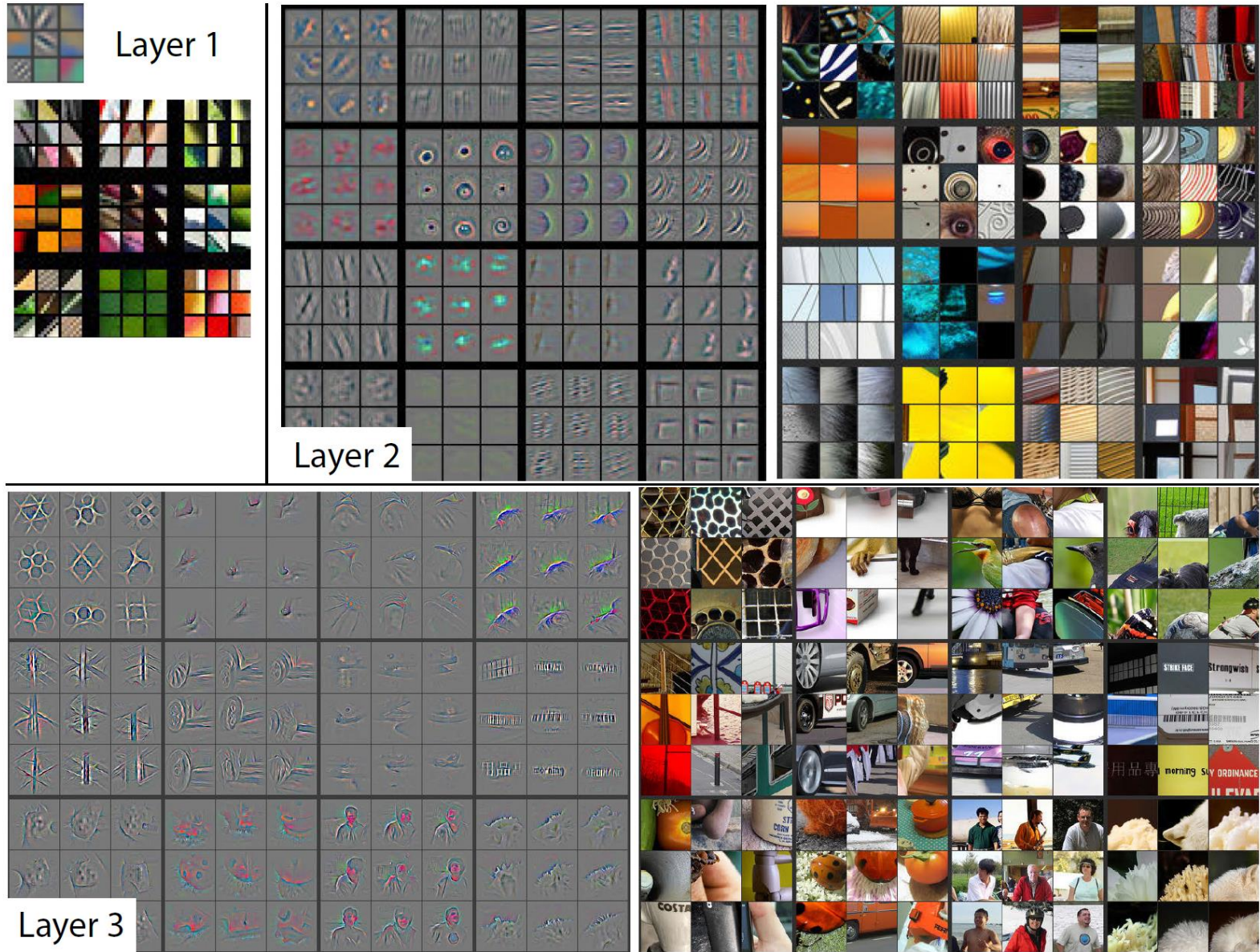
[4096] FC7: 4096 neurons

[1000] FC8: 1000 neurons (class scores)

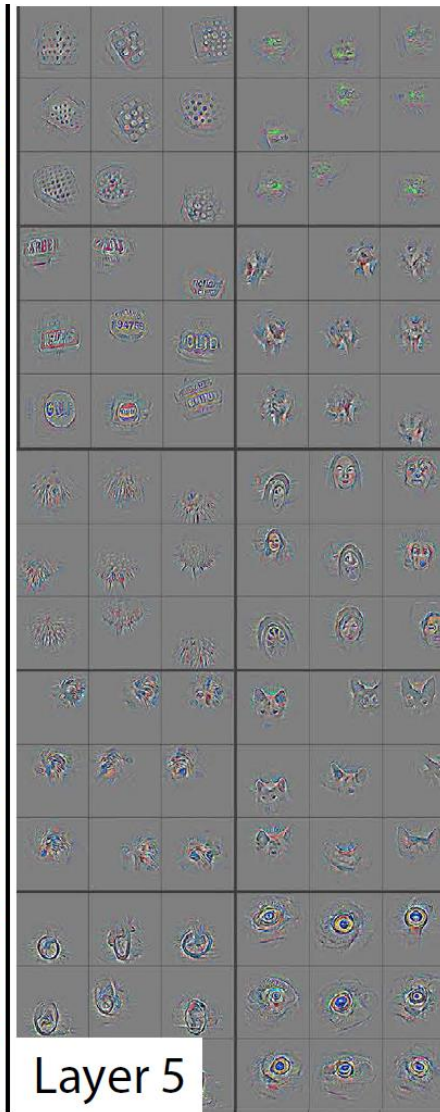
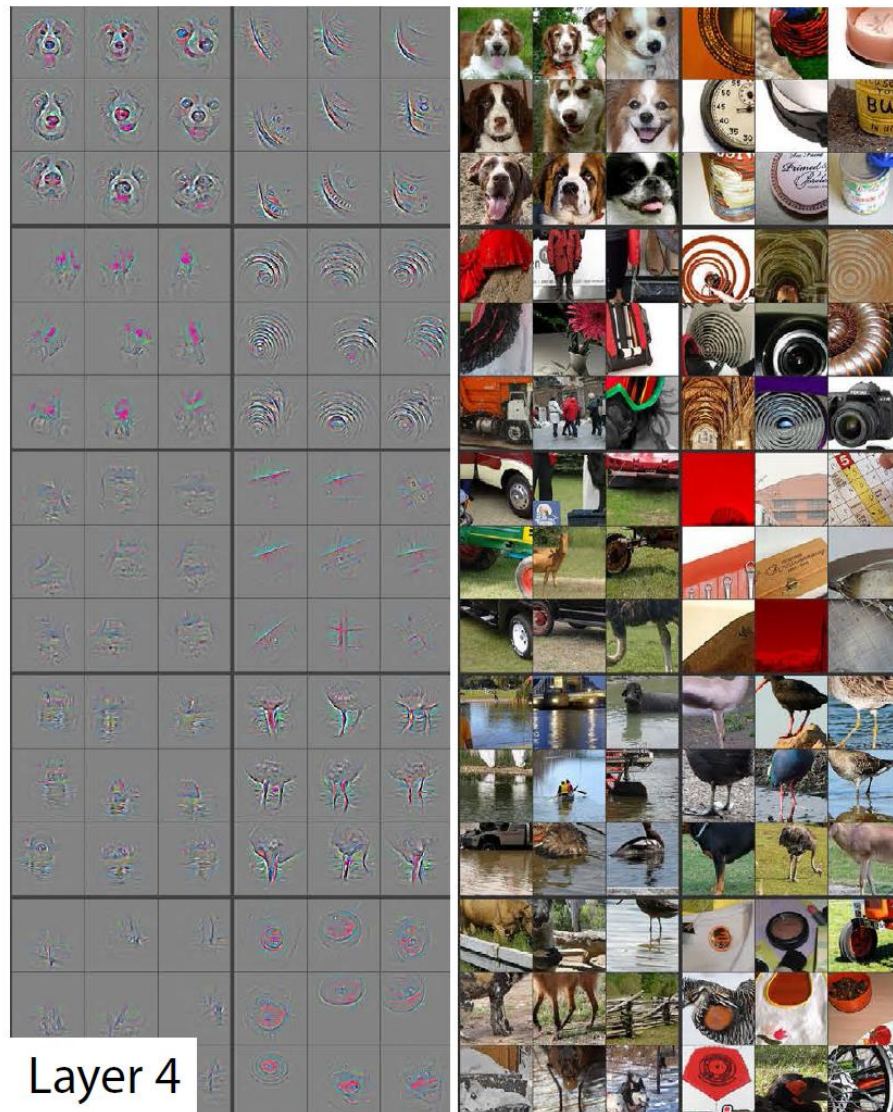
- Total number of parameters: 60M
- Total number of flops: 832M
- Ensemble 7 CNN:
- ImageNet error 16.4%

[Krizhevsky et. al. NIPS 2012]

CNN Visualization with ZFNet



CNN Visualization with ZFNet



Minor modifications to AlexNet:

Full (simplified) AlexNet architecture:

[227x227x3] INPUT

[55x55x96] CONV1: 96 ~~11x11 filters at stride 4~~, pad 0 (instead: 7x7 stride 2)

[27x27x96] MAX POOL1: 3x3 filters at stride 2

[27x27x96] NORM1: Normalization layer

[27x27x256] CONV2: 256 5x5 filters at stride 1, pad 2

[13x13x256] MAX POOL2: 3x3 filters at stride 2

[13x13x256] NORM2: Normalization layer

[13x13x384] CONV3: ~~384~~ 3x3 filters at stride 1, pad 1 (instead: 512)

[13x13x384] CONV4: ~~384~~ 3x3 filters at stride 1, pad 1 (instead: 1024)

[13x13x256] CONV5: ~~256~~ 3x3 filters at stride 1, pad 1 (instead: 512)

[6x6x256] MAX POOL3: 3x3 filters at stride 2

[4096] FC6: 4096 neurons

[4096] FC7: 4096 neurons

[1000] FC8: 1000 neurons (class scores)

- ImageNet error 16.4% -> 14.8%

[Zeiler and Fergus, 2013]

Interpretable Networks: B-cos Networks

Full (non-linear) operation: $\mathbf{h}_1 = \max(0, W_1 \mathbf{x} + \mathbf{b}_1)$

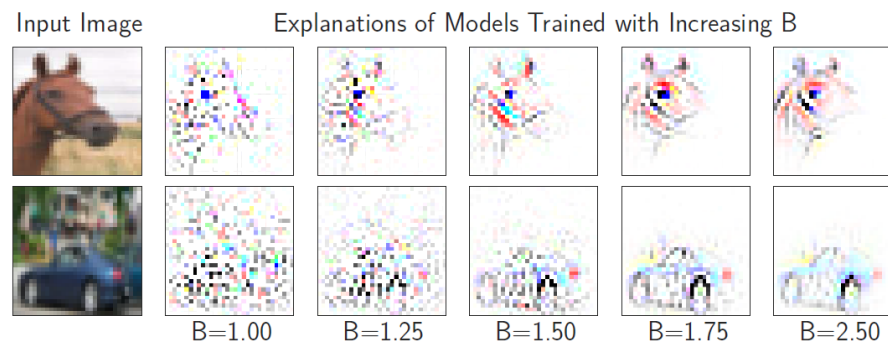
Linear operation can be written as:

$$\mathbf{w}^T \mathbf{x} = \|\mathbf{w}\| \|\mathbf{x}\| c(\mathbf{x}, \mathbf{w})$$

with $c(\mathbf{x}, \mathbf{w}) = \cos(\angle(\mathbf{x}, \mathbf{w}))$

Replace Linear operation with non-linear B-cos transformation:

$$\text{B-cos}(\mathbf{x}; \mathbf{w}) = \underbrace{\|\hat{\mathbf{w}}\|}_{=1} \|\mathbf{x}\| |c(\mathbf{x}, \hat{\mathbf{w}})|^B \times \text{sgn}(c(\mathbf{x}, \hat{\mathbf{w}})).$$



$\mathbf{w}_{1 \rightarrow l}$ ignores the activation function and multiplies all weights of the deep network together.

Fig. 4. Col. 1: Input images. Cols. 2-6: Explanations for classes ‘horse’ and ‘car’ of models trained with increasing values of B. With higher B, the linear transforms $\mathbf{W}_{1 \rightarrow l}$ increasingly align with discriminative patterns and thus become more interpretable.

[B-cos Networks: Alignment is All We Need for Interpretability; Böhle et al. CVPR 2022]

Interpretable Networks: B-cos Networks

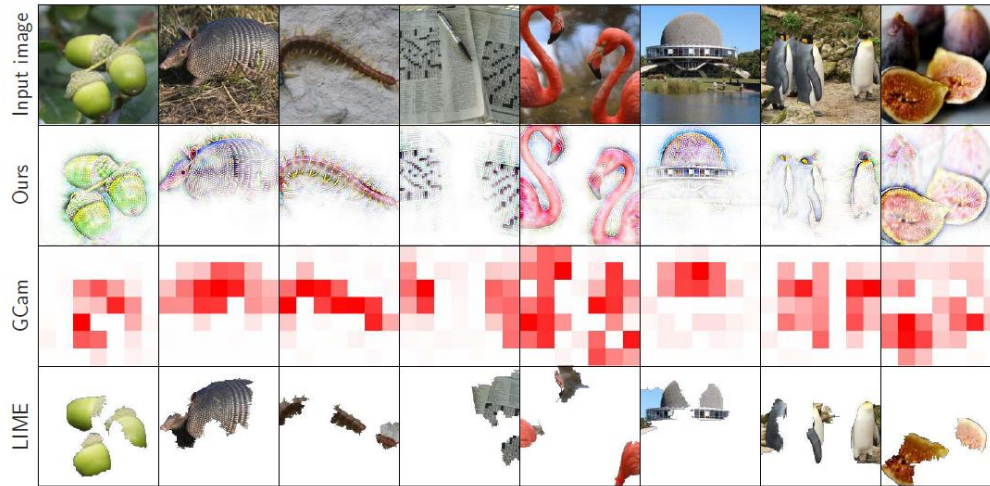


Fig. 7. Comparison between model-inherent explanations and the strongest post-hoc methods. More results in supplement (Sec. A.)

Model	Top-1 Accuracy (%)			
	pretrained	B-cos [11]	B-cosified	Δ_{acc}
ResNet-18	69.8	68.7	71.5	+2.8
ResNet-50-v1	76.1	75.9	76.5	+0.6
ResNet-50-v2	80.9	75.9	77.3	+1.4
DenseNet-121	74.4	73.6	76.3	+2.7
ViT-Ti	70.3	60.0	69.3	+9.3
ViT-S	74.4	69.2	75.2	+6.0
ViT-B	75.3	74.4	75.3	+0.9
ViT-L	75.8	75.1	75.5	+0.4
ViT _c -Ti	72.6	67.3	72.3	+5.0
ViT _c -S	75.7	74.5	76.0	+1.5
ViT _c -B	76.8	77.1	76.7	-0.4
ViT _c -L	77.9	77.8	77.1	-0.7

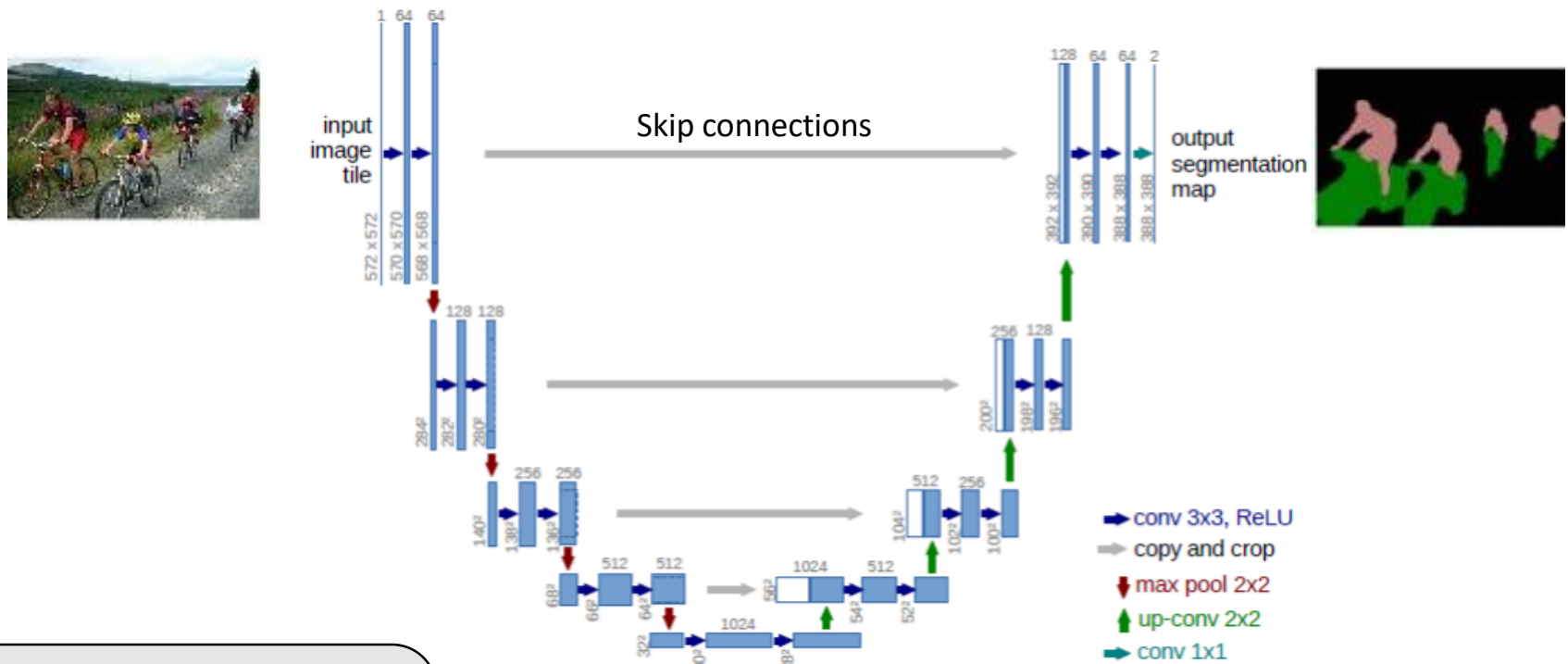
Table 4: **Classification Accuracy.** We report the top-1 classification accuracy on the ImageNet validation set of the pre-trained models (**pretrained**) and the B-cosified models (**B-cosified**) after fine-tuning them. Additionally, we report the accuracy of the corresponding B-cos models trained from scratch (**B-cos**) as well as the difference to them (Δ_{acc}),

[B-cosification: Transforming Deep Neural Networks to be Inherently Interpretable, Arva et al. NeurIPS 2025]

[B-cos Networks: Alignment is All We Need for Interpretability; Böhle et al. CVPR 2022]

- Types of Machine Learning Problems
- Decision Trees
- Deep Learning: Discriminative Models
 - Neural Networks
 - Convolutional Neural Networks
- Image Classification: ResNet
- Understanding Image Classification Networks
- Semantic Segmentation: U-Net
- Active and Semi-Supervised Learning

U-Net: Popular Network for Sematic Segmentation



Despite spatially small (3x3) filters you can do semantic reasoning since large receptive field.

[U-Net: Convolutional Networks for Biomedical Image Segmentation, O. Ronneberger, P. Fischer, and T. Brox, MICCAI 2015]



U-net: Convolutional Networks for Biomedical Image Segmentation

Olaf Ronneberger, Philipp Fischer, and Thomas Brox

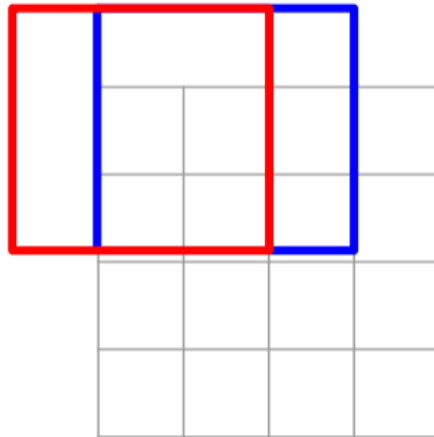
Computer Science Department and
BIOSS Centre for Biological Signalling Studies
University of Freiburg, Germany

Olaf Ronneberger, University of Freiburg, Germany, 22.5.2015

Learning Upsampling: Transpose Convolution

Recall: Normal 3 x 3 convolution, stride 1

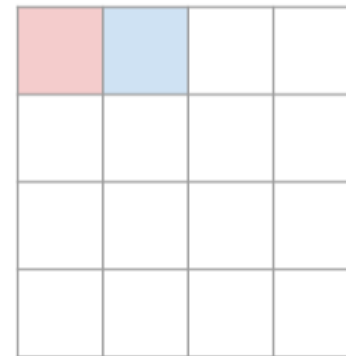
(the same filter)



Input: 4 x 4



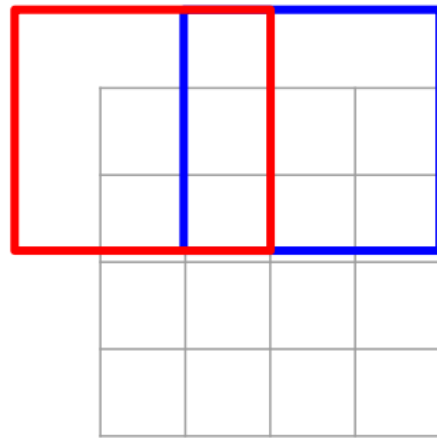
Dot product
between filter
and input



Output: 4 x 4

Learning Upsampling: Transpose Convolution

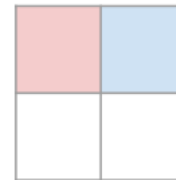
Recall: Normal 3 x 3 convolution, stride 2 pad 1



Input: 4 x 4



Dot product
between filter
and input

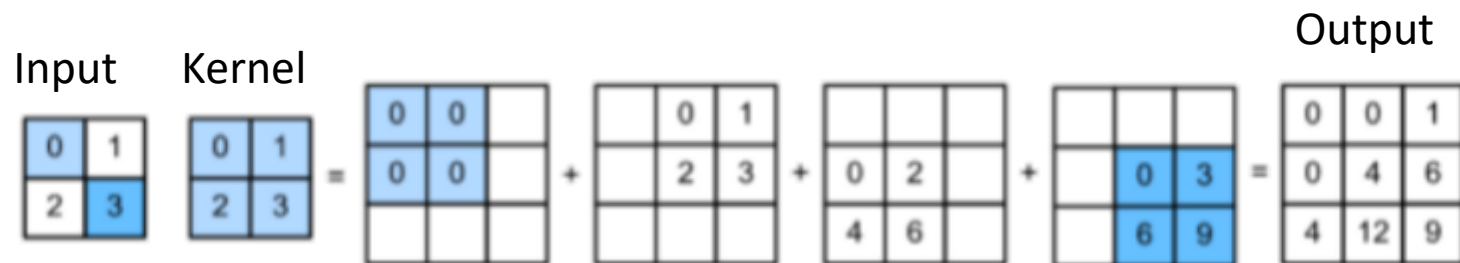


Output: 2 x 2

Filter moves 2 pixels in
the input for every one
pixel in the output

Stride gives ratio between
movement in input and
output

Learning Upsampling: Transpose Convolution



Other names:

- Deconvolution
- Up-Convolution
- Fractionally strided convolution
- Backward strided convolution

- Types of Machine Learning Problems
- Decision Trees
- Deep Learning: Discriminative Models
 - Neural Networks
 - Convolutional Neural Networks
- Image Classification: ResNet
- Understanding Image Classification Networks
- Semantic Segmentation: U-Net
- Active and Semi-Supervised Learning

Two Types of Models:

1. Discriminative Models: $y = f_w(x)$ or $p(y|x)$
2. Generative Models: $p(x)$, $p(x|y)$, or $p(y, x)$

Variants of Machine Learning Models:

1. *Structured models versus not structured models*
2. ...

Three Classes of Machine Learning Problems:

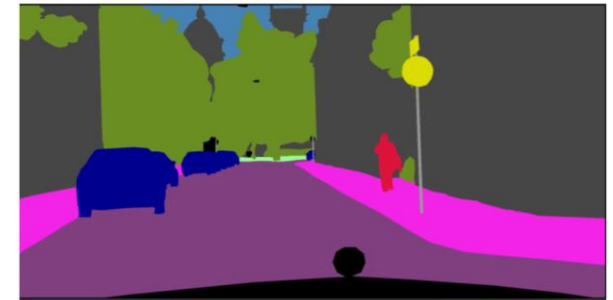
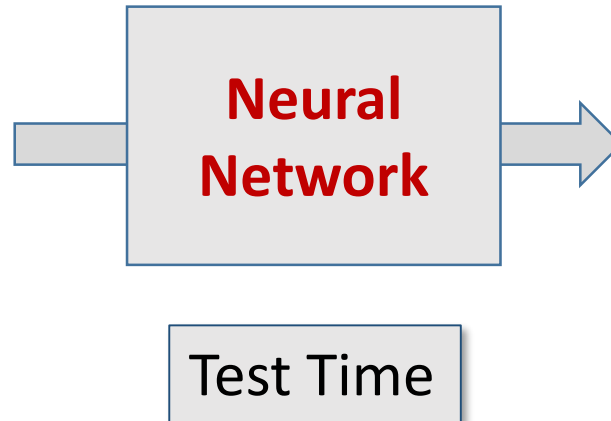
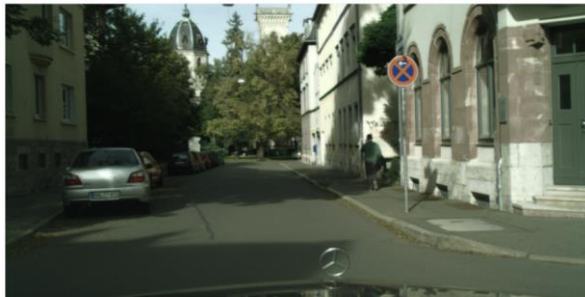
1. **Supervised learning:** fully labelled (annotated) dataset
2. **Unsupervised learning:** no annotations at all (e.g. clustering)
3. **Reinforcement learning:** there is a reward for correct action
(*we focus on supervised learning*)

Lost of variants of Machine Learning Problems:

1. On-line learning: Training samples are not available at once
2. Continual Learning: incrementally learning/adapting your model
3. **Self-Supervised:** Use the data itself for learning
4. **Domain adaption:** Train in Domain A and test in Domain B
5. **Pre-Text Tasks:** Train for Task A and fine-tune for task B
6. **Active learning:** What unlabeled data should I label next?
7. **Semi-Supervised learning:** Dataset is partially labelled
8.

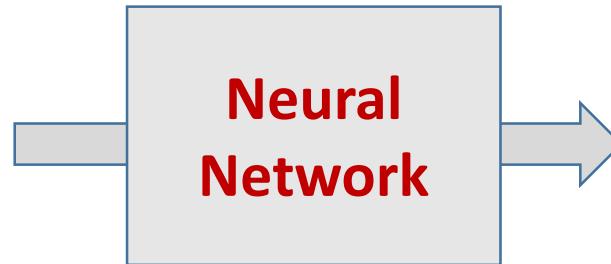
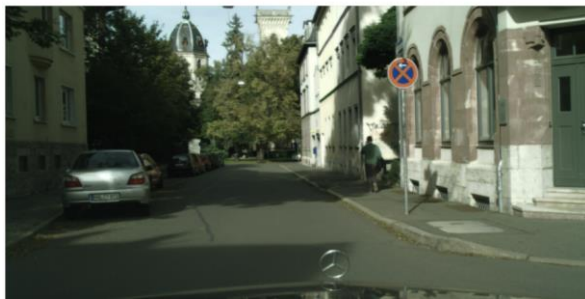
Motivation

 Car	 Sky
 Road	 Vegetation
 Sidewalk	 Street Sign
 Person	 Building

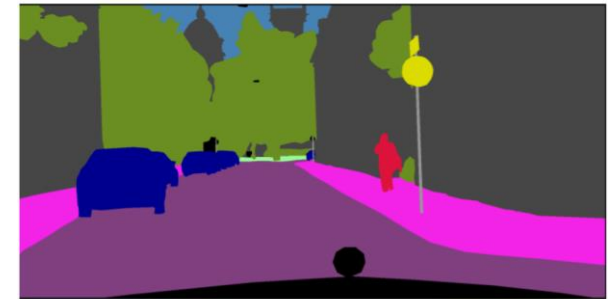


Motivation

 Car	 Sky
 Road	 Vegetation
 Sidewalk	 Street Sign
 Person	 Building



Training Time
(Supervised
learning)

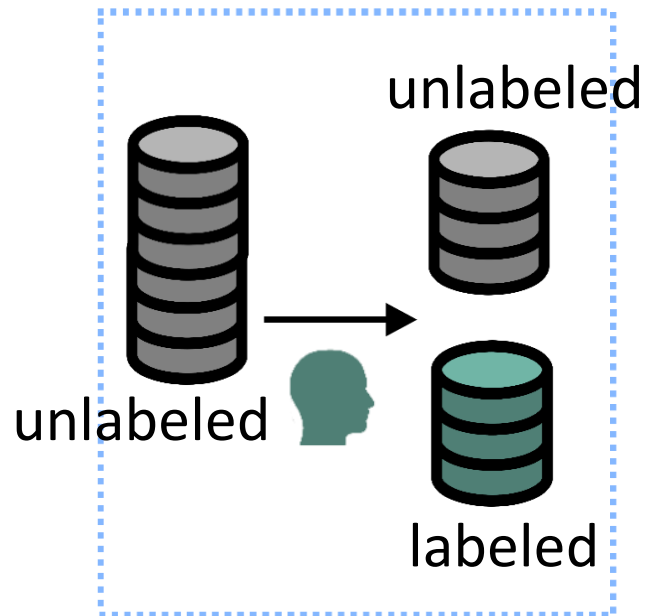


Expensive: 10-12\$ per Image
Time consuming: 1-2 hours

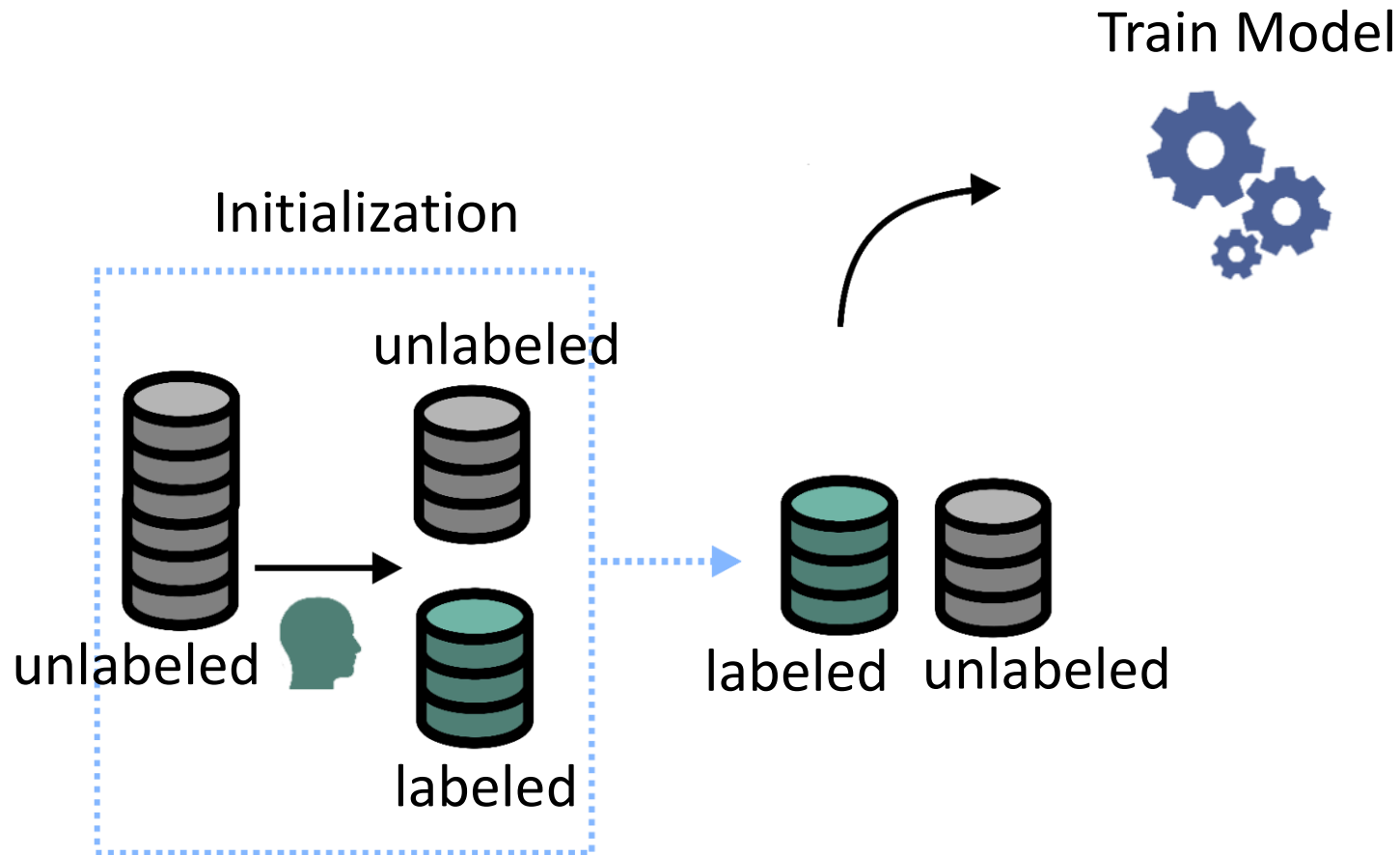
[The Cityscapes Dataset for Semantic Urban Scene Understanding. M. Cordts, et. al. CVPR 2016]

Active Learning

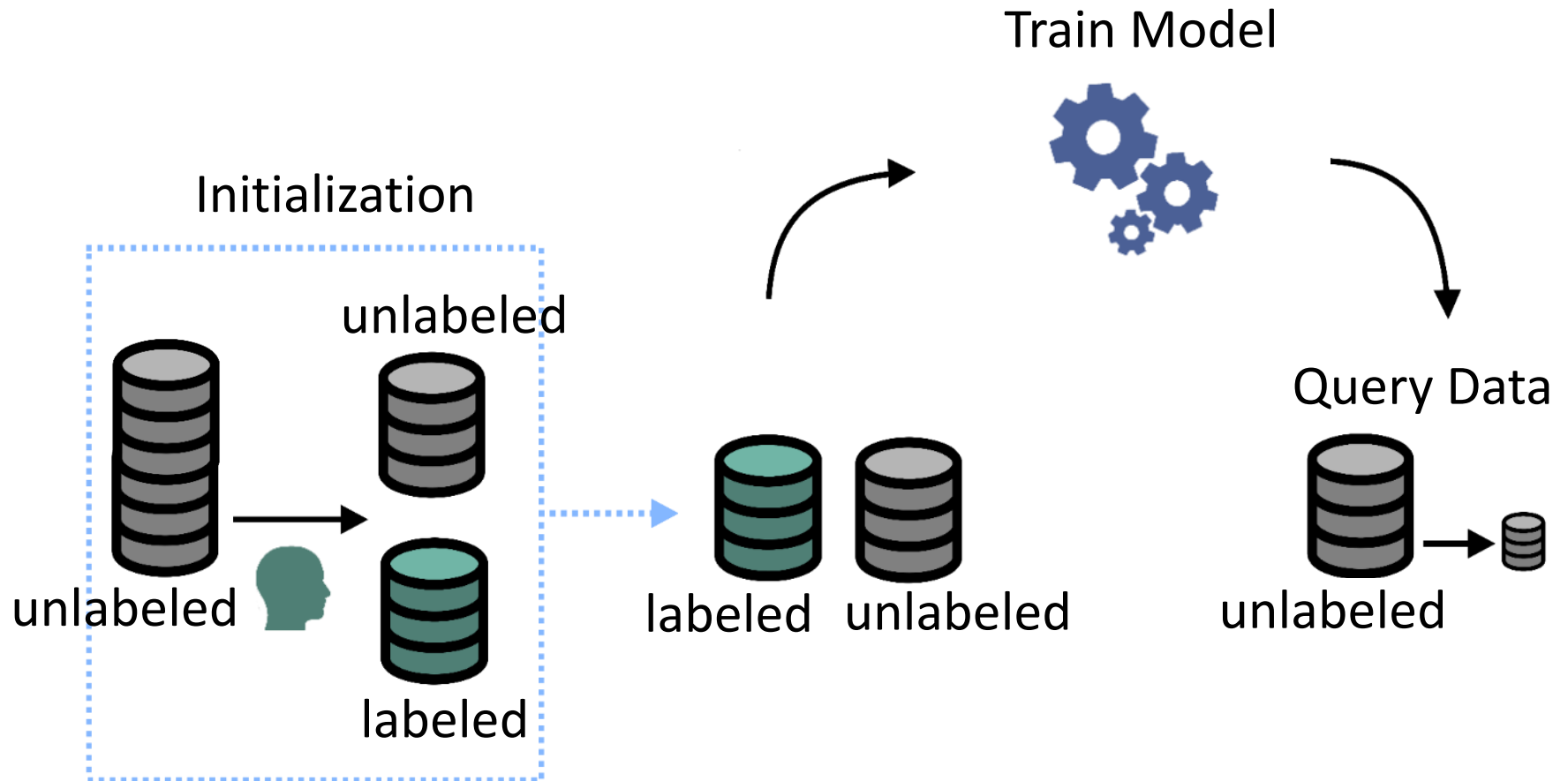
Initialization



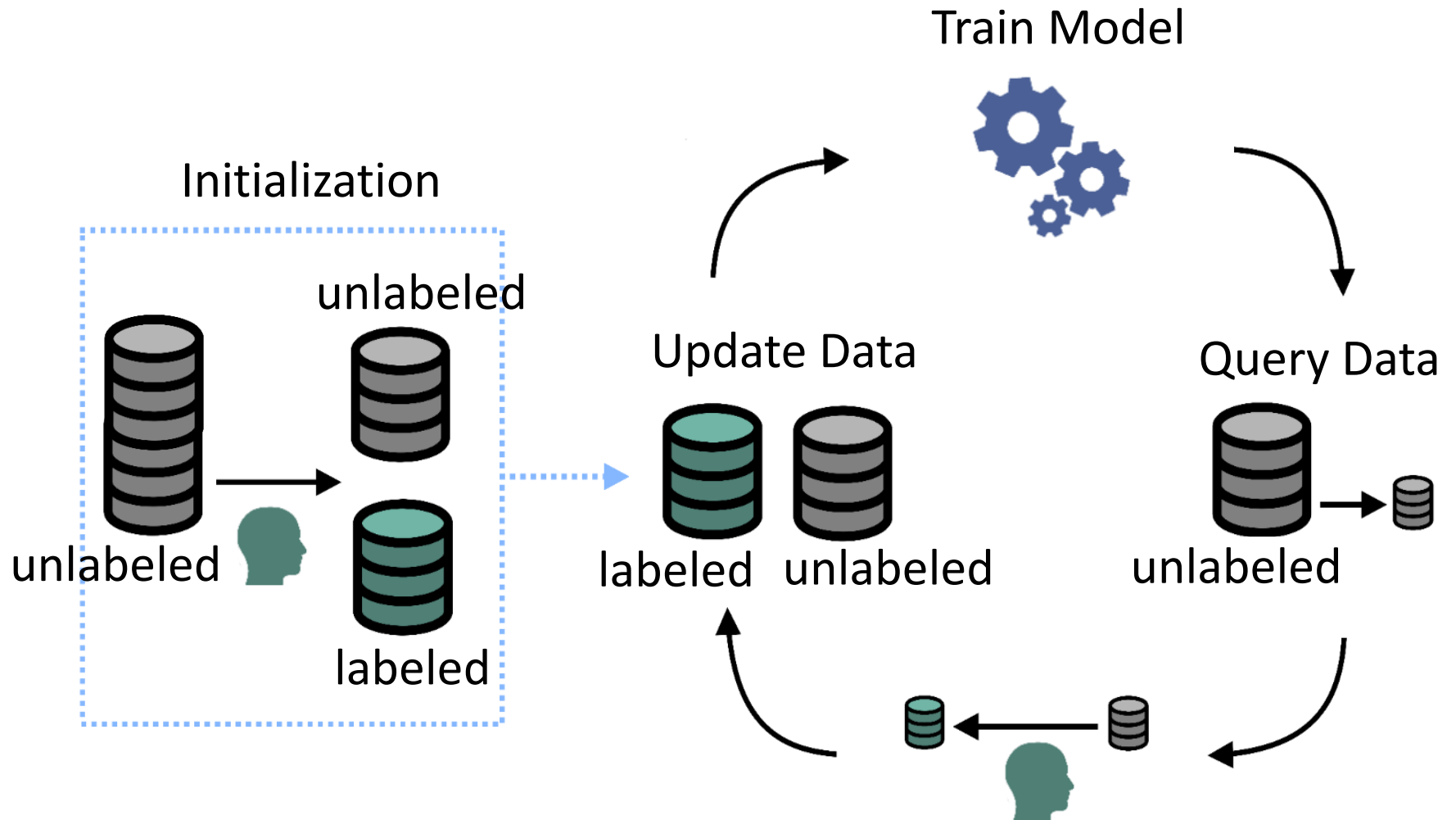
Active Learning



Active Learning



Active Learning

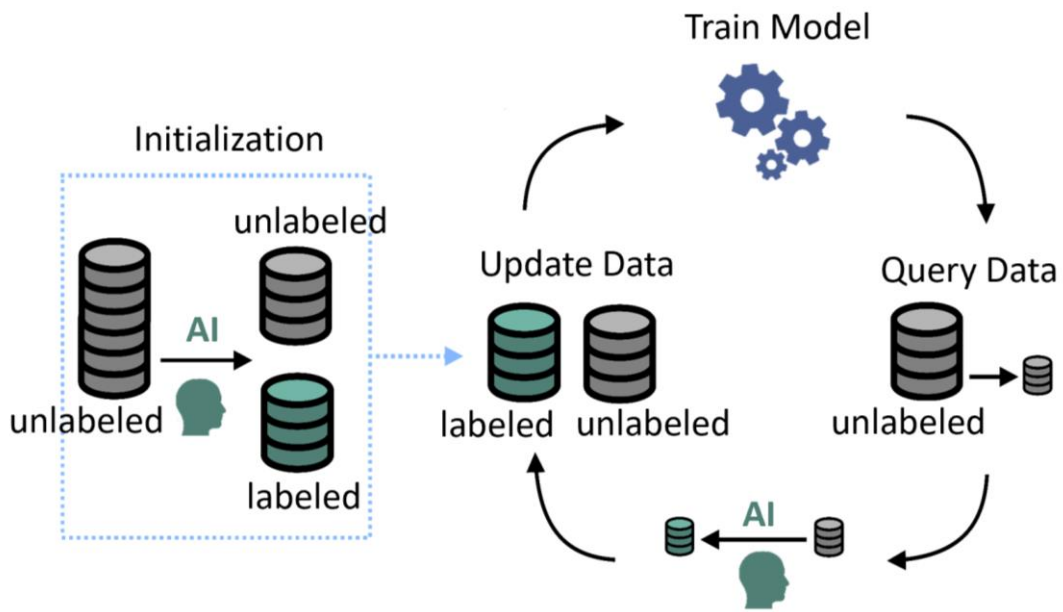


Best Practices in Active Learning for Semantic Segmentation

S. Mittal, J. Niemeijer, J.P. Schäfer, T. Brox

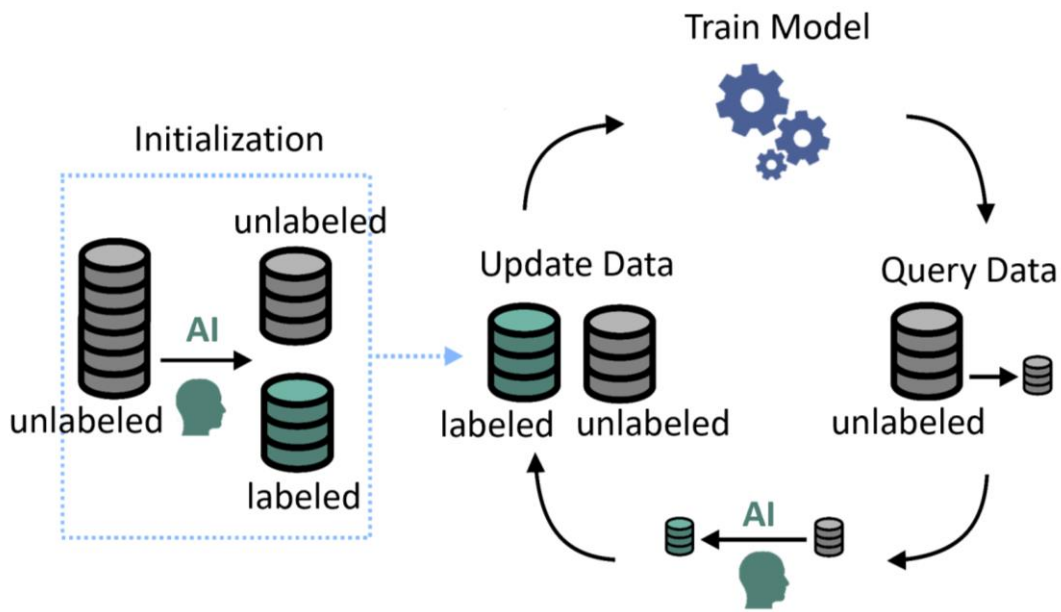
GCPR 2023

Active Learning



- Random

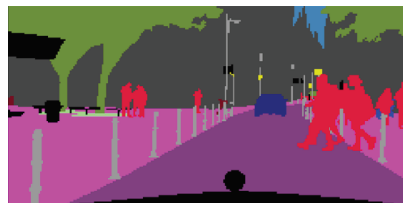
Active Learning



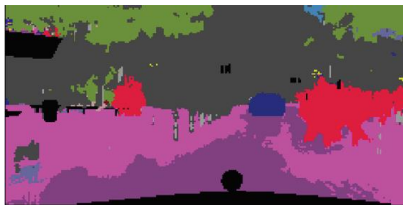
- Random
- Uncertainty



Query image



Ground Truth

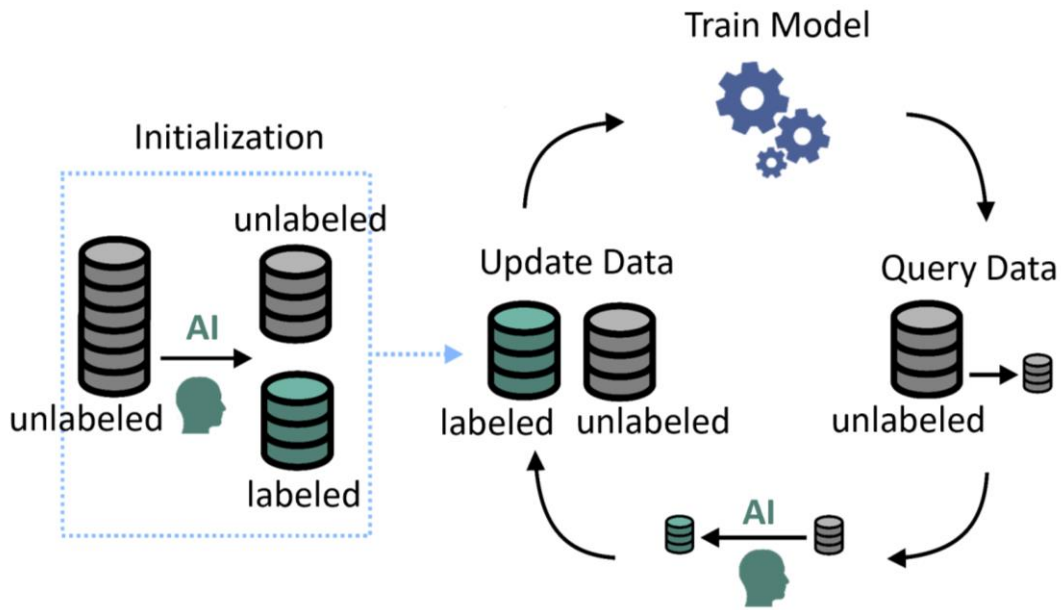


Initial Result



Uncertainty

Active Learning



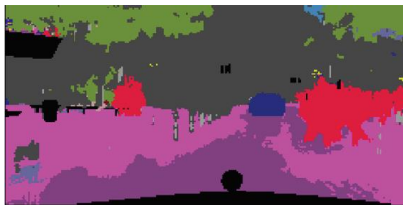
- Random
- Uncertainty
- Core Set



Query image



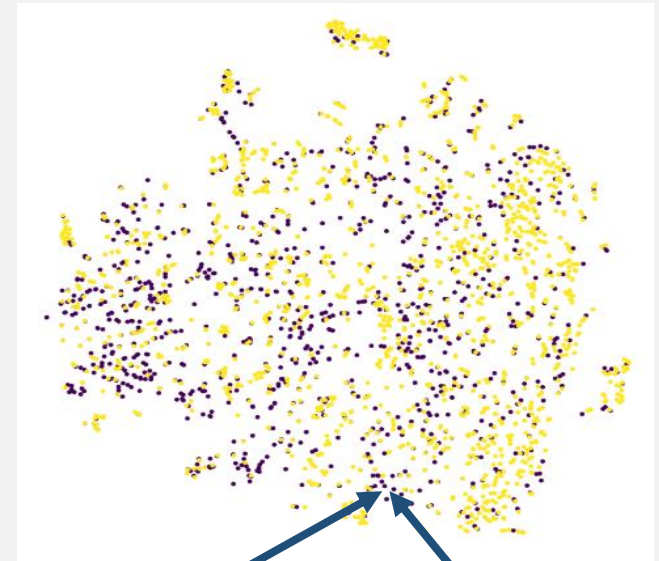
Ground Truth



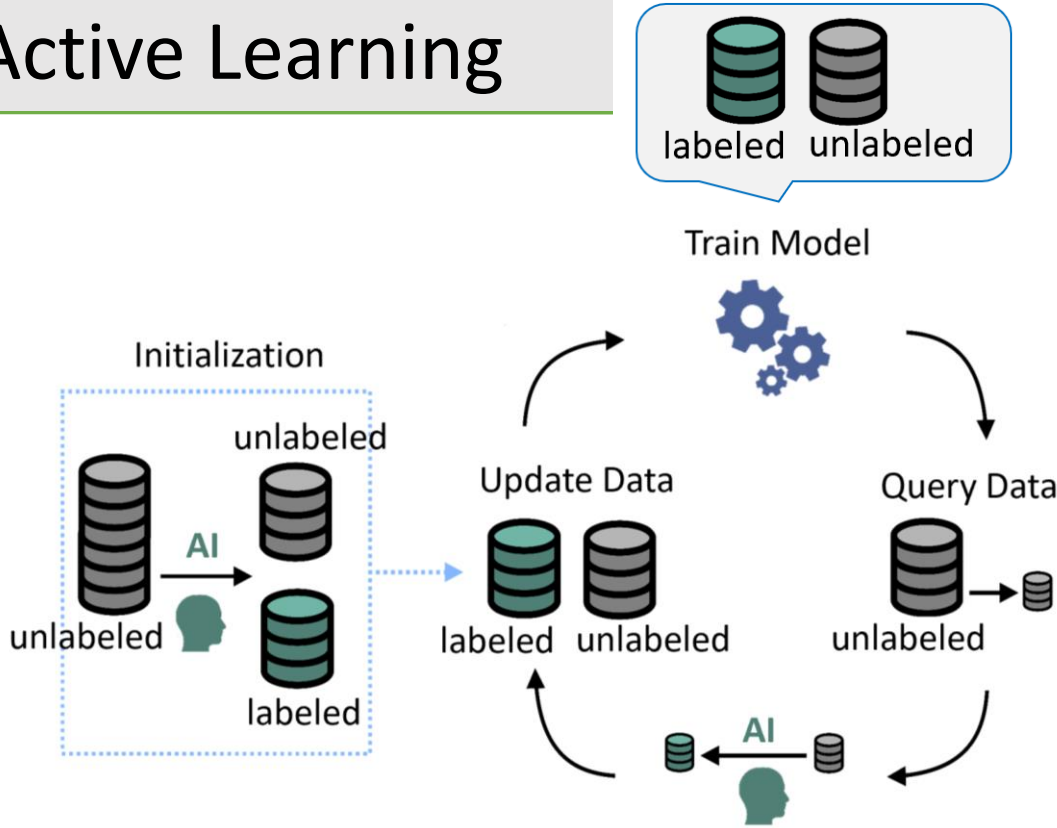
Initial Result



Uncertainty



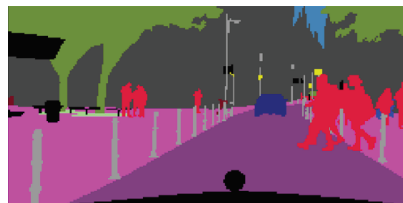
Active Learning



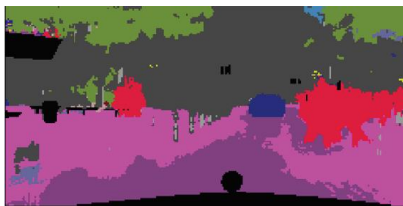
- Random
- Uncertainty
- Core Set



Query image



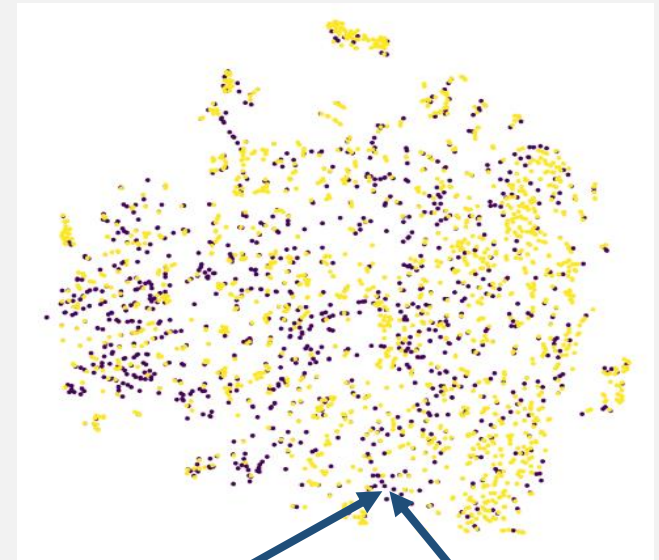
Ground Truth



Initial Result



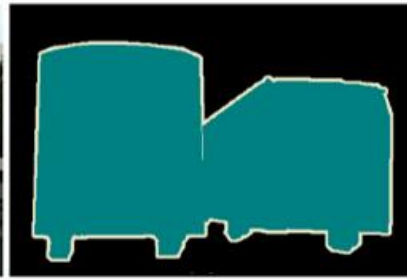
Uncertainty



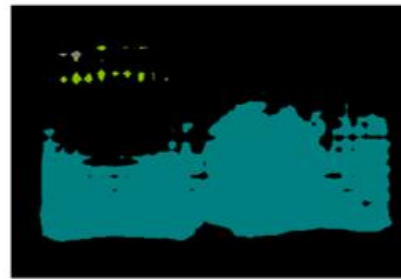
Semi-Supervised Learning



Input



Ground Truth



5% of Training Data



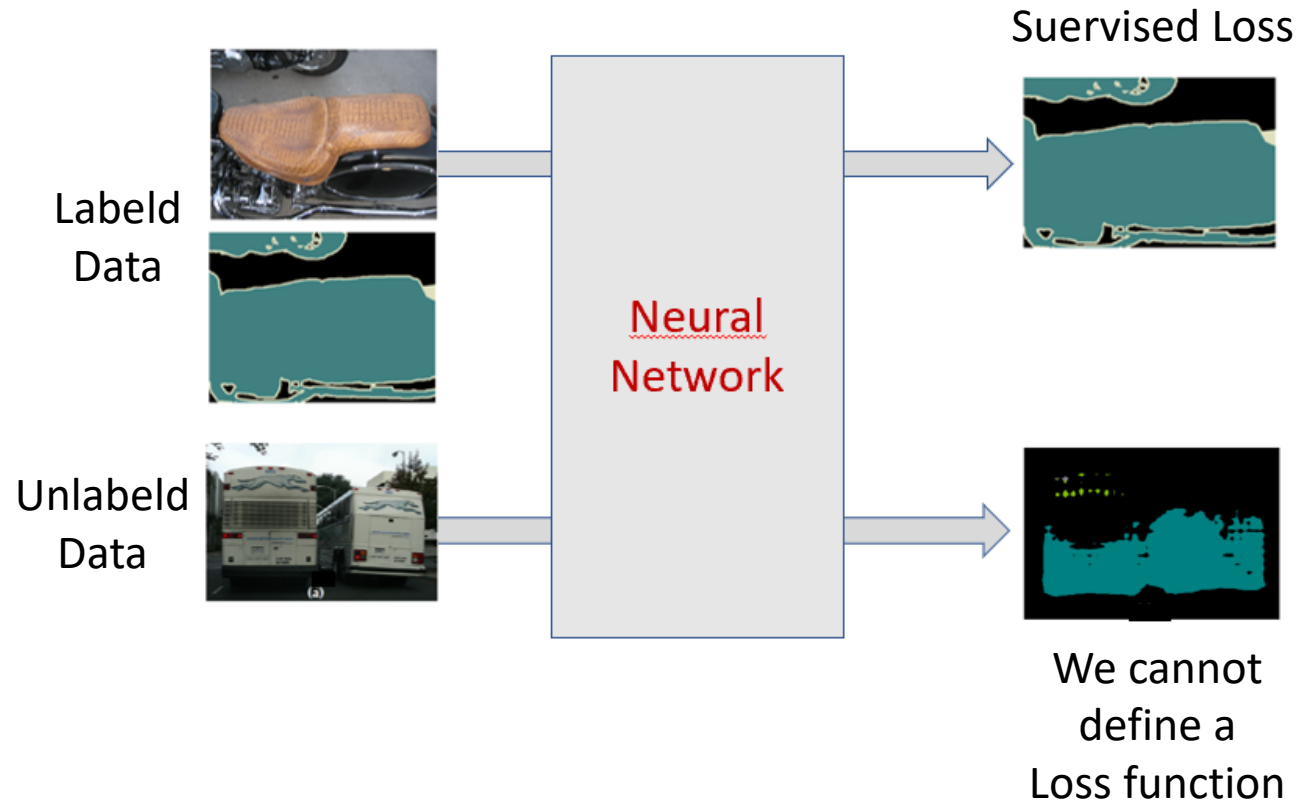
100% of Training Data



5% Training Data
+ 95% Unlabelled Data

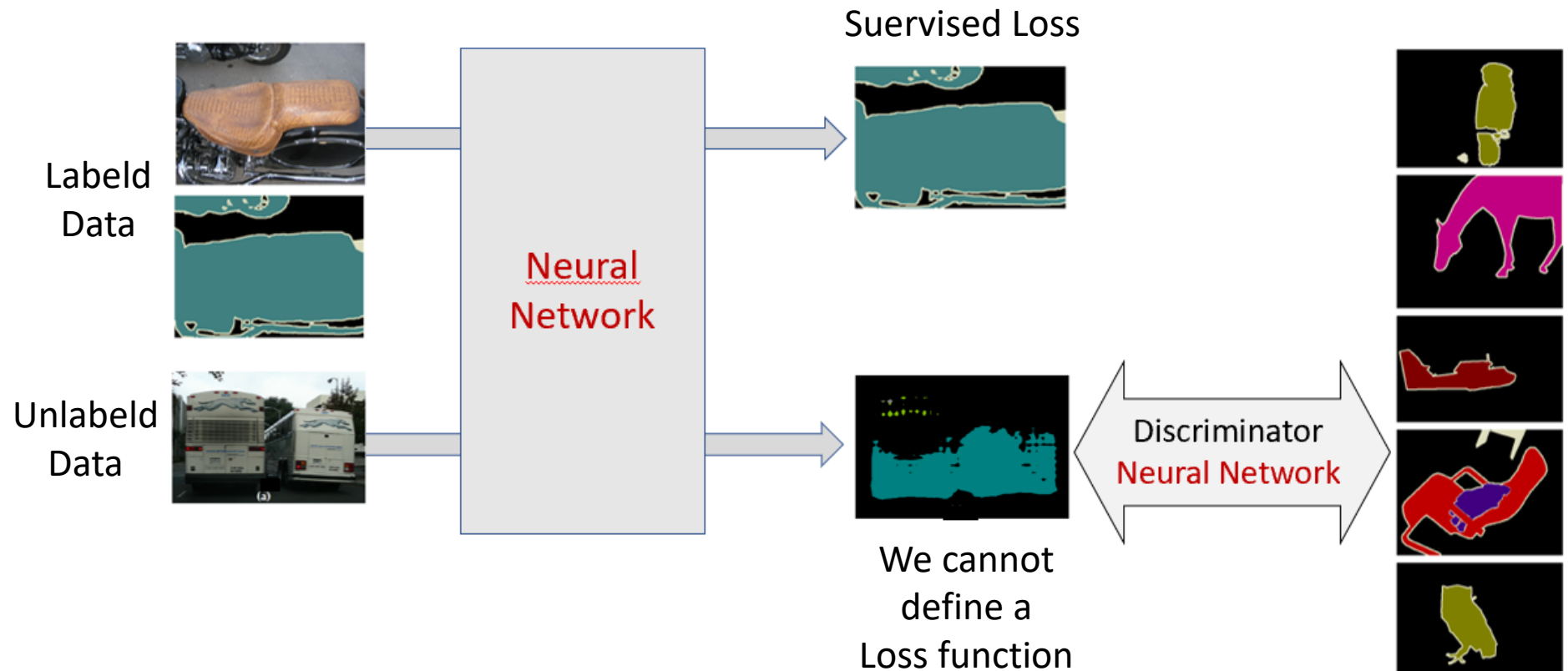
[Semi-Supervised Semantic Segmentation with High- and Low-level Consistency. S. Mittal, et al. PAMI 2021]

Semi-Supervised Learning



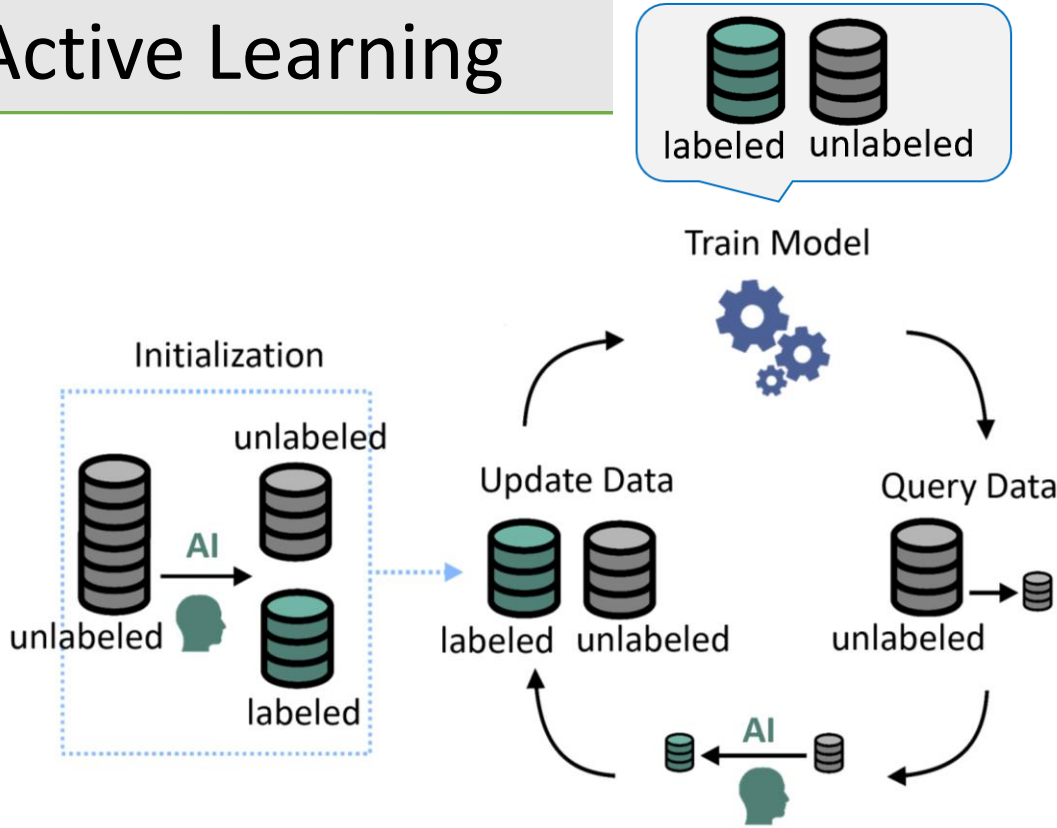
[Semi-Supervised Semantic Segmentation with High- and Low-level Consistency. S. Mittal, et al. PAMI 2021]

Semi-Supervised Learning



[Semi-Supervised Semantic Segmentation with High- and Low-level Consistency. S. Mittal, et al. PAMI 2021]

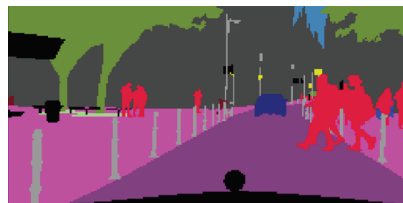
Active Learning



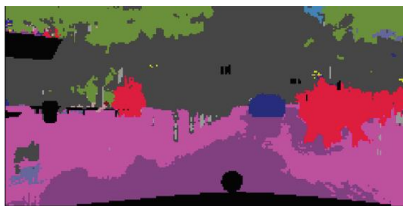
- Random
- Uncertainty
- Core Set



Query image



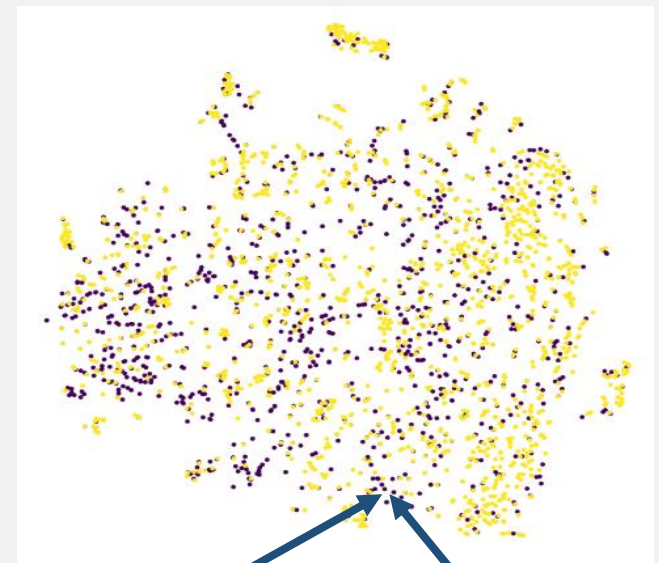
Ground Truth



Initial Result



Uncertainty



Active Learning - Results



A2D2

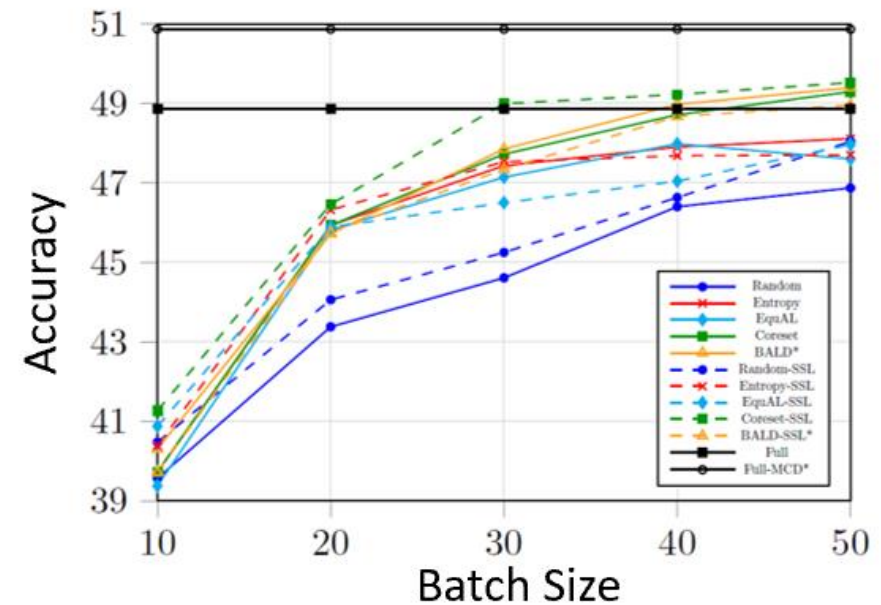
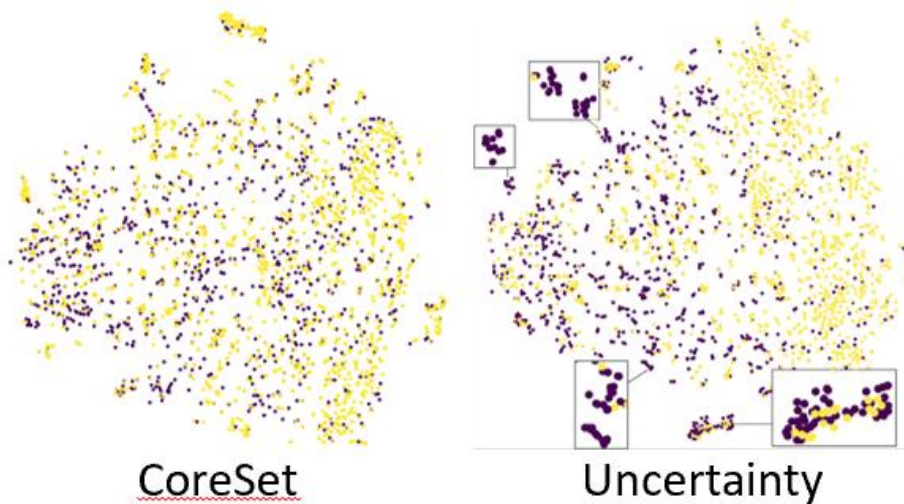
- **Redundant Dataset:**
CoreSet + Semi Sup. Learning

Active Learning - Results



A2D2

- **Redundant Dataset:**
CoreSet + Semi Sup. Learning

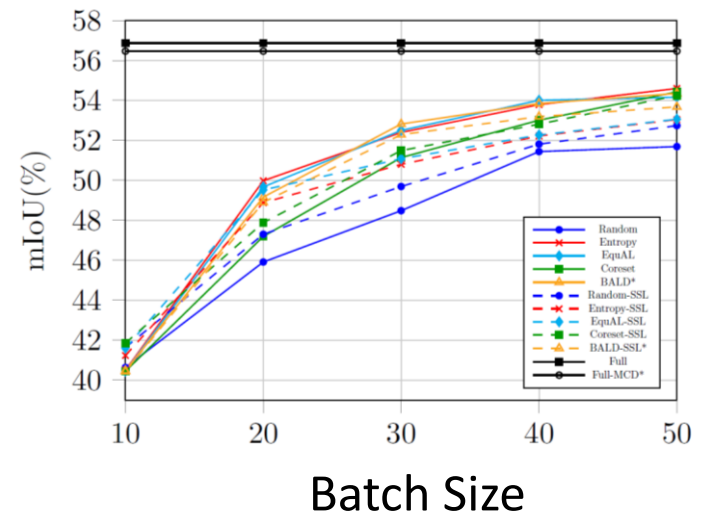
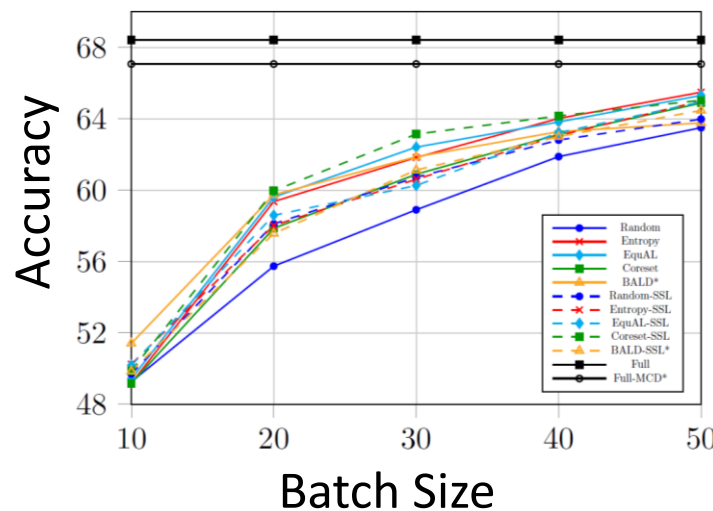


Active Learning - Results



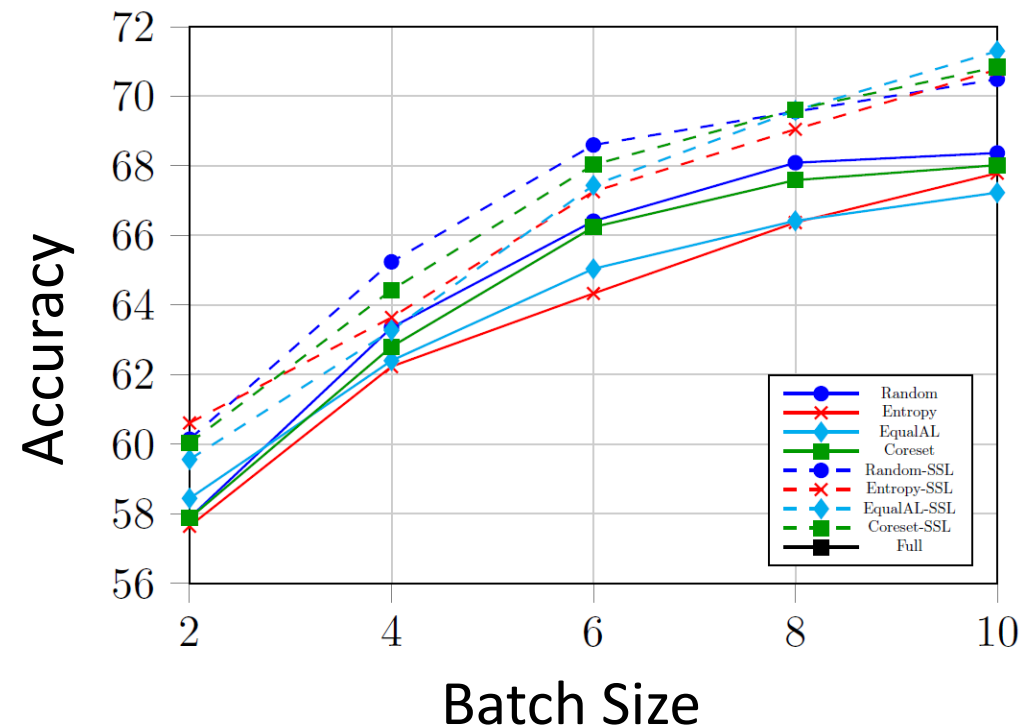
Cityscapes

- **Redundant Dataset:**
CoreSet + Semi Sup. Learning
- **Diverse Dataset:**
No clear recommendation (CoreSet + Semi Sup. Learning)



Active Learning - Results

- **Redundant Dataset:**
CoreSet + Semi Sup. Learning
- **Diverse Dataset:**
No clear recommendation
(CoreSet + Semi Sup. Learning)
- **Very little annotation budget:**
Random can even perform best



- Types of Machine Learning Problems
- Decision Trees
- Deep Learning: Discriminative Models
 - Neural Networks
 - Convolutional Neural Networks
- Image Classification: ResNet
- Understanding Image Classification Networks
- Semantic Segmentation: U-Net
- Active and Semi-Supervised Learning